

Contents

Python Handbook	2
Part I: Foundations of Python	2
1. Core Language Features	2
Variables, Types, and Scoping	2
Data Structures: Lists, Tuples, Sets, and Dictionaries	4
Control Flow: Conditionals, Loops, and Exceptions	8
List, Dictionary, and Set Comprehensions	14
Common Built-ins: <code>print</code> , <code>zip</code> , <code>sorted</code> , <code>enumerate</code> , <code>any</code> , <code>all</code> , <code>sum</code> , etc.	17
Arbitrary Precision Arithmetic	28
2. Functions and Functional Programming	32
Defining and Calling Functions	32
<code>map</code> , <code>filter</code> , <code>reduce</code> , and <code>lambda</code>	38
<code>functools</code> : <code>partial</code> , <code>lru_cache</code> , and <code>reduce</code>	44
<code>itertools</code> : <code>chain</code> , <code>combinations</code> , <code>groupby</code> , etc.	49
3. Object-Oriented Programming	56
Classes, Inheritance, and <code>super()</code>	56
Special Methods: <code>__init__</code> , <code>__str__</code> , <code>__repr__</code> , etc.	63
Properties and Descriptors	73
<code>@classmethod</code> , <code>@staticmethod</code> , and <code>@property</code>	80
Part II: Intermediate Python	88
4. Advanced Functional Programming	88
Decorators: Function and Class Decorators	88
Context Managers: <code>__enter__</code> and <code>__exit__</code> , <code>contextlib</code> , and <code>contextmanager</code>	97
Generators and Iterator Protocols	107
5. Data Classes and Validation	118
<code>@dataclass</code> , <code>field</code> , and Default Values	118
Pydantic.BaseModel for Data Validation	125
6. Concurrency and Parallelism	131
<code>threading</code> and <code>multiprocessing</code>	131
<code>concurrent.futures</code> API	143
Comparison of Concurrency Models	147
7. Type Hints and Annotations	151
Basic Type Annotations	152
<code>List</code> , <code>Dict</code> , <code>Union</code> , <code>Optional</code> , <code>TypeVar</code> , <code>Callable</code>	153
Type Checking with <code>mypy</code>	156
Part III: Advanced Python	159
8. Asynchronous Programming	159
<code>async def</code> , <code>await</code> , <code>async for</code> , and <code>async with</code>	159
Asynchronous Context Managers and Generators	163
Additional Asynchronous Patterns	168
9. CPython Internals	176
Bytecode and the Python Virtual Machine	177

Python's Object Model: Everything is an Object	179
Memory Management and Garbage Collection	180
Dictionary Implementation	182
List Implementation	184
Integer Implementation and Arbitrary Precision	185
The Global Interpreter Lock (GIL)	187
Extension Modules and the C API	189
Conclusion: Implications for Python Programmers	192
10. Metaprogramming and Design Patterns	192
Dunder Methods: <code>__init__</code> , <code>__repr__</code> , <code>__eq__</code> , <code>__lt__</code> , <code>__hash__</code> , <code>__getitem__</code> , etc.	192
Design Patterns: Singleton, Factory, Builder, Adapter, Proxy, Command, Template Method	206
Metaprogramming with <code>type()</code> , Dynamic Class Creation, <code>__new__</code> , Metaclasses, and <code>__metaclass__</code>	221
Descriptors: <code>__get__</code> , <code>__set__</code> , <code>__delete__</code> , and Use Cases . . .	236
11. Memory Optimization and Extensions	242
<code>__slots__</code> for Memory Optimization	242
Monkey Patching and Dynamic Attributes	247
C Extensions and Interfacing with C using <code>ctypes/cffi</code>	250
12. Profiling, Optimization, and Packaging	258
Profiling with <code>cProfile</code> , <code>line_profiler</code> , and Memory Profiling	258
Identifying and Fixing Bottlenecks	263
Packaging and Distributing Modules	271
13. New Python Features	283
Python 3.10 to 3.13 Features	283

Python Handbook

Part I: Foundations of Python

1. Core Language Features

Variables, Types, and Scoping

Python is a dynamically typed language where variables are created through assignment statements. Unlike statically typed languages, Python variables do not need explicit type declarations. When a value is assigned to a variable, Python automatically determines its type.

```
# Variable assignment examples
name = "Alice"           # String
age = 30                 # Integer
height = 5.9            # Float
is_student = True       # Boolean
complex_number = 3 + 4j  # Complex number
```

Python's type system is strong but dynamic, meaning once a variable has a value, its type is determined, and operations that violate type constraints will raise errors. However, a variable can be reassigned to a value of a different type at any point.

```
# Dynamic typing demonstration
x = 10           # x is now an integer
print(x + 5)     # Outputs: 15

x = "hello"      # x is now a string
print(x + " world") # Outputs: "hello world"

# Type errors still occur with invalid operations
y = "50"
# print(y + 10) # This would raise TypeError: can only concatenate str (not "int") to str
```

In Python, each variable has a scope that determines where that variable can be accessed. Python has four main scope levels:

1. **Local scope:** Variables defined within a function
2. **Enclosing scope:** Variables in the outer function when functions are nested
3. **Global scope:** Variables defined at the top level of a module or declared global
4. **Built-in scope:** Names that are built into Python

Python follows the LEGB rule (Local, Enclosing, Global, Built-in) when resolving variable names.

```
x = "global x" # Global scope

def outer_function():
    x = "outer x" # Enclosing scope

    def inner_function():
        x = "inner x" # Local scope
        print("Inner:", x)

    inner_function()
    print("Outer:", x)

outer_function()
print("Global:", x)

# Output:
# Inner: inner x
# Outer: outer x
# Global: global x
```

To modify a global variable within a function, you must use the `global` keyword:

```
counter = 0 # Global variable

def increment():
    global counter # Declare counter as global
    counter += 1
    return counter

print(increment()) # Outputs: 1
print(counter) # Outputs: 1
```

Similarly, to modify a variable from an enclosing scope, you use the `nonlocal` keyword:

```
def outer():
    count = 0

    def inner():
        nonlocal count # Use the count variable from outer function
        count += 1
        return count

    return inner

increment = outer()
print(increment()) # Outputs: 1
print(increment()) # Outputs: 2
```

Data Structures: Lists, Tuples, Sets, and Dictionaries

Python provides several built-in data structures that serve different purposes. Understanding when to use each is fundamental to writing efficient Python code.

Lists Lists are ordered, mutable collections that can contain items of different types. They are created using square brackets or the `list()` constructor.

```
# Creating lists
empty_list = []
numbers = [1, 2, 3, 4, 5]
mixed = [1, "hello", 3.14, True]
nested = [1, [2, 3], [4, [5, 6]]]

# List from other iterables
characters = list("python") # ['p', 'y', 't', 'h', 'o', 'n']

# Accessing elements (zero-indexed)
print(numbers[0]) # Outputs: 1
```

```

print(numbers[-1])      # Outputs: 5 (negative indexing starts from the end)

# Slicing: list[start:stop:step]
print(numbers[1:4])     # Outputs: [2, 3, 4]
print(numbers[:2])      # Outputs: [1, 3, 5]
print(numbers[::-1])    # Outputs: [5, 4, 3, 2, 1] (reverse)

# Common operations
numbers.append(6)        # Add to the end
numbers.insert(0, 0)     # Insert at index 0
numbers.extend([7, 8])   # Add multiple items
popped = numbers.pop()   # Remove and return last item
numbers.remove(3)        # Remove first occurrence of 3
del numbers[0]           # Delete item at index 0
numbers.clear()          # Remove all items

# Checking membership
print(3 in [1, 2, 3])    # Outputs: True

# List methods
nums = [3, 1, 4, 1, 5, 9]
print(len(nums))         # Outputs: 6
print(nums.count(1))     # Outputs: 2 (occurrences of 1)
print(nums.index(5))     # Outputs: 4 (index of first 5)
nums.sort()              # In-place sort
nums.reverse()           # In-place reverse

```

Lists are implemented as dynamic arrays, providing $O(1)$ access time by index, but $O(n)$ time for insertions and deletions at arbitrary positions (except at the end of the list).

Tuples Tuples are similar to lists but are immutable, meaning they cannot be modified after creation. They are created using parentheses or the `tuple()` constructor.

```

# Creating tuples
empty_tuple = ()
single_item = (1,) # Note the comma for single-item tuples
coordinates = (10, 20)
mixed_tuple = (1, "hello", 3.14)
nested_tuple = (1, (2, 3), (4, (5, 6)))

# Tuple from other iterables
vowels = tuple("aeiou") # ('a', 'e', 'i', 'o', 'u')

# Accessing elements (same as lists)

```

```

print(coordinates[0])    # Outputs: 10
print(coordinates[-1])  # Outputs: 20

# Tuple packing and unpacking
point = 3, 4            # Packing
x, y = point            # Unpacking
print(x, y)            # Outputs: 3 4

# Multiple assignment
a, b, c = 1, 2, 3

# Extended unpacking (Python 3.x)
first, *rest = [1, 2, 3, 4, 5]
print(first, rest)      # Outputs: 1 [2, 3, 4, 5]

# Common operations
print(len(coordinates)) # Outputs: 2
print(coordinates.count(10)) # Outputs: 1
print(coordinates.index(20)) # Outputs: 1
print(10 in coordinates) # Outputs: True

# Immutability demonstration
# coordinates[0] = 30    # This would raise TypeError

```

Tuples are generally more memory-efficient than lists and can be used as dictionary keys or elements of sets, which lists cannot. They are ideal for representing fixed collections like coordinates, RGB values, or database records.

Sets Sets are unordered collections of unique elements. They support mathematical set operations and are created using curly braces or the `set()` constructor.

```

# Creating sets
empty_set = set() # Not {}, which creates an empty dictionary
fruits = {"apple", "banana", "cherry"}
unique_digits = set([1, 2, 3, 1, 2]) # {1, 2, 3}

# Common operations
fruits.add("orange") # Add a single element
fruits.update(["mango", "grape"]) # Add multiple elements
fruits.remove("banana") # Remove (raises KeyError if not found)
fruits.discard("kiwi") # Remove if present, no error if not found
popped = fruits.pop() # Remove and return an arbitrary element
fruits.clear() # Remove all elements

# Set operations

```

```

s1 = {1, 2, 3}
s2 = {3, 4, 5}

print(s1.union(s2))           # {1, 2, 3, 4, 5} or s1 | s2
print(s1.intersection(s2))   # {3} or s1 & s2
print(s1.difference(s2))     # {1, 2} or s1 - s2
print(s1.symmetric_difference(s2)) # {1, 2, 4, 5} or s1 ^ s2

# Membership and subset tests
print(3 in s1)                # Outputs: True
print(s1.issubset({1, 2, 3, 4})) # Outputs: True
print(s1.issuperset({1, 2}))    # Outputs: True
print(s1.isdisjoint({6, 7}))    # Outputs: True (no elements in common)

```

Sets are implemented using hash tables, providing $O(1)$ average time complexity for add, remove, and membership testing operations. They are ideal for removing duplicates from sequences and for membership testing when order doesn't matter.

Dictionaries Dictionaries are unordered collections of key-value pairs. They are created using curly braces with key-value pairs or the `dict()` constructor.

```

# Creating dictionaries
empty_dict = {}
person = {"name": "Alice", "age": 30, "city": "New York"}
grades = dict(Alice=90, Bob=85, Charlie=95)

# Alternative creation methods
items = [("name", "Bob"), ("age", 25)]
person2 = dict(items)

# Accessing elements
print(person["name"])        # Outputs: Alice
print(person.get("age"))     # Outputs: 30
print(person.get("job", "Unknown")) # Outputs: Unknown (default if key not found)

# Adding or updating elements
person["job"] = "Engineer"
person.update({"email": "alice@example.com", "phone": "555-1234"})

# Removing elements
del person["age"]
job = person.pop("job")      # Remove and return value
item = person.popitem()     # Remove and return (key, value) pair
person.clear()              # Remove all items

# Dictionary methods

```

```

keys = person.keys()      # Returns view of keys
values = person.values()  # Returns view of values
items = person.items()    # Returns view of (key, value) pairs

# Dictionary comprehension
squares = {x: x*x for x in range(6)} # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# Membership testing (checks keys)
print("name" in person)    # Outputs: True

# Default dictionaries
from collections import defaultdict
word_counts = defaultdict(int)
for word in ["apple", "banana", "apple", "cherry"]:
    word_counts[word] += 1
print(dict(word_counts))   # Outputs: {'apple': 2, 'banana': 1, 'cherry': 1}

# Ordered dictionaries (preserve insertion order)
from collections import OrderedDict
od = OrderedDict([('a', 1), ('b', 2), ('c', 3)])

# Counter (specialized dictionary for counting)
from collections import Counter
c = Counter("mississippi")
print(c) # Outputs: Counter({'i': 4, 's': 4, 'p': 2, 'm': 1})
print(c.most_common(2)) # Outputs: [('i', 4), ('s', 4)]

```

Dictionaries are implemented as hash tables, providing $O(1)$ average time complexity for insertion, deletion, and lookup operations. They are one of Python's most versatile data structures and are used extensively for representing structured data, configuration settings, caches, and counting.

Control Flow: Conditionals, Loops, and Exceptions

Control flow structures enable the program to make decisions, repeat actions, and handle errors. Python provides clean, readable syntax for these critical programming constructs.

Conditional Statements Conditional statements allow the program to execute different code blocks based on whether certain conditions are true or false.

```

# Basic if statement
age = 20
if age >= 18:
    print("You are an adult")

```



```

# if-else statement
if age >= 18:
    print("You are an adult")
else:
    print("You are a minor")

# if-elif-else statement
if age < 13:
    print("Child")
elif age < 18:
    print("Teenager")
elif age < 65:
    print("Adult")
else:
    print("Senior")

# Conditional expressions (ternary operator)
status = "adult" if age >= 18 else "minor"
print(status) # Outputs: adult

# Truthiness in Python
# The following values are considered False:
# - False
# - None
# - Zero (0, 0.0, 0j)
# - Empty sequences and collections: "", [], (), {}, set()
# Everything else is considered True

# Examples
name = ""
if not name: # Checks if name is empty
    print("Name is empty")

numbers = [1, 2, 3]
if numbers: # Checks if list is not empty
    print("List has elements")

# Combining conditions with logical operators
user_age = 22
has_id = True

if user_age >= 21 and has_id:
    print("Can enter the venue and drink")
elif user_age >= 18 and has_id:
    print("Can enter the venue but cannot drink")
else:

```

```

    print("Cannot enter the venue")

# Short-circuit evaluation
# In logical operations, Python stops evaluation as soon as the result is determined
# - 'and' returns the first False value or the last value if all are True
# - 'or' returns the first True value or the last value if all are False

print(0 and 5)      # Outputs: 0 (first False value)
print(2 and 5)      # Outputs: 5 (last value since all are True)
print(0 or 5)       # Outputs: 5 (first True value)
print(0 or False)   # Outputs: False (last value since all are False)

# This can be used for providing default values or short conditionals
name = user_input or "Guest" # Use "Guest" if user_input is empty or False

```

Conditional statements in Python use indentation to define blocks of code, which makes the code cleaner and more readable compared to languages that use braces or other delimiters.

Loops Loops allow the program to execute a block of code multiple times. Python provides two main loop constructs: **for** loops and **while** loops.

```

# For loops iterate over a sequence (list, tuple, string, etc.)
fruits = ["apple", "banana", "cherry"]
for fruit in fruits:
    print(fruit)

# Range function for numeric loops
for i in range(5): # 0, 1, 2, 3, 4
    print(i)

# range(start, stop, step)
for i in range(2, 10, 2): # 2, 4, 6, 8
    print(i)

# Enumerate for index and value
for index, value in enumerate(fruits):
    print(f"Index {index}: {value}")

# Loop with dictionary
person = {"name": "Alice", "age": 30, "city": "New York"}
for key in person:
    print(f"{key}: {person[key]}")

# Or more explicitly
for key, value in person.items():

```

```

    print(f"{key}: {value}")

# While loops continue until a condition becomes False
count = 0
while count < 5:
    print(count)
    count += 1

# Infinite loop with break
while True:
    user_input = input("Enter 'quit' to exit: ")
    if user_input == "quit":
        break
    print(f"You entered: {user_input}")

# Using continue to skip the rest of the current iteration
for num in range(10):
    if num % 2 == 0:
        continue # Skip even numbers
    print(num) # Will print 1, 3, 5, 7, 9

# else clause in loops (executes when loop completes normally, not with break)
for num in range(5):
    print(num)
else:
    print("Loop completed normally")

# vs. a loop that breaks
for num in range(5):
    if num == 3:
        break
    print(num)
else:
    print("This won't be printed because the loop was broken")

# Nested loops
for i in range(3):
    for j in range(2):
        print(f"i={i}, j={j}")

```

In Python, all loops can include an optional **else** clause that executes when the loop completes normally (not when terminated by a **break** statement). This is a unique feature not found in many other programming languages.

Exceptions Exception handling allows a program to respond to exceptional situations and continue execution rather than terminating abruptly. Python

uses try, except, else, and finally blocks for exception handling.

```
# Basic try-except
try:
    result = 10 / 0 # This will raise a ZeroDivisionError
except ZeroDivisionError:
    print("Error: Division by zero")

# Handling multiple exception types
try:
    number = int("abc") # This will raise a ValueError
except ValueError:
    print("Error: Invalid number format")
except ZeroDivisionError:
    print("Error: Division by zero")

# Capturing the exception object
try:
    with open("nonexistent_file.txt", "r") as file:
        content = file.read()
except FileNotFoundError as e:
    print(f"Error: {e}")

# Generic exception handling (not recommended in most cases)
try:
    # Some code that might raise various exceptions
    result = 10 / 0
except Exception as e:
    print(f"An error occurred: {e}")

# try-except-else-finally
try:
    num = int(input("Enter a number: "))
    result = 100 / num
except ValueError:
    print("Please enter a valid number")
except ZeroDivisionError:
    print("Cannot divide by zero")
else:
    # Executed if no exception is raised
    print(f"Result: {result}")
finally:
    # Always executed, regardless of whether an exception occurred
    print("Execution completed")

# Raising exceptions
```

```

def validate_age(age):
    if not isinstance(age, int):
        raise TypeError("Age must be an integer")
    if age < 0:
        raise ValueError("Age cannot be negative")
    return age

try:
    validate_age(-5)
except (TypeError, ValueError) as e:
    print(f"Validation error: {e}")

# Creating custom exceptions
class InsufficientFundsError(Exception):
    """Raised when a withdrawal would result in a negative balance"""
    def __init__(self, balance, withdrawal_amount):
        self.balance = balance
        self.withdrawal_amount = withdrawal_amount
        self.deficit = withdrawal_amount - balance
        message = f"Cannot withdraw ${withdrawal_amount}. Balance is ${balance}, deficit: $"
        super().__init__(message)

class BankAccount:
    def __init__(self, balance=0):
        self.balance = balance

    def withdraw(self, amount):
        if amount > self.balance:
            raise InsufficientFundsError(self.balance, amount)
        self.balance -= amount
        return amount

# Using the custom exception
account = BankAccount(100)
try:
    account.withdraw(150)
except InsufficientFundsError as e:
    print(e)
    print(f"Deficit: ${e.deficit}")

```

Python's exception handling is an essential tool for writing robust code. By properly handling exceptions, you can make your programs more resilient and provide better feedback when errors occur.

List, Dictionary, and Set Comprehensions

Comprehensions are concise ways to create lists, dictionaries, and sets from existing iterables. They often lead to more readable and efficient code compared to traditional loops.

List Comprehensions List comprehensions provide a compact way to create lists based on existing lists or other iterables.

```
# Basic syntax: [expression for item in iterable]

# Creating a list of squares
squares = [x**2 for x in range(10)]
print(squares) # [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# Equivalent using a for loop
squares_loop = []
for x in range(10):
    squares_loop.append(x**2)

# With a conditional filter
even_squares = [x**2 for x in range(10) if x % 2 == 0]
print(even_squares) # [0, 4, 16, 36, 64]

# Nested list comprehension
matrix = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
flattened = [num for row in matrix for num in row]
print(flattened) # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Equivalent nested for loops
flattened_loop = []
for row in matrix:
    for num in row:
        flattened_loop.append(num)

# List comprehension with multiple conditions
filtered = [x for x in range(100) if x % 2 == 0 if x % 5 == 0]
print(filtered) # [0, 10, 20, 30, 40, 50, 60, 70, 80, 90]

# List comprehension with if-else
parity = ["even" if x % 2 == 0 else "odd" for x in range(5)]
print(parity) # ['even', 'odd', 'even', 'odd', 'even']

# List comprehension with strings
words = ["apple", "banana", "cherry", "date"]
lengths = [len(word) for word in words]
```

```

print(lengths) # [5, 6, 6, 4]

# Creating a list of tuples
coordinates = [(x, y) for x in range(3) for y in range(2)]
print(coordinates) # [(0, 0), (0, 1), (1, 0), (1, 1), (2, 0), (2, 1)]

# Using functions in list comprehensions
import math
roots = [math.sqrt(x) for x in range(1, 6)]
print(roots) # [1.0, 1.4142135623730951, 1.7320508075688772, 2.0, 2.23606797749979]

```

List comprehensions are particularly useful when transforming or filtering data. They can replace many common for loop patterns and make code more declarative.

Dictionary Comprehensions Dictionary comprehensions create dictionaries using a similar syntax to list comprehensions but with key-value pairs.

```

# Basic syntax: {key_expression: value_expression for item in iterable}

# Creating a dictionary of squares
squares_dict = {x: x**2 for x in range(6)}
print(squares_dict) # {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}

# With a conditional filter
even_squares = {x: x**2 for x in range(10) if x % 2 == 0}
print(even_squares) # {0: 0, 2: 4, 4: 16, 6: 36, 8: 64}

# Converting two lists to a dictionary
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]
name_to_age = {name: age for name, age in zip(names, ages)}
print(name_to_age) # {'Alice': 25, 'Bob': 30, 'Charlie': 35}

# Swapping keys and values in a dictionary
original = {"a": 1, "b": 2, "c": 3}
inverted = {value: key for key, value in original.items()}
print(inverted) # {1: 'a', 2: 'b', 3: 'c'}

# Creating a dictionary from another with modifications
prices = {"apple": 0.5, "banana": 0.25, "cherry": 0.75}
sale_prices = {fruit: price * 0.8 for fruit, price in prices.items()}
print(sale_prices) # {'apple': 0.4, 'banana': 0.2, 'cherry': 0.6}

# Filtering a dictionary
expensive = {fruit: price for fruit, price in prices.items() if price > 0.3}

```

```

print(expensive) # {'apple': 0.5, 'cherry': 0.75}

# Dictionary comprehension with conditional values
stock_status = {
    fruit: "available" if price < 0.5 else "premium"
    for fruit, price in prices.items()
}
print(stock_status) # {'apple': 'premium', 'banana': 'available', 'cherry': 'premium'}

# Creating a frequency counter
text = "hello world"
char_count = {char: text.count(char) for char in set(text)}
print(char_count) # {'h': 1, 'e': 1, 'l': 3, 'o': 2, ' ': 1, 'w': 1, 'r': 1, 'd': 1}

```

Dictionary comprehensions are excellent for transforming, filtering, or creating dictionaries on the fly. They're often used for data mapping and transformation tasks.

Set Comprehensions Set comprehensions create sets using a similar syntax to list comprehensions but with curly braces.

```

# Basic syntax: {expression for item in iterable}

# Creating a set of squares
squares_set = {x**2 for x in range(10)}
print(squares_set) # {0, 1, 4, 9, 16, 25, 36, 49, 64, 81}

# With a conditional filter
even_squares = {x**2 for x in range(10) if x % 2 == 0}
print(even_squares) # {0, 4, 16, 36, 64}

# Creating a set of unique characters from a string
text = "mississippi"
unique_chars = {char for char in text}
print(unique_chars) # {'m', 'i', 's', 'p'}

# Creating a set of words from a sentence (with processing)
sentence = "The quick brown fox jumps over the lazy dog"
unique_words = {word.lower() for word in sentence.split()}
print(unique_words) # {'the', 'quick', 'brown', 'fox', 'jumps', 'over', 'lazy', 'dog'}

# Creating a set of tuples
pairs = {(x, y) for x in range(2) for y in range(2)}
print(pairs) # {(0, 0), (0, 1), (1, 0), (1, 1)}

# Creating a set of divisors

```



```
divisors_of_100 = {i for i in range(1, 101) if 100 % i == 0}
print(divisors_of_100)  # {1, 2, 4, 5, 10, 20, 25, 50, 100}
```

```
# Finding common elements from two lists
list1 = [1, 2, 3, 4, 5]
list2 = [3, 4, 5, 6, 7]
common = {x for x in list1 if x in list2}
print(common)  # {3, 4, 5}
```

Set comprehensions are particularly useful when you need to filter duplicate values or perform set operations.

Comprehensions are powerful tools in Python that can make your code more concise and readable. However, for very complex operations, traditional loops may sometimes be more appropriate for clarity and maintainability.

Common Built-ins: print, zip, sorted, enumerate, any, all, sum, etc.

Python provides numerous built-in functions that simplify common programming tasks. Understanding these functions is essential for writing efficient and pythonic code.

print() The `print()` function outputs text to the console. It's highly versatile and configurable.

```
# Basic usage
print("Hello, World!")  # Hello, World!

# Multiple arguments
print("Hello", "World", "!")  # Hello World !

# Specifying separators and end characters
print("Hello", "World", sep="-", end="!\n")  # Hello-World!

# Using format strings
name = "Alice"
age = 30
print(f"{name} is {age} years old")  # Alice is 30 years old

# File output
with open("output.txt", "w") as f:
    print("This goes to the file", file=f)

# Suppressing newline
print("Hello", end=" ")
print("World")  # Hello World
```

zip() The `zip()` function combines multiple iterables into a single iterable of tuples.

```
# Basic usage
names = ["Alice", "Bob", "Charlie"]
ages = [25, 30, 35]
zipped = zip(names, ages)
print(list(zipped))  # [('Alice', 25), ('Bob', 30), ('Charlie', 35)]

# Unequal length iterables
numbers = [1, 2, 3, 4]
letters = ['a', 'b', 'c']
print(list(zip(numbers, letters)))  # [(1, 'a'), (2, 'b'), (3, 'c')]

# Using zip with dictionaries
data = dict(zip(names, ages))
print(data)  # {'Alice': 25, 'Bob': 30, 'Charlie': 35}

# Unzipping a zipped sequence
pairs = [('Alice', 25), ('Bob', 30), ('Charlie', 35)]
names, ages = zip(*pairs)
print(names)  # ('Alice', 'Bob', 'Charlie')
print(ages)   # (25, 30, 35)

# Using zip in a for loop
for name, age in zip(names, ages):
    print(f"{name} is {age} years old")

# Using zip_longest from itertools (fill missing values)
from itertools import zip_longest
numbers = [1, 2, 3]
letters = ['a', 'b', 'c', 'd', 'e']
print(list(zip_longest(numbers, letters, fillvalue=0)))
# [(1, 'a'), (2, 'b'), (3, 'c'), (0, 'd'), (0, 'e')]
```

sorted() The `sorted()` function returns a new sorted list from an iterable.

```
# Basic usage
numbers = [3, 1, 4, 1, 5, 9, 2]
print(sorted(numbers))  # [1, 1, 2, 3, 4, 5, 9]

# Descending order
print(sorted(numbers, reverse=True))  # [9, 5, 4, 3, 2, 1, 1]

# Sorting strings (case-sensitive)
names = ["alice", "Bob", "charlie", "David"]
```

```

print(sorted(names)) # ['Bob', 'David', 'alice', 'charlie'] (uppercase comes before lowercase)

# Case-insensitive sorting
print(sorted(names, key=str.lower)) # ['alice', 'Bob', 'charlie', 'David']

# Sorting with a custom key
students = [
    {"name": "Alice", "grade": 85},
    {"name": "Bob", "grade": 92},
    {"name": "Charlie", "grade": 78}
]

# Sort by grade
print(sorted(students, key=lambda student: student["grade"]))
# [{'name': 'Charlie', 'grade': 78}, {'name': 'Alice', 'grade': 85}, {'name': 'Bob', 'grade': 92}]

# Sort by name
print(sorted(students, key=lambda student: student["name"]))
# [{'name': 'Alice', 'grade': 85}, {'name': 'Bob', 'grade': 92}, {'name': 'Charlie', 'grade': 78}]

# Multiple level sorting with itemgetter
from operator import itemgetter
inventory = [
    {"name": "apple", "price": 0.5, "stock": 10},
    {"name": "banana", "price": 0.25, "stock": 5},
    {"name": "orange", "price": 0.5, "stock": 7}
]

# Sort by price, then by stock
print(sorted(inventory, key=itemgetter("price", "stock")))
# [{'name': 'banana', 'price': 0.25, 'stock': 5}, {'name': 'orange', 'price': 0.5, 'stock': 7}, {'name': 'apple', 'price': 0.5, 'stock': 10}]

# Sorting complex objects
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __repr__(self):
        return f"Person('{self.name}', {self.age})"

people = [Person("Alice", 30), Person("Bob", 25), Person("Charlie", 35)]
print(sorted(people, key=lambda p: p.age))
# [Person('Bob', 25), Person('Alice', 30), Person('Charlie', 35)]

```

The `sorted()` function is stable, meaning that elements with equal keys will

maintain their relative order. It's a powerful tool for data manipulation and analysis.

enumerate() The `enumerate()` function adds a counter to an iterable, returning an enumerate object that yields pairs of index and value.

```
# Basic usage
fruits = ["apple", "banana", "cherry"]
for index, fruit in enumerate(fruits):
    print(f"{index}: {fruit}")
# 0: apple
# 1: banana
# 2: cherry

# Starting from a different number
for index, fruit in enumerate(fruits, start=1):
    print(f"{index}: {fruit}")
# 1: apple
# 2: banana
# 3: cherry

# Creating a dictionary with enumerate
fruit_indices = {fruit: index for index, fruit in enumerate(fruits)}
print(fruit_indices) # {'apple': 0, 'banana': 1, 'cherry': 2}

# Using enumerate with list comprehension
indexed_fruits = [(index, fruit) for index, fruit in enumerate(fruits)]
print(indexed_fruits) # [(0, 'apple'), (1, 'banana'), (2, 'cherry')]

# Finding positions of specific elements
positions = [index for index, fruit in enumerate(fruits) if fruit.startswith("b")]
print(positions) # [1]
```

`enumerate()` is particularly useful when you need both the index and value of items in an iterable during iteration.

any() and all() The `any()` function returns `True` if at least one element in an iterable is true. The `all()` function returns `True` if all elements in an iterable are true.

```
# any() examples
booleans = [False, False, True, False]
print(any(booleans)) # True

numbers = [0, 0, 5, 0]
print(any(numbers)) # True (5 is truthy)
```

```

empty = []
print(any(empty))      # False (empty iterable)

# all() examples
all_true = [True, True, True]
print(all(all_true))   # True

mixed = [True, True, False]
print(all(mixed))      # False

numbers = [1, 2, 3, 4, 5]
print(all(numbers))    # True (all numbers are truthy)

with_zero = [1, 2, 0, 4, 5]
print(all(with_zero))  # False (0 is falsy)

empty = []
print(all(empty))      # True (empty iterable)

# Practical examples
# Check if all numbers are positive
def all_positive(numbers):
    return all(num > 0 for num in numbers)

print(all_positive([1, 2, 3]))    # True
print(all_positive([1, -2, 3]))   # False

# Check if any number is even
def has_even(numbers):
    return any(num % 2 == 0 for num in numbers)

print(has_even([1, 2, 3]))    # True
print(has_even([1, 3, 5]))    # False

# Check if all strings match a pattern
import re
emails = ["user@example.com", "invalid.email", "another@domain.org"]
pattern = r"^[a-zA-Z0-9_.+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+$"
all_valid = all(re.match(pattern, email) for email in emails)
print(all_valid)              # False

any_valid = any(re.match(pattern, email) for email in emails)
print(any_valid)              # True

```

The `any()` and `all()` functions are useful for testing conditions across collections and can make code more readable when used with generator expressions.

sum() The `sum()` function adds all items in an iterable and returns the sum.

```
# Basic usage
numbers = [1, 2, 3, 4, 5]
print(sum(numbers)) # 15

# With start value (default is 0)
print(sum(numbers, 10)) # 25 (10 + 1 + 2 + 3 + 4 + 5)

# With generator expression
print(sum(x * x for x in range(5))) # 30 (02 + 12 + 22 + 32 + 42)

# Float sums (be aware of floating-point precision issues)
floats = [0.1, 0.2, 0.3]
print(sum(floats)) # 0.6000000000000001

# Summing booleans (True counts as 1, False as 0)
booleans = [True, False, True, True]
print(sum(booleans)) # 3

# Alternative for string concatenation (avoid sum for this)
strings = ["Hello", " ", "World"]
# print(sum(strings)) # TypeError
print("".join(strings)) # "Hello World"

# Summing complex data
transactions = [
    {"amount": 100, "type": "deposit"},
    {"amount": 50, "type": "withdrawal"},
    {"amount": 75, "type": "deposit"}
]

total_deposits = sum(t["amount"] for t in transactions if t["type"] == "deposit")
print(total_deposits) # 175

# Using sum with zip
prices = [10, 20, 30]
quantities = [2, 3, 1]
total_cost = sum(p * q for p, q in zip(prices, quantities))
print(total_cost) # 20 + 60 + 30 = 110
```

The `sum()` function is a versatile tool for numerical calculations and aggregations.

min() and max() The `min()` and `max()` functions return the smallest and largest items in an iterable, respectively.

```

# Basic usage
numbers = [4, 2, 9, 1, 5]
print(min(numbers)) # 1
print(max(numbers)) # 9

# With strings (lexicographic comparison)
words = ["apple", "banana", "cherry"]
print(min(words)) # "apple"
print(max(words)) # "cherry"

# With a key function
print(min(words, key=len)) # "apple"
print(max(words, key=len)) # "banana" (tied with "cherry", but comes first)

# With named arguments
print(min(5, 10, 3, 8)) # 3
print(max(5, 10, 3, 8)) # 10

# Default for empty sequence
print(min([], default=0)) # 0
# print(min([])) # ValueError: min() arg is an empty sequence

# With dictionaries (compares keys by default)
scores = {"Alice": 85, "Bob": 92, "Charlie": 78}
print(min(scores)) # "Alice"
print(max(scores)) # "Charlie"

# Getting min/max values from a dictionary
print(min(scores.values())) # 78
print(max(scores.values())) # 92

# Getting key with min/max value
min_student = min(scores, key=scores.get)
max_student = max(scores, key=scores.get)
print(f"{min_student}: {scores[min_student]}") # Charlie: 78
print(f"{max_student}: {scores[max_student]}") # Bob: 92

# Finding the student with the longest name
longest_name = max(scores, key=len)
print(longest_name) # "Charlie"

# Getting multiple maximums
def get_max_values(dictionary):
    max_value = max(dictionary.values())
    return {k: v for k, v in dictionary.items() if v == max_value}

```

```
test_scores = {"Alice": 95, "Bob": 92, "Charlie": 95, "Dave": 88}
top_students = get_max_values(test_scores)
print(top_students)  # {'Alice': 95, 'Charlie': 95}
```

The `min()` and `max()` functions are essential for finding extremes in data collections and can be customized with the `key` parameter for complex data structures.

len() The `len()` function returns the number of items in an object.

```
# Basic usage with different types
print(len("Python"))      # 6 (string)
print(len([1, 2, 3, 4]))  # 4 (list)
print(len((1, 2, 3)))      # 3 (tuple)
print(len({1, 2, 3, 3}))   # 3 (set - unique elements only)
print(len({"a": 1, "b": 2})) # 2 (dictionary - counts keys)

# With custom classes (implement __len__)
class CustomList:
    def __init__(self, items):
        self.items = items

    def __len__(self):
        return len(self.items)

custom = CustomList([1, 2, 3, 4, 5])
print(len(custom))  # 5

# Empty collections
print(len(""))      # 0
print(len([]))      # 0
print(len(set()))    # 0
print(len({}))       # 0

# Using len in conditions (checking if collection is empty)
data = []
if len(data) == 0:
    print("Data is empty")
# Or more pythonically:
if not data:
    print("Data is empty")

# Finding the length of nested structures
nested_list = [[1, 2], [3, 4, 5], [6]]
total_elements = sum(len(sublist) for sublist in nested_list)
print(total_elements)  # 6
```

The `len()` function works with any object that defines the `__len__()` method

and is commonly used to check the size of collections.

range() The `range()` function returns a sequence of numbers, commonly used in for loops.

```
# Basic usage
for i in range(5): # 0, 1, 2, 3, 4
    print(i, end=" ") # 0 1 2 3 4

# Start and stop values
for i in range(2, 6): # 2, 3, 4, 5
    print(i, end=" ") # 2 3 4 5

# With step value
for i in range(1, 10, 2): # 1, 3, 5, 7, 9
    print(i, end=" ") # 1 3 5 7 9

# Negative step for counting down
for i in range(10, 0, -1): # 10, 9, 8, ..., 1
    print(i, end=" ") # 10 9 8 7 6 5 4 3 2 1

# Creating a list from range
numbers = list(range(5))
print(numbers) # [0, 1, 2, 3, 4]

# Range objects are lazy and memory-efficient
large_range = range(1, 1000000)
print(type(large_range)) # <class 'range'>
print(large_range[0]) # 1
print(large_range[-1]) # 999999

# Checking membership
print(5 in range(10)) # True
print(10 in range(10)) # False

# Finding the length of a range
print(len(range(5))) # 5
print(len(range(1, 10, 2))) # 5

# Using range with index access
r = range(10, 20)
print(r[0]) # 10
print(r[5]) # 15

# Using range in list comprehensions
squares = [x**2 for x in range(5)]
```

```
print(squares) # [0, 1, 4, 9, 16]
```

The `range()` function is a memory-efficient way to generate sequences of numbers without storing the entire sequence in memory at once.

abs(), round(), and pow() These numeric functions perform common mathematical operations.

```
# abs() returns the absolute value
print(abs(-5))      # 5
print(abs(3.14))    # 3.14
print(abs(complex(3, 4))) # 5.0 (magnitude of complex number)

# round() rounds a number to a specified precision
print(round(3.14159)) # 3 (nearest integer)
print(round(3.14159, 2)) # 3.14 (2 decimal places)
print(round(3.5))      # 4 (rounds to even when tie)
print(round(4.5))      # 4 (rounds to even when tie)
print(round(1.5))      # 2
print(round(2.5))      # 2

# round() with negative precision
print(round(1234, -2)) # 1200 (nearest hundred)
print(round(1250, -2)) # 1200 (nearest hundred, rounds to even)
print(round(1350, -2)) # 1400

# pow() raises a number to a power
print(pow(2, 3))      # 8 (same as 2 ** 3)
print(pow(2, -1))     # 0.5 (same as 2 ** -1)
print(pow(2, 0.5))    # 1.4142135623730951 (square root of 2)

# pow() with modulo (more efficient than (x ** y) % z)
print(pow(2, 3, 5))   # 3 (2^3 % 5 = 8 % 5 = 3)
print(pow(3, 4, 7))   # 4 (3^4 % 7 = 81 % 7 = 4)
```

These functions provide essential mathematical operations and are often more readable than their operator equivalents.

chr() and ord() The `chr()` function returns a character from a Unicode code point, while `ord()` returns the Unicode code point of a character.

```
# chr() - integer to character
print(chr(65))      # 'A'
print(chr(97))      # 'a'
print(chr(8364))    # '€'

# ord() - character to integer
```

```

print(ord('A'))    # 65
print(ord('a'))    # 97
print(ord('€'))    # 8364

# Creating a range of characters
alphabet = [chr(i) for i in range(ord('A'), ord('Z')+1)]
print(alphabet)    # ['A', 'B', 'C', ..., 'Z']

# Simple Caesar cipher
def caesar_cipher(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            ascii_offset = ord('a') if char.islower() else ord('A')
            # Convert to 0-25, add shift, wrap around with %, convert back
            result += chr((ord(char) - ascii_offset + shift) % 26 + ascii_offset)
        else:
            result += char
    return result

encrypted = caesar_cipher("Hello, World!", 3)
print(encrypted)   # "Khoor, Zruog!"
decrypted = caesar_cipher(encrypted, -3)
print(decrypted)   # "Hello, World!"

```

These functions are useful for text processing, character manipulation, and working with Unicode data.

input() and eval() The `input()` function reads a line from the console, while `eval()` evaluates a string as a Python expression.

```

# Basic input
name = input("Enter your name: ") # Prompts the user and returns the input as a string
print(f"Hello, {name}!")

# Converting input to numbers manually
age = int(input("Enter your age: "))
height = float(input("Enter your height in meters: "))

# Using eval() to evaluate expressions (use with caution)
math_expr = input("Enter a math expression: ") # e.g., "3 * 4 + 2"
result = eval(math_expr)
print(f"Result: {result}") # Result: 14

# Using eval() with variables
x = 10

```

```

y = 20
expr = input("Enter an expression using x and y: ") # e.g., "x + 2*y"
result = eval(expr)
print(f"Result: {result}") # Result: 50

# Warning: eval() can execute any code, which can be dangerous
# It's generally safer to avoid eval() for user input
# Instead, use safer alternatives like ast.literal_eval() for limited parsing
import ast
safe_input = input("Enter a list or dict literal: ") # e.g., "[1, 2, 3]"
try:
    result = ast.literal_eval(safe_input)
    print(f"Parsed safely: {result}")
except (SyntaxError, ValueError):
    print("Invalid input")

# Alternative to eval() for arithmetic
from operator import add, sub, mul, truediv
operators = {'+': add, '-': sub, '*': mul, '/': truediv}

def safe_eval(expression):
    # Very simple parser for basic arithmetic
    # Real-world applications would use a proper parsing library
    tokens = expression.replace('(', ' ( ').replace(')', ' ) ').split()
    # This is a simplified example and doesn't handle all cases
    # In practice, use a dedicated library like `simpleeval`
    pass

```

While `input()` is essential for interactive programs, `eval()` should be used with caution because it can execute arbitrary code.

Arbitrary Precision Arithmetic

Python supports arbitrary precision integers and floating-point numbers, allowing calculations with very large numbers without overflow.

Integers In Python 3, integers have unlimited precision, meaning they can grow as large as needed to accurately represent the value.

```

# Large integer calculations
factorial_20 = 2432902008176640000 # 20!
print(factorial_20) # 2432902008176640000

# Computing large factorials
def factorial(n):
    result = 1
    for i in range(2, n + 1):

```



```

# Basic operations
a = Decimal('0.1')
b = Decimal('0.2')
c = a + b
print(c) # 0.3 (exact)

# Compare with float arithmetic
print(0.1 + 0.2) # 0.30000000000000004 (float imprecision)

# High-precision calculations
pi = Decimal('3.14159265358979323846264338327950')
r = Decimal('2.5')
area = pi * r ** 2
print(area) # 19.6349540849362077403311481853

# Controlling rounding
getcontext().rounding = 'ROUND_HALF_UP'
print(Decimal('1.5').quantize(Decimal('1')) # 2
print(Decimal('2.5').quantize(Decimal('1')) # 3

getcontext().rounding = 'ROUND_HALF_EVEN' # Banker's rounding
print(Decimal('1.5').quantize(Decimal('1')) # 2
print(Decimal('2.5').quantize(Decimal('1')) # 2

# Financial calculations
principal = Decimal('1000.00')
rate = Decimal('0.05') # 5% interest
periods = 10

# Compound interest formula:  $A = P(1 + r)^t$ 
amount = principal * (1 + rate) ** periods
print(f"${amount:.2f}") # $1628.89

# Creating decimals from floats (can lead to unexpected results)
print(Decimal(0.1)) # 0.1000000000000000055511151231257827021181583404541015625
# Better to create from strings for exact representation
print(Decimal('0.1')) # 0.1

# Setting exact values within context
with getcontext().copy() as ctx:
    ctx.prec = 2
    print(Decimal('1.123') + Decimal('2.456')) # 3.6

```

The `decimal` module is essential for applications requiring exact decimal representations and precise control over rounding behavior, such as financial systems.

```

from fractions import Fraction

# Creating fractions
f1 = Fraction(1, 3) # 1/3
f2 = Fraction('0.25') # 1/4
f3 = Fraction('2.5') # 5/2
print(f1, f2, f3) # 1/3 1/4 5/2

# Creating fractions from floating-point numbers (may have surprising results)
print(Fraction(0.1)) # 3602879701896397/36028797018963968
# For exact representation, use strings
print(Fraction('0.1')) # 1/10

# Arithmetic operations
sum_f = f1 + f2
print(sum_f) # 7/12
print(f1 * f3) # 5/6
print(f1 / f2) # 4/3
print(f1 ** 2) # 1/9

# Converting to other types
print(float(f1)) # 0.3333333333333333
print(Decimal(f1)) # 0.33333333333333333333333333333333

# Limiting denominator size
pi_approx = Fraction(355, 113) # Good approximation of pi
print(pi_approx) # 355/113
print(float(pi_approx)) # 3.1415929203539825

from math import pi
f_pi = Fraction(pi) # Exact fraction representation of float pi
print(f_pi) # Very large fraction
limited = f_pi.limit_denominator(1000) # Approximation with denominator <= 1000
print(limited) # 355/113

# Finding the greatest common divisor
import math
print(math.gcd(36, 48)) # 12

# Getting numerator and denominator
f = Fraction(5, 15)
print(f) # 1/3 (automatically simplified)
print(f.numerator, f.denominator) # 1 3

```

The `fractions` module is useful for applications requiring exact rational arithmetic, such as computer algebra systems and precise scientific calculations.

2. Functions and Functional Programming

Defining and Calling Functions

Functions are reusable blocks of code designed to perform specific tasks. They help organize code, reduce repetition, and improve maintainability.

Basic Function Definition and Calling

```
# Basic function definition
def greet(name):
    """Return a greeting message."""
    return f"Hello, {name}!"

# Calling the function
message = greet("Alice")
print(message)  # Hello, Alice!

# Function with multiple parameters
def add(a, b):
    """Add two numbers and return the result."""
    return a + b

result = add(3, 5)
print(result)  # 8

# Function with default parameters
def power(base, exponent=2):
    """Calculate base raised to the exponent power."""
    return base ** exponent

print(power(3))      # 9 (uses default exponent=2)
print(power(3, 3))   # 27 (overrides default)

# Functions with variable-length arguments (varargs)
def sum_all(*args):
    """Sum all arguments."""
    return sum(args)

print(sum_all(1, 2, 3, 4))  # 10

# Functions with keyword arguments
def create_profile(name, age, **kwargs):
    """Create a user profile dictionary."""
```



```

    profile = {'name': name, 'age': age}
    profile.update(kwargs)
    return profile

profile = create_profile('Alice', 30, occupation='Engineer', city='New York')
print(profile) # {'name': 'Alice', 'age': 30, 'occupation': 'Engineer', 'city': 'New York'}

# Keyword-only arguments (Python 3+)
def configure(*, host='localhost', port=8080):
    """Configure server (keyword-only arguments)."""
    return f"Server configured at {host}:{port}"

# print(configure('example.com', 9000)) # TypeError
print(configure(host='example.com', port=9000)) # Server configured at example.com:9000

# Positional-only arguments (Python 3.8+)
def divide(a, b, /):
    """Divide a by b (positional-only arguments)."""
    return a / b

print(divide(10, 2)) # 5.0
# print(divide(a=10, b=2)) # TypeError

# Mixed parameter types
def mixed_params(a, b, /, c, *, d, e):
    """
    a, b are positional-only
    c can be positional or keyword
    d, e are keyword-only
    """
    return a + b + c + d + e

print(mixed_params(1, 2, 3, d=4, e=5)) # 15
print(mixed_params(1, 2, c=3, d=4, e=5)) # 15

```

Understanding function parameters and argument passing is fundamental to effective Python programming.

Return Values Functions can return zero, one, or multiple values.

```

# No return value (returns None implicitly)
def greet_user(name):
    print(f"Hello, {name}!")

result = greet_user("Alice")
print(result) # None

```

```

# Returning a single value
def square(x):
    return x ** 2

print(square(4))  # 16

# Early returns
def absolute(x):
    if x >= 0:
        return x
    return -x # Only executed if x is negative

print(absolute(-5))  # 5

# Returning multiple values (as a tuple)
def get_coordinates():
    return 10, 20

x, y = get_coordinates()
print(x, y)  # 10 20

# Or explicitly as a tuple
def get_dimensions():
    return (100, 200)

width, height = get_dimensions()
print(width, height)  # 100 200

# Returning dictionaries
def get_user():
    return {
        'name': 'Alice',
        'age': 30,
        'city': 'New York'
    }

user = get_user()
print(user['name'])  # Alice

# Returning functions (closures)
def create_multiplier(factor):
    def multiply(x):
        return x * factor
    return multiply

```

```
double = create_multiplier(2)
triple = create_multiplier(3)
print(double(5))    # 10
print(triple(5))    # 15
```

Return values allow functions to produce results that can be used in subsequent operations.

Docstrings and Function Annotations Documenting functions helps other developers understand their purpose, parameters, and return values.

```
# Function with docstring
def calculate_area(length, width):
    """
    Calculate the area of a rectangle.

    Args:
        length (float): The length of the rectangle.
        width (float): The width of the rectangle.

    Returns:
        float: The area of the rectangle.
    """
    return length * width

# Accessing the docstring
help(calculate_area)
print(calculate_area.__doc__)

# Type annotations (Python 3.5+)
def add(a: int, b: int) -> int:
    """Add two integers and return the result."""
    return a + b

# Annotations don't enforce types, they're just hints
result = add("Hello, ", "World!") # Works, but not what annotation suggests
print(result) # "Hello, World!"

# Accessing annotations
print(add.__annotations__) # {'a': <class 'int'>, 'b': <class 'int'>, 'return': <class 'int'>}

# More complex annotations
from typing import List, Dict, Union, Optional

def process_data(items: List[int], options: Optional[Dict[str, str]] = None) -> Union[int, float]:
    """
```

Process a list of integers based on provided options.

Args:

items: A list of integers to process

options: Optional dictionary with processing options

Returns:

The processed result as either an integer or float

"""

```
if options and options.get('mode') == 'average':
```

```
    return sum(items) / len(items)
```

```
return sum(items)
```

Run-time type checking with annotations

Using a third-party library like mypy for static type checking or:

```
def enforce_types(func):
```

```
    """Decorator to enforce type annotations at runtime."""
```

```
    def wrapper(*args, **kwargs):
```

```
        # Check argument types against annotations (simplified version)
```

```
        for name, value in zip(func.__annotations__, args):
```

```
            if name != 'return' and not isinstance(value, func.__annotations__[name]):
```

```
                raise TypeError(f"Argument {name} must be {func.__annotations__[name]}")
```

```
    result = func(*args, **kwargs)
```

```
    # Check return type
```

```
    if 'return' in func.__annotations__ and not isinstance(result, func.__annotations__['return']):
```

```
        raise TypeError(f"Return value must be {func.__annotations__['return']}")
```

```
    return result
```

```
    return wrapper
```

```
@enforce_types
```

```
def multiply(a: int, b: int) -> int:
```

```
    return a * b
```

```
print(multiply(3, 4))          # 12
```

```
# print(multiply(3, "4"))     # TypeError: Argument b must be <class 'int'>
```

Docstrings and type annotations improve code readability and help catch errors early, especially when used with static type checkers like mypy.

Scope and Closures Understanding variable scope and closures is essential for effective function design and use.

Local and global scope

```
x = 10 # Global variable
```

```
def func1():
```

```

    print(x)  # Accesses global x

def func2():
    x = 20  # Creates local x, does not affect global x
    print(x)

func1()  # 10
func2()  # 20
print(x)  # 10 (global x is unchanged)

# The global keyword
def func3():
    global x  # Declares that we want to use global x
    x = 30  # Modifies global x
    print(x)

func3()  # 30
print(x)  # 30 (global x has been modified)

# Nonlocal scope (for nested functions)
def outer():
    y = 10  # Outer function's local variable

    def inner():
        print(y)  # Can access outer function's variable

    inner()  # 10

outer()

# Modifying nonlocal variables
def counter():
    count = 0  # Local to counter, but nonlocal to increment

    def increment():
        nonlocal count  # Indicates we want to modify count from counter
        count += 1
        return count

    return increment  # Returns the inner function

increment_func = counter()
print(increment_func())  # 1
print(increment_func())  # 2
print(increment_func())  # 3

```

```

# Closures: inner functions that remember the state of their enclosing scope
def create_multiplier(factor):
    # factor is part of the closure for multiply
    def multiply(x):
        return x * factor
    return multiply

double = create_multiplier(2)
triple = create_multiplier(3)
print(double(5))  # 10 (5*2)
print(triple(5)) # 15 (5*3)

# Inspecting closure variables
print(double.__closure__) # Shows the closure cells
print(double.__closure__[0].cell_contents) # 2

# Using closures for data hiding
def create_counter():
    """Create a counter with increment and get methods."""
    count = 0

    def increment():
        nonlocal count
        count += 1

    def get_count():
        return count

    return {'increment': increment, 'get_count': get_count}

counter = create_counter()
counter['increment']()
counter['increment']()
print(counter['get_count']()) # 2
# There's no direct way to access count from outside

```

Closures are functions that capture and remember the environment in which they were created. They're fundamental for functional programming patterns and creating function factories.

map, filter, reduce, and lambda

These functions enable functional programming paradigms in Python, allowing operations on sequences without explicit loops.

map() The `map()` function applies a given function to each item in an iterable and returns an iterator of results.

```
# Basic map usage
numbers = [1, 2, 3, 4, 5]
squares = map(lambda x: x**2, numbers)
print(list(squares))  # [1, 4, 9, 16, 25]

# Equivalent list comprehension
squares_comp = [x**2 for x in numbers]
print(squares_comp)  # [1, 4, 9, 16, 25]

# Using a named function
def cube(x):
    return x**3

cubes = map(cube, numbers)
print(list(cubes))  # [1, 8, 27, 64, 125]

# Multiple iterables
list1 = [1, 2, 3]
list2 = [10, 20, 30]
sums = map(lambda x, y: x + y, list1, list2)
print(list(sums))  # [11, 22, 33]

# Mapping with a built-in function
words = ["hello", "world", "python"]
lengths = map(len, words)
print(list(lengths))  # [5, 5, 6]

# Mapping objects
users = [
    {"name": "Alice", "age": 30},
    {"name": "Bob", "age": 25},
    {"name": "Charlie", "age": 35}
]
names = map(lambda user: user["name"], users)
print(list(names))  # ['Alice', 'Bob', 'Charlie']

# Lazy evaluation
large_range = range(1, 1000000)
mapped = map(lambda x: x**2, large_range)  # Creates iterator, doesn't compute values yet
print(next(mapped))  # 1 (computes only the first value)
print(next(mapped))  # 4 (computes only the second value)

# Handling different-length iterables
```

```

nums1 = [1, 2, 3, 4]
nums2 = [10, 20, 30] # Shorter than nums1
result = map(lambda x, y: x + y, nums1, nums2)
print(list(result)) # [11, 22, 33] (stops at shortest iterable)

```

`map()` is useful for transforming all elements in a sequence without writing an explicit loop. It's memory-efficient because it returns an iterator rather than creating the entire result list at once.

filter() The `filter()` function constructs an iterator from elements of an iterable for which a function returns true.

```

# Basic filter usage
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
even = filter(lambda x: x % 2 == 0, numbers)
print(list(even)) # [2, 4, 6, 8, 10]

# Equivalent list comprehension
even_comp = [x for x in numbers if x % 2 == 0]
print(even_comp) # [2, 4, 6, 8, 10]

# Using a named function
def is_positive(x):
    return x > 0

values = [-2, -1, 0, 1, 2]
positive = filter(is_positive, values)
print(list(positive)) # [1, 2]

# Filtering with None removes falsy values
mixed = [0, 1, False, True, None, "", "hello", []]
truthy = filter(None, mixed)
print(list(truthy)) # [1, True, 'hello']

```

```

# Filtering objects
users = [
    {"name": "Alice", "active": True},
    {"name": "Bob", "active": False},
    {"name": "Charlie", "active": True}
]
active_users = filter(lambda user: user["active"], users)
print(list(active_users)) # [{"name": "Alice", "active": True}, {"name": "Charlie", "active": True}]

```

```

# Filtering strings
words = ["apple", "banana", "cherry", "date", "elderberry"]
long_words = filter(lambda word: len(word) > 5, words)

```



```

print(list(long_words)) # ['banana', 'cherry', 'elderberry']

# Combining filter and map
numbers = list(range(1, 11))
even_squares = map(lambda x: x**2, filter(lambda x: x % 2 == 0, numbers))
print(list(even_squares)) # [4, 16, 36, 64, 100]

# Equivalent list comprehension (usually more readable)
even_squares_comp = [x**2 for x in numbers if x % 2 == 0]
print(even_squares_comp) # [4, 16, 36, 64, 100]

filter() is useful for selecting elements from a sequence based on a condition.
Like map(), it returns an iterator for memory efficiency.

reduce() The reduce() function, from the functools module, applies a
function of two arguments cumulatively to the items of a sequence.

from functools import reduce

# Basic reduce usage - sum
numbers = [1, 2, 3, 4, 5]
sum_result = reduce(lambda x, y: x + y, numbers)
print(sum_result) # 15 (1+2+3+4+5)

# How reduce works step by step:
# Step 1: x=1, y=2 => result=3
# Step 2: x=3, y=3 => result=6
# Step 3: x=6, y=4 => result=10
# Step 4: x=10, y=5 => result=15

# With an initial value
sum_with_initial = reduce(lambda x, y: x + y, numbers, 10)
print(sum_with_initial) # 25 (10+1+2+3+4+5)

# Product of numbers
product = reduce(lambda x, y: x * y, numbers)
print(product) # 120 (1*2*3*4*5)

# Finding maximum
maximum = reduce(lambda x, y: x if x > y else y, numbers)
print(maximum) # 5

# Finding minimum
minimum = reduce(lambda x, y: x if x < y else y, numbers)
print(minimum) # 1

```

```

# Concat strings
words = ["hello", " ", "world", "!"]
sentence = reduce(lambda x, y: x + y, words)
print(sentence) # "hello world!"

# Flatten a list of lists
nested = [[1, 2], [3, 4], [5, 6]]
flattened = reduce(lambda x, y: x + y, nested)
print(flattened) # [1, 2, 3, 4, 5, 6]

# Building more complex accumulations
transactions = [
    {"amount": 100, "type": "deposit"},
    {"amount": 50, "type": "withdrawal"},
    {"amount": 75, "type": "deposit"}
]

def account_reducer(acc, transaction):
    if transaction["type"] == "deposit":
        acc["balance"] += transaction["amount"]
    else:
        acc["balance"] -= transaction["amount"]
    acc["transactions"] += 1
    return acc

initial_state = {"balance": 0, "transactions": 0}
final_state = reduce(account_reducer, transactions, initial_state)
print(final_state) # {'balance': 125, 'transactions': 3}

```

`reduce()` is powerful for cumulative operations and can often replace loops that build up a single result. It's especially useful for operations where each step depends on the result of the previous step.

lambda Lambda functions are small anonymous functions defined with the `lambda` keyword. They can have any number of arguments but only one expression.

```

# Basic lambda function
square = lambda x: x**2
print(square(5)) # 25

# Equivalent to:
def square_func(x):
    return x**2

# Lambdas with multiple arguments

```

```

add = lambda x, y: x + y
print(add(3, 4)) # 7

# Lambdas in sorting
pairs = [(1, 'one'), (2, 'two'), (3, 'three'), (4, 'four')]
# Sort by second element
pairs.sort(key=lambda pair: pair[1])
print(pairs) # [(4, 'four'), (1, 'one'), (3, 'three'), (2, 'two')]

# Or with a custom object
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age
    def __repr__(self):
        return f"Person('{self.name}', {self.age})"

people = [Person("Alice", 30), Person("Bob", 25), Person("Charlie", 35)]
# Sort by age
people.sort(key=lambda person: person.age)
print(people) # [Person('Bob', 25), Person('Alice', 30), Person('Charlie', 35)]

# Lambdas with conditionals
is_even = lambda x: x % 2 == 0
print(is_even(4)) # True
print(is_even(5)) # False

# Conditional expression in lambda
classify = lambda x: "even" if x % 2 == 0 else "odd"
print(classify(4)) # "even"
print(classify(5)) # "odd"

# Immediately invoked lambda
result = (lambda x, y: x + y)(3, 4)
print(result) # 7

# Lambdas in higher-order functions
numbers = [1, 2, 3, 4, 5]
squares = list(map(lambda x: x**2, numbers))
even_numbers = list(filter(lambda x: x % 2 == 0, numbers))
print(squares) # [1, 4, 9, 16, 25]
print(even_numbers) # [2, 4]

# Lambdas as function factories
def multiplier(factor):
    return lambda x: x * factor

```

```
double = multiplier(2)
triple = multiplier(3)
print(double(5))    # 10
print(triple(5))    # 15
```

```
# Lambda limitations
# Cannot contain statements, only expressions
# Cannot have docstrings
# Are generally less readable for complex operations
```

Lambda functions are useful for small, one-off functions, especially when passed to higher-order functions like `map()`, `filter()`, and `sorted()`. However, for more complex or reused functionality, named functions are generally more readable and maintainable.

functools: partial, lru_cache, and reduce

The `functools` module provides higher-order functions and operations on callable objects. It includes several tools that enhance functional programming.

partial() The `partial()` function creates a new function with some of the arguments of the original function fixed.

```
from functools import partial

# Basic partial function
def power(base, exponent):
    return base ** exponent

# Create a function for squaring (exponent fixed to 2)
square = partial(power, exponent=2)
print(square(3))    # 9
print(square(4))    # 16

# Create a function for powers of 2 (base fixed to 2)
powers_of_two = partial(power, 2)
print(powers_of_two(3))    # 8 (2^3)
print(powers_of_two(4))    # 16 (2^4)

# Partial with multiple arguments
def format_string(prefix, text, suffix):
    return f"{prefix}{text}{suffix}"

# Create HTML tag functions
bold = partial(format_string, "<b>", suffix="</b>")
italic = partial(format_string, "<i>", suffix="</i>")
```

```

print(bold("Hello"))      # <b>Hello</b>
print(italic("World"))    # <i>World</i>

# Practical example: fixed precision formatter
def round_to(value, precision):
    return round(value, precision)

round_to_2 = partial(round_to, precision=2)
round_to_4 = partial(round_to, precision=4)

print(round_to_2(3.14159)) # 3.14
print(round_to_4(3.14159)) # 3.1416

# Partial with sorted
sorted_by_last_letter = partial(sorted, key=lambda x: x[-1])
fruits = ["apple", "banana", "cherry", "date"]
print(sorted_by_last_letter(fruits)) # ['banana', 'apple', 'date', 'cherry']

# Using partial with map
numbers = [1, 2, 3, 4, 5]
add_ten = partial(lambda x, y: x + y, 10)
result = list(map(add_ten, numbers))
print(result) # [11, 12, 13, 14, 15]

# Currying with partial
def curry_function(func, arity):
    """
    Curry a function of arity arguments into nested unary functions.
    """
    if arity <= 1:
        return func

    return lambda x: curry_function(partial(func, x), arity - 1)

def add_three(a, b, c):
    return a + b + c

curried_add = curry_function(add_three, 3)
print(curried_add(1)(2)(3)) # 6

```

`partial()` is useful for function specialization and creating reusable function variants without duplicating code.

`lru_cache()` The `lru_cache()` decorator implements memoization, caching the results of function calls based on arguments to avoid redundant calculations.

```

from functools import lru_cache
import time

# Basic caching
@lru_cache(maxsize=None) # No limit on cache size
def fibonacci(n):
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# Without caching, this would be extremely slow
print(fibonacci(100)) # 354224848179261915075

# Timing comparison
def fibonacci_uncached(n):
    if n <= 1:
        return n
    return fibonacci_uncached(n-1) + fibonacci_uncached(n-2)

# Would take too long for n=35, but cached version is fast
start = time.time()
result1 = fibonacci(35)
cached_time = time.time() - start

start = time.time()
result2 = fibonacci_uncached(20) # Using smaller n for uncached version
uncached_time = time.time() - start

print(f"Cached time: {cached_time:.6f} seconds")
print(f"Uncached time: {uncached_time:.6f} seconds")

# Cache information
print(fibonacci.cache_info()) # Shows hits, misses, size, and maxsize

# Clearing the cache
fibonacci.cache_clear()
print(fibonacci.cache_info()) # Shows empty cache

# Using maxsize to limit cache
@lru_cache(maxsize=32)
def expensive_calculation(n):
    time.sleep(0.1) # Simulate expensive operation
    return n * 2

# First call is slow, second is instant
start = time.time()

```

```

expensive_calculation(5)
first_call = time.time() - start

start = time.time()
expensive_calculation(5)  # Cached result
second_call = time.time() - start

print(f"First call: {first_call:.6f} seconds")
print(f"Second call: {second_call:.6f} seconds")

# Cache with typed=True (arguments of different types are cached separately)
@lru_cache(maxsize=None, typed=True)
def add(a, b):
    print(f"Computing {a} + {b}")
    return a + b

print(add(1, 2))          # Computing 1 + 2, then 3
print(add(1, 2))          # Cached result: 3
print(add(1.0, 2.0))      # Computing 1.0 + 2.0, then 3.0 (different types)
print(add(1, 2))          # Cached result: 3

# Real-world example: API call caching
@lru_cache(maxsize=100)
def fetch_data(url, param1, param2):
    print(f"Fetching data from {url} with params {param1}, {param2}")
    # In real code, this would be an actual API request
    return f"Data from {url}: {param1}-{param2}"

print(fetch_data("api.example.com", "value1", "value2"))
print(fetch_data("api.example.com", "value1", "value2")) # Cached
print(fetch_data("api.example.com", "value1", "value3")) # Different param, not cached

```

`lru_cache()` is invaluable for improving performance of recursive functions, expensive computations, and API calls. The “LRU” stands for “Least Recently Used,” meaning when the cache is full, the least recently used items are discarded first.

reduce() (covered earlier) We’ve already covered `reduce()` in detail, but as a reminder, it’s part of the `functools` module and applies a binary function cumulatively to the items of a sequence.

```

from functools import reduce

numbers = [1, 2, 3, 4, 5]
sum_result = reduce(lambda x, y: x + y, numbers)
print(sum_result)  # 15

```

Other useful functools utilities

```
from functools import wraps, singledispatch, total_ordering, cached_property
```

```
# wraps - preserves function metadata in decorators
```

```
def my_decorator(func):
    @wraps(func) # Preserves __name__, __doc__, etc.
    def wrapper(*args, **kwargs):
        """Wrapper docstring"""
        print("Before function call")
        result = func(*args, **kwargs)
        print("After function call")
        return result
    return wrapper
```

```
@my_decorator
```

```
def say_hello(name):
    """Say hello to someone."""
    print(f"Hello, {name}!")
    return name
```

```
say_hello("Alice")
```

```
print(say_hello.__name__) # 'say_hello' (preserved by @wraps)
```

```
print(say_hello.__doc__) # 'Say hello to someone.' (preserved by @wraps)
```

```
# singledispatch - function overloading based on argument type
```

```
@singledispatch
```

```
def format_object(obj):
    return str(obj)
```

```
@format_object.register
```

```
def _(obj: int):
    return f"Integer: {obj}"
```

```
@format_object.register
```

```
def _(obj: list):
    return f"List with {len(obj)} items"
```

```
@format_object.register(dict) # Alternative registration syntax
```

```
def _(obj):
    return f"Dictionary with {len(obj)} keys"
```

```
print(format_object("hello")) # 'hello'
```

```
print(format_object(42)) # 'Integer: 42'
```

```
print(format_object([1, 2, 3])) # 'List with 3 items'
```

```
print(format_object({"a": 1})) # 'Dictionary with 1 keys'
```



```

# total_ordering - generates comparison methods from a minimal set
@total_ordering
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def __eq__(self, other):
        return (self.age, self.name) == (other.age, other.name)

    def __lt__(self, other):
        return (self.age, self.name) < (other.age, other.name)

# Now all comparison operators work
p1 = Person("Alice", 30)
p2 = Person("Bob", 25)
print(p1 > p2)    # True
print(p1 >= p2)   # True
print(p1 < p2)    # False
print(p1 <= p2)   # False

# cached_property (Python 3.8+) - property with automatic caching
class DataProcessor:
    def __init__(self, data):
        self.data = data

    @cached_property
    def processed_data(self):
        print("Processing data...")
        # Simulate expensive calculation
        import time
        time.sleep(1)
        return [x * 2 for x in self.data]

processor = DataProcessor([1, 2, 3, 4, 5])
print(processor.processed_data) # Prints "Processing data..." then [2, 4, 6, 8, 10]
print(processor.processed_data) # Just returns [2, 4, 6, 8, 10] (cached)

```

These `functools` utilities enhance Python's functional programming capabilities and provide elegant solutions for common programming patterns.

itertools: chain, combinations, groupby, etc.

The `itertools` module provides a collection of fast, memory-efficient tools for creating iterators for efficient looping. They're inspired by constructs from APL,

Haskell, and SML.

chain() The `chain()` function combines multiple iterables into a single sequential iterator.

```
from itertools import chain

# Basic usage
list1 = [1, 2, 3]
list2 = [4, 5, 6]
list3 = [7, 8, 9]

# Chain multiple iterables
combined = chain(list1, list2, list3)
print(list(combined)) # [1, 2, 3, 4, 5, 6, 7, 8, 9]

# Chain different types of iterables
numbers = [1, 2, 3]
letters = "abc"
pairs = {("x", 1), ("y", 2)}
mixed = chain(numbers, letters, pairs)
print(list(mixed)) # [1, 2, 3, 'a', 'b', 'c', ('x', 1), ('y', 2)]

# Using chain.from_iterable with a list of iterables
nested = [[1, 2], [3, 4, 5], [6, 7]]
flattened = chain.from_iterable(nested)
print(list(flattened)) # [1, 2, 3, 4, 5, 6, 7]

# With generator expressions
ranges = (range(i, i+3) for i in range(3))
result = chain.from_iterable(ranges)
print(list(result)) # [0, 1, 2, 1, 2, 3, 2, 3, 4]

# Practical example: processing multiple data sources
filenames = ['data1.txt', 'data2.txt', 'data3.txt']

def read_file(filename):
    # In real code, this would read from actual files
    if filename == 'data1.txt':
        return [1, 2, 3]
    elif filename == 'data2.txt':
        return [4, 5]
    else:
        return [6, 7, 8]

# Process all data sequentially
```

```
all_data = chain.from_iterable(read_file(f) for f in filenames)
print(list(all_data)) # [1, 2, 3, 4, 5, 6, 7, 8]
```

`chain()` is useful for processing multiple sequences as if they were a single sequence, avoiding the need to create a new concatenated list.

combinations() and **permutations()** These functions generate combinations and permutations from an iterable.

```
from itertools import combinations, permutations, combinations_with_replacement

# Basic combinations (order doesn't matter)
items = ['A', 'B', 'C', 'D']
# All combinations of 2 elements
combs = combinations(items, 2)
print(list(combs))
# [('A', 'B'), ('A', 'C'), ('A', 'D'), ('B', 'C'), ('B', 'D'), ('C', 'D')]

# Combinations of different sizes
for r in range(1, len(items) + 1):
    print(f"{r}-combinations:", list(combinations(items, r)))

# 1-combinations: [('A',), ('B',), ('C',), ('D',)]
# 2-combinations: [('A', 'B'), ('A', 'C'), ('A', 'D'), ('B', 'C'), ('B', 'D'), ('C', 'D')]
# 3-combinations: [('A', 'B', 'C'), ('A', 'B', 'D'), ('A', 'C', 'D'), ('B', 'C', 'D')]
# 4-combinations: [('A', 'B', 'C', 'D')]

# Permutations (order matters)
perms = permutations(items, 2)
print(list(perms))
# [('A', 'B'), ('A', 'C'), ('A', 'D'), ('B', 'A'), ('B', 'C'), ('B', 'D'),
#  ('C', 'A'), ('C', 'B'), ('C', 'D'), ('D', 'A'), ('D', 'B'), ('D', 'C')]

# All permutations (all items)
all_perms = permutations(items)
print(len(list(all_perms))) # 24 (4! = 24)

# Combinations with replacement (elements can be repeated)
combs_repl = combinations_with_replacement(items, 2)
print(list(combs_repl))
# [('A', 'A'), ('A', 'B'), ('A', 'C'), ('A', 'D'), ('B', 'B'), ('B', 'C'),
#  ('B', 'D'), ('C', 'C'), ('C', 'D'), ('D', 'D')]

# Practical example: generate all possible dice combinations
dice = [1, 2, 3, 4, 5, 6]
# All possible outcomes when rolling 2 dice
```

```

dice_outcomes = list(combinations_with_replacement(dice, 2))
print(len(dice_outcomes))  # 21

# Count frequency of sums
from collections import Counter
dice_sums = Counter(sum(outcome) for outcome in dice_outcomes)
print(dice_sums)
# Counter({7: 6, 6: 5, 8: 5, 5: 4, 9: 4, 4: 3, 10: 3, 3: 2, 11: 2, 2: 1, 12: 1})

```

Combinations and permutations are useful for generating all possible arrangements or selections from a set of items.

groupby() The `groupby()` function creates an iterator that returns consecutive keys and groups from an iterable.

```

from itertools import groupby

# Basic usage
data = [
    {"name": "Alice", "department": "HR"},
    {"name": "Bob", "department": "Engineering"},
    {"name": "Charlie", "department": "Engineering"},
    {"name": "Dave", "department": "HR"},
    {"name": "Eve", "department": "Finance"}
]

# Important: data must be sorted by the grouping key for groupby to work correctly
data.sort(key=lambda x: x["department"])

# Group by department
for department, group in groupby(data, key=lambda x: x["department"]):
    print(f"Department: {department}")
    for employee in group:
        print(f"    - {employee['name']}")

# Output:
# Department: Engineering
#     - Bob
#     - Charlie
# Department: Finance
#     - Eve
# Department: HR
#     - Alice
#     - Dave

# Grouping consecutive numbers

```

```

numbers = [1, 1, 1, 2, 2, 3, 3, 3, 3, 1, 1]
for key, group in groupby(numbers):
    print(f"{key}: {list(group)}")

# Output:
# 1: [1, 1, 1]
# 2: [2, 2]
# 3: [3, 3, 3, 3]
# 1: [1, 1]

# Grouping strings by their first letter
words = ["apple", "banana", "apricot", "cherry", "blueberry", "cantaloupe"]
words.sort() # Sort first for groupby to work correctly

for letter, group in groupby(words, key=lambda word: word[0]):
    print(f"{letter}: {list(group)}")

# Output:
# a: ['apple', 'apricot']
# b: ['banana', 'blueberry']
# c: ['cantaloupe', 'cherry']

# Creating a dictionary from groups
data = [
    ("Math", "Alice", 85),
    ("Math", "Bob", 92),
    ("Science", "Alice", 78),
    ("Science", "Bob", 85),
    ("Math", "Charlie", 90)
]
data.sort(key=lambda x: x[0]) # Sort by subject

subject_grades = {}
for subject, group in groupby(data, key=lambda x: x[0]):
    subject_grades[subject] = list(group)

print(subject_grades)
# {'Math': [('Math', 'Alice', 85), ('Math', 'Bob', 92), ('Math', 'Charlie', 90)],
#  'Science': [('Science', 'Alice', 78), ('Science', 'Bob', 85)]}

# Practical example: finding runs of consecutive numbers
from operator import itemgetter

def find_consecutive_runs(numbers):
    result = []
    # Enumerate and sort the numbers

```

```

    for k, g in groupby(enumerate(sorted(numbers)), lambda x: x[0] - x[1]):
        group = list(map(itemgetter(1), g))
        result.append((group[0], group[-1]))
    return result

nums = [1, 2, 3, 5, 6, 7, 9, 10, 11, 12]
runs = find_consecutive_runs(nums)
print(runs)  # [(1, 3), (5, 7), (9, 12)]

```

groupby() is useful for grouping data based on a key function. Remember that the input sequence needs to be sorted by the same key function for groupby() to work as expected.

count(), cycle(), and repeat() These functions create infinite iterators that can be useful in various contexts.

```

from itertools import count, cycle, repeat
import itertools

# count() - count from n with step size
counter = count(10, 2)  # Start at 10, increment by 2
for i in range(5):
    print(next(counter), end=' ')  # 10 12 14 16 18

# Using count with zip to add indices
colors = ['red', 'green', 'blue']
indexed_colors = list(zip(count(), colors))
print(indexed_colors)  # [(0, 'red'), (1, 'green'), (2, 'blue')]

# Using count with map
squared_evens = list(itertools.islice(map(lambda x: x**2, count(0, 2)), 5))
print(squared_evens)  # [0, 4, 16, 36, 64]

# cycle() - cycle through elements of an iterable indefinitely
cycle_days = cycle(['Monday', 'Tuesday', 'Wednesday', 'Thursday', 'Friday', 'Saturday', 'Sunday'])
for i in range(10):
    print(next(cycle_days), end=' ')
# Monday Tuesday Wednesday Thursday Friday Saturday Sunday Monday Tuesday Wednesday

# Using cycle for round-robin scheduling
tasks = ['Task A', 'Task B', 'Task C']
workers = ['Worker 1', 'Worker 2']
assignments = list(zip(cycle(workers), tasks))
print(assignments)
# [('Worker 1', 'Task A'), ('Worker 2', 'Task B'), ('Worker 1', 'Task C')]

```

```

# repeat() - repeat an object indefinitely or n times
repeater = repeat('Hello', 3)
print(list(repeater)) # ['Hello', 'Hello', 'Hello']

# Using repeat with map to apply a function multiple times
from math import sqrt
data = [1, 4, 9, 16, 25]
transformed = list(map(sqrt, data))
print(transformed) # [1.0, 2.0, 3.0, 4.0, 5.0]

# repeat indefinitely (controlled by other iterables)
print(list(zip([1, 2, 3], repeat('X')))) # [(1, 'X'), (2, 'X'), (3, 'X')]

# Using repeat for constant arguments in map
print(list(map(pow, range(5), repeat(2)))) # [0, 1, 4, 9, 16]

```

These infinite iterators are useful for creating sequences and repeating elements. They're often used with other functions like `zip()`, `map()`, and `islice()` to control how many elements are processed.

Other useful itertools functions

```

from itertools import islice, takewhile, dropwhile, product, zip_longest, accumulate

# islice() - slice an iterator
numbers = range(10) # 0-9
print(list(islice(numbers, 3))) # [0, 1, 2]
print(list(islice(numbers, 2, 6))) # [2, 3, 4, 5]
print(list(islice(numbers, 1, 9, 2))) # [1, 3, 5, 7]

# With infinite iterators
print(list(islice(count(), 5))) # [0, 1, 2, 3, 4]

# takewhile() - take items while predicate is true
numbers = [1, 2, 3, 4, 5, 2, 1]
result = takewhile(lambda x: x < 4, numbers)
print(list(result)) # [1, 2, 3]

# dropwhile() - drop items while predicate is true, then return all items
result = dropwhile(lambda x: x < 4, numbers)
print(list(result)) # [4, 5, 2, 1]

# product() - cartesian product of input iterables
print(list(product('AB', '12')))
# [('A', '1'), ('A', '2'), ('B', '1'), ('B', '2')]

```

```

# Multiple iterables
print(list(product([1, 2], [3, 4], ['x', 'y'])))
# [(1, 3, 'x'), (1, 3, 'y'), (1, 4, 'x'), (1, 4, 'y'),
#  (2, 3, 'x'), (2, 3, 'y'), (2, 4, 'x'), (2, 4, 'y')]

# Repeat parameter
print(list(product('AB', repeat=2)))
# [('A', 'A'), ('A', 'B'), ('B', 'A'), ('B', 'B')]

# zip_longest() - zip iterables, filling missing values with fillvalue
list1 = [1, 2, 3]
list2 = ['a', 'b', 'c', 'd', 'e']
print(list(zip(list1, list2))) # [(1, 'a'), (2, 'b'), (3, 'c')]
print(list(zip_longest(list1, list2, fillvalue=0)))
# [(1, 'a'), (2, 'b'), (3, 'c'), (0, 'd'), (0, 'e')]

# accumulate() - cumulative sum or custom function
print(list(accumulate([1, 2, 3, 4, 5]))) # [1, 3, 6, 10, 15]

# With custom function
import operator
print(list(accumulate([1, 2, 3, 4, 5], operator.mul))) # [1, 2, 6, 24, 120]

# Maximum accumulation
print(list(accumulate([5, 2, 10, 3, 7], max))) # [5, 5, 10, 10, 10]

# Practical example: calculating a running average
data = [3, 5, 2, 8, 6]
totals = accumulate(data)
averages = [total / (i+1) for i, total in enumerate(totals)]
print(averages) # [3.0, 4.0, 3.3333333333333335, 4.5, 4.8]

```

The `itertools` module provides a comprehensive set of tools for creating and working with iterators, making it possible to write clean and efficient code for sequence processing tasks.

3. Object-Oriented Programming

Object-Oriented Programming (OOP) is a programming paradigm that uses “objects” to model real-world entities. Python is a multi-paradigm language that fully supports OOP.

Classes, Inheritance, and `super()`

Class Definition and Instantiation


```

# Basic class definition
class Person:
    """A simple class representing a person."""

    # Class attribute (shared by all instances)
    species = "Homo sapiens"

    # Initializer / Constructor
    def __init__(self, name, age):
        """Initialize a Person instance."""
        # Instance attributes (unique to each instance)
        self.name = name
        self.age = age

    # Instance method
    def greet(self):
        """Return a greeting."""
        return f"Hello, my name is {self.name}"

    # Another instance method
    def celebrate_birthday(self):
        """Increment age and return birthday message."""
        self.age += 1
        return f"Happy {self.age}th birthday, {self.name}!"

# Creating instances (objects)
alice = Person("Alice", 30)
bob = Person("Bob", 25)

# Accessing attributes
print(alice.name)          # Alice
print(bob.age)             # 25
print(alice.species)       # Homo sapiens
print(Person.species)      # Homo sapiens (accessed from class)

# Calling methods
print(alice.greet())        # Hello, my name is Alice
print(bob.celebrate_birthday()) # Happy 26th birthday, Bob!
print(bob.age)              # 26 (age was incremented)

# Modifying attributes
alice.name = "Alicia"
print(alice.name)          # Alicia

# Adding attributes dynamically
alice.email = "alice@example.com"

```

```

print(alice.email)          # alice@example.com
# print(bob.email)          # AttributeError (bob doesn't have email attribute)

# Checking instance type
print(isinstance(alice, Person)) # True
print(isinstance("string", Person)) # False

# Checking class attributes
print(issubclass(Person, object)) # True (all classes inherit from object)

```

Classes define a blueprint for creating objects, with attributes for state and methods for behavior.

Inheritance Inheritance allows a class to inherit attributes and methods from another class, promoting code reuse and establishing a hierarchical relationship.

```

# Base class (parent class)
class Animal:
    """A simple animal class."""

    def __init__(self, name, species):
        """Initialize an Animal instance."""
        self.name = name
        self.species = species

    def make_sound(self):
        """Generic sound method."""
        return "Some generic sound"

    def describe(self):
        """Return a description."""
        return f"{self.name} is a {self.species}"

# Derived class (child class)
class Dog(Animal):
    """A dog class inheriting from Animal."""

    def __init__(self, name, breed):
        """Initialize a Dog instance."""
        # Call parent initializer
        super().__init__(name, species="Dog")
        self.breed = breed

    # Override parent method
    def make_sound(self):
        """Dogs bark."""

```

```

        return "Woof!"

    # Add new method
    def fetch(self, item):
        """Dogs can fetch items."""
        return f"{self.name} fetched the {item}"

# Another derived class
class Cat(Animal):
    """A cat class inheriting from Animal."""

    def __init__(self, name, color):
        """Initialize a Cat instance."""
        super().__init__(name, species="Cat")
        self.color = color

    def make_sound(self):
        """Cats meow."""
        return "Meow!"

    def scratch(self):
        """Cats can scratch."""
        return f"{self.name} scratches"

# Creating instances
dog = Dog("Rex", "German Shepherd")
cat = Cat("Whiskers", "Gray")

# Using inherited methods
print(dog.describe()) # Rex is a Dog
print(cat.describe()) # Whiskers is a Cat

# Using overridden methods
print(dog.make_sound()) # Woof!
print(cat.make_sound()) # Meow!

# Using class-specific methods
print(dog.fetch("ball")) # Rex fetched the ball
print(cat.scratch())     # Whiskers scratches

# Checking inheritance relationships
print(isinstance(dog, Dog))    # True
print(isinstance(dog, Animal)) # True
print(isinstance(dog, Cat))    # False
print(issubclass(Dog, Animal)) # True
print(issubclass(Animal, Dog)) # False

```

Inheritance allows for creating specialized classes that reuse and extend the functionality of existing classes.

Multiple Inheritance Python supports multiple inheritance, allowing a class to inherit from multiple parent classes.

```
# First parent class
class Flyer:
    """A class for flying creatures."""

    def fly(self):
        """Fly method."""
        return "Flying high!"

    def navigate(self):
        """Navigate through air."""
        return "Navigating through air"

# Second parent class
class Swimmer:
    """A class for swimming creatures."""

    def swim(self):
        """Swim method."""
        return "Swimming gracefully!"

    def navigate(self):
        """Navigate through water."""
        return "Navigating through water"

# Class with multiple inheritance
class Duck(Flyer, Swimmer):
    """A duck class inheriting from both Flyer and Swimmer."""

    def make_sound(self):
        """Ducks quack."""
        return "Quack!"

    # No need to override navigate - will use first parent's version
    # due to Method Resolution Order (MRO)

# Creating an instance
duck = Duck()

# Using methods from different parents
print(duck.fly())    # Flying high!
```

```

print(duck.swim())      # Swimming gracefully!
print(duck.navigate())  # Navigating through air (from Flyer)

# Method Resolution Order (MRO)
print(Duck.__mro__)
# (<class '__main__.Duck'>, <class '__main__.Flyer'>, <class '__main__.Swimmer'>, <class 'object'>)

# Explicitly calling a specific parent's method
print(Swimmer.navigate(duck)) # Navigating through water

# Diamond inheritance problem
class A:
    def method(self):
        return "A's method"

class B(A):
    def method(self):
        return "B's method"

class C(A):
    def method(self):
        return "C's method"

# D inherits from both B and C, which both inherit from A
class D(B, C):
    pass

d = D()
print(d.method()) # B's method (follows MRO: D -> B -> C -> A)
print(D.__mro__)  # Shows the Method Resolution Order

```

Multiple inheritance can be powerful but adds complexity due to method resolution. Python uses the C3 linearization algorithm to determine the method resolution order (MRO), which dictates the order in which parent classes are searched for methods.

super() The `super()` function is used to call methods from a parent or sibling class, particularly useful in inheritance hierarchies.

```

# Basic super() usage
class Parent:
    def __init__(self, name):
        self.name = name

    def greet(self):
        return f"Hello, I'm {self.name}"

```

```

class Child(Parent):
    def __init__(self, name, age):
        super().__init__(name) # Call Parent's __init__
        self.age = age

    def greet(self):
        parent_greeting = super().greet() # Call Parent's greet
        return f"{parent_greeting} and I'm {self.age} years old"

# Create Child instance
child = Child("Alice", 10)
print(child.greet()) # Hello, I'm Alice and I'm 10 years old

# Super with multiple inheritance
class A:
    def method(self):
        return "A's method"

class B(A):
    def method(self):
        return f"B's method, then {super().method()}"

class C(A):
    def method(self):
        return f"C's method, then {super().method()}"

class D(B, C):
    def method(self):
        return f"D's method, then {super().method()}"

d = D()
print(d.method())
# D's method, then B's method, then C's method, then A's method
# (follows MRO: D -> B -> C -> A)

# Super with explicit arguments (rarely needed)
class Base:
    def __init__(self):
        print("Base init")

class Middle(Base):
    def __init__(self):
        print("Middle init")
        super().__init__()

```

```
class Top(Middle):
    def __init__(self):
        print("Top init")
        super().__init__()
```

```
top = Top()
# Output:
# Top init
# Middle init
# Base init
```

The `super()` function provides a more maintainable way to access methods from parent classes, especially in complex inheritance hierarchies. It automatically follows the method resolution order (MRO).

Special Methods: `__init__`, `__str__`, `__repr__`, etc.

Python's special methods (a.k.a. dunder or magic methods) allow classes to implement standard behavior and integrate with built-in functions and operators.

Basic Special Methods

```
class Vector:
    """A simple 2D vector class."""

    def __init__(self, x, y):
        """Initialize with x, y coordinates."""
        self.x = x
        self.y = y

    def __str__(self):
        """String representation for users."""
        return f"Vector({self.x}, {self.y})"

    def __repr__(self):
        """String representation for developers."""
        return f"Vector({self.x}, {self.y})"

    def __eq__(self, other):
        """Equality comparison."""
        if not isinstance(other, Vector):
            return NotImplemented
        return self.x == other.x and self.y == other.y

    def __add__(self, other):
        """Vector addition."""
        if not isinstance(other, Vector):
```

```

        return NotImplemented
    return Vector(self.x + other.x, self.y + other.y)

def __sub__(self, other):
    """Vector subtraction."""
    if not isinstance(other, Vector):
        return NotImplemented
    return Vector(self.x - other.x, self.y - other.y)

def __mul__(self, scalar):
    """Scalar multiplication."""
    if not isinstance(scalar, (int, float)):
        return NotImplemented
    return Vector(self.x * scalar, self.y * scalar)

def __rmul__(self, scalar):
    """Reverse scalar multiplication."""
    return self.__mul__(scalar)

def __neg__(self):
    """Negation."""
    return Vector(-self.x, -self.y)

def __abs__(self):
    """Absolute value (magnitude)."""
    return (self.x**2 + self.y**2)**0.5

def __bool__(self):
    """Boolean value."""
    return bool(self.x or self.y)

# Creating vectors
v1 = Vector(3, 4)
v2 = Vector(2, 7)

# String representation
print(v1)          # Vector(3, 4)
print(repr(v2))    # Vector(2, 7)

# Operations
print(v1 + v2)     # Vector(5, 11)
print(v1 - v2)     # Vector(1, -3)
print(v1 * 2)      # Vector(6, 8)
print(3 * v2)      # Vector(6, 21)
print(-v1)         # Vector(-3, -4)

```



```

# Other methods
print(abs(v1))    # 5.0 (magnitude)
print(bool(v1))   # True
print(bool(Vector(0, 0))) # False

# Equality
print(v1 == Vector(3, 4)) # True
print(v1 == v2)           # False

```

Special methods allow classes to behave like built-in types and integrate with Python's language features.

Container Special Methods

```

class Deck:
    """A simple card deck class."""

    def __init__(self):
        """Initialize a standard deck of cards."""
        self.cards = []
        for suit in ['Hearts', 'Diamonds', 'Clubs', 'Spades']:
            for rank in range(1, 14):
                self.cards.append((rank, suit))

    def __len__(self):
        """Return number of cards."""
        return len(self.cards)

    def __getitem__(self, position):
        """Get card at position."""
        return self.cards[position]

    def __setitem__(self, position, card):
        """Set card at position."""
        self.cards[position] = card

    def __delitem__(self, position):
        """Delete card at position."""
        del self.cards[position]

    def __contains__(self, card):
        """Check if card is in deck."""
        return card in self.cards

    def __iter__(self):
        """Return iterator for cards."""

```

```

        return iter(self.cards)

    def __reversed__(self):
        """Return reversed iterator."""
        return reversed(self.cards)

# Create a deck
deck = Deck()

# Length
print(len(deck)) # 52

# Accessing items
print(deck[0])    # (1, 'Hearts') (Ace of Hearts)
print(deck[-1])   # (13, 'Spades') (King of Spades)

# Setting items
deck[0] = (1, 'Joker')
print(deck[0])    # (1, 'Joker')

# Deleting items
del deck[0]
print(len(deck)) # 51

# Membership test
print((1, 'Hearts') in deck) # False (we deleted it)
print((2, 'Hearts') in deck) # True

# Iteration
for card in deck:
    if card[0] == 10 and card[1] == 'Diamonds':
        print("Found 10 of Diamonds!")
        break

# Slicing (works because __getitem__ supports slices)
first_five = deck[:5]
print(first_five) # First five cards

# Reversed iteration
last_three = list(reversed(deck))[:3]
print(last_three) # Last three cards

```

Container special methods allow classes to behave like lists, dictionaries, and other container types, supporting operations like indexing, slicing, and iteration.

Attribute Access Special Methods

```

class LoggedAccess:
    """A class that logs attribute access."""

    def __init__(self, initial_attrs=None):
        """Initialize with optional attributes dictionary."""
        # Use a private dict to store attributes
        self.__dict__['_data'] = initial_attrs or {}
        self.__dict__['_log'] = []

    def __getattr__(self, name):
        """Called when attribute lookup fails on __getattribute__."""
        print(f"Getting attribute: {name}")
        self._log.append(f"Get: {name}")
        if name in self._data:
            return self._data[name]
        raise AttributeError(f"'{self.__class__.__name__}' has no attribute '{name}'")

    def __setattr__(self, name, value):
        """Called when setting an attribute."""
        print(f"Setting attribute: {name} = {value}")
        self._log.append(f"Set: {name} = {value}")
        self._data[name] = value

    def __delattr__(self, name):
        """Called when deleting an attribute."""
        print(f"Deleting attribute: {name}")
        self._log.append(f"Del: {name}")
        if name in self._data:
            del self._data[name]
        else:
            raise AttributeError(f"'{self.__class__.__name__}' has no attribute '{name}'")

    def get_log(self):
        """Return access log."""
        return self._log

# Create instance
obj = LoggedAccess({'x': 1, 'y': 2})

# Access attributes
print(obj.x)          # Getting attribute: x, 1
print(obj.y)          # Getting attribute: y, 2

# Set attribute
obj.z = 3              # Setting attribute: z = 3

```

```

# Delete attribute
del obj.y          # Deleting attribute: y

# Try accessing deleted attribute
try:
    print(obj.y)
except AttributeError as e:
    print(e)        # 'LoggedAccess' has no attribute 'y'

# Check log
print(obj.get_log())
# ['Get: x', 'Get: y', 'Set: z = 3', 'Del: y', 'Get: y']

# Property lookup order example
class AttrOrder:
    def __init__(self):
        self.regular_attr = "Instance attribute"

    @property
    def prop(self):
        return "Property value"

    def __getattr__(self, name):
        print(f"__getattr__ called for {name}")
        return super().__getattr__(name)

    def __getattribute__(self, name):
        print(f"__getattribute__ called for {name}")
        return super().__getattribute__(name)

attr_obj = AttrOrder()
print(attr_obj.regular_attr)  # __getattribute__ called..., then "Instance attribute"
print(attr_obj.prop)         # __getattribute__ called..., then "Property value"
print(attr_obj.missing)      # __getattribute__ called..., __getattr__ called..., then "Fallback for {name}"

```

Attribute access special methods allow for customizing how objects handle attribute lookups, assignments, and deletions. These methods are often used for implementing dynamic attributes, proxy objects, and data validation.

Context Manager Special Methods

```

class File:
    """A simple file context manager."""

    def __init__(self, filename, mode):
        """Initialize with filename and mode."""

```

```

        self.filename = filename
        self.mode = mode
        self.file = None

    def __enter__(self):
        """Enter context: open file and return it."""
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Exit context: close file and handle exceptions."""
        if self.file:
            self.file.close()
        # Return True to suppress exceptions, False to propagate
        return False

# Using the context manager
try:
    with File('example.txt', 'w') as f:
        f.write('Hello, world!')

    with File('example.txt', 'r') as f:
        content = f.read()
        print(content) # Hello, world!
except FileNotFoundError:
    print("File operations skipped (example.txt might not exist)")

# Context manager with exception handling
class SuppressErrors:
    """A context manager that suppresses specified exceptions."""

    def __init__(self, *exceptions):
        """Initialize with exceptions to suppress."""
        self.exceptions = exceptions or (Exception,)

    def __enter__(self):
        """Enter context."""
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Exit context and handle exceptions."""
        # Return True if exception should be suppressed
        return exc_type is not None and isinstance(exc_type, self.exceptions)

# Using the context manager to suppress specific exceptions
with SuppressErrors(ZeroDivisionError):

```

```

    print(1 / 0)  # Error suppressed
    print("This won't be printed")

print("Execution continues")

with SuppressErrors(ValueError):
    # This error won't be suppressed as it's not a ValueError
    try:
        print(1 / 0)
    except ZeroDivisionError:
        print("Error not suppressed")

```

Context manager special methods (`__enter__` and `__exit__`) allow classes to define behavior for setup and teardown operations in `with` statements. They're commonly used for resource management (file handling, database connections) and temporary state changes.

Descriptor Special Methods

```

class Validator:
    """A descriptor that validates values before setting them."""

    def __init__(self, name, validation_func, error_msg):
        """Initialize with attribute name, validation function, and error message."""
        self.name = name
        self.validation_func = validation_func
        self.error_msg = error_msg

    def __get__(self, instance, owner):
        """Get value from instance."""
        if instance is None:
            return self  # When accessed from class, return descriptor itself
        return instance.__dict__.get(self.name, None)

    def __set__(self, instance, value):
        """Validate and set value in instance."""
        if not self.validation_func(value):
            raise ValueError(f"{self.error_msg}: {value}")
        instance.__dict__[self.name] = value

    def __delete__(self, instance):
        """Delete value from instance."""
        if self.name in instance.__dict__:
            del instance.__dict__[self.name]

```

```

class Person:
    """A class using descriptors for validation."""

    name = Validator('name', lambda x: isinstance(x, str) and x, "Name must be a non-empty s
    age = Validator('age', lambda x: isinstance(x, int) and 0 <= x <= 120, "Age must be betw
    email = Validator('email', lambda x: isinstance(x, str) and '@' in x, "Invalid email for

    def __init__(self, name, age, email=None):
        """Initialize with name, age, and optional email."""
        self.name = name
        self.age = age
        if email:
            self.email = email

    def __str__(self):
        """String representation."""
        return f"{self.name}, {self.age} years old"

# Create a person
person = Person("Alice", 30, "alice@example.com")
print(person) # Alice, 30 years old

# Validation works on assignment
try:
    person.age = 150 # Too high
except ValueError as e:
    print(e) # Age must be between 0 and 120: 150

try:
    person.email = "invalid-email" # Missing @
except ValueError as e:
    print(e) # Invalid email format: invalid-email

# Valid assignments
person.age = 35
person.email = "alice.new@example.com"
print(f"{person.name}, {person.age}, {person.email}") # Alice, 35, alice.new@example.com

# Access descriptor from class
print(Person.name) # <__main__.Validator object at ...>

# Data descriptor example
class NonNegative:
    """A descriptor that ensures values are non-negative."""

    def __init__(self, name):

```

```

        self.name = name
        self.private_name = f"_{name}"

    def __get__(self, instance, owner):
        if instance is None:
            return self
        return getattr(instance, self.private_name, 0)

    def __set__(self, instance, value):
        if value < 0:
            raise ValueError(f"{self.name} must be non-negative")
        setattr(instance, self.private_name, value)

class Product:
    """A class representing a product with non-negative price and quantity."""

    price = NonNegative("price")
    quantity = NonNegative("quantity")

    def __init__(self, name, price, quantity):
        self.name = name
        self.price = price
        self.quantity = quantity

    @property
    def total(self):
        """Calculate total value."""
        return self.price * self.quantity

# Create a product
product = Product("Widget", 10.0, 5)
print(f"{product.name}: ${product.price} × {product.quantity} = ${product.total}")
# Widget: $10.0 × 5 = $50.0

# Try setting negative values
try:
    product.price = -1.0
except ValueError as e:
    print(e) # price must be non-negative

try:
    product.quantity = -3
except ValueError as e:
    print(e) # quantity must be non-negative

```

Descriptors provide a powerful way to define how attribute access is handled,

enabling features like validation, computed properties, and type conversion at the attribute level.

Properties and Descriptors

Properties and descriptors allow for controlled access to object attributes, enabling validation, computed values, and other custom behavior.

Properties Properties provide a way to define methods that behave like attributes, with optional getter, setter, and deleter methods.

```
class Temperature:
    """A class that stores temperature in Celsius but can convert to Fahrenheit."""

    def __init__(self, celsius=0):
        """Initialize with temperature in Celsius."""
        self._celsius = celsius

    @property
    def celsius(self):
        """Get temperature in Celsius."""
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """Set temperature in Celsius, with validation."""
        if value < -273.15:
            raise ValueError("Temperature cannot be below absolute zero")
        self._celsius = value

    @property
    def fahrenheit(self):
        """Get temperature in Fahrenheit."""
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        """Set temperature in Fahrenheit, converting to Celsius."""
        celsius = (value - 32) * 5/9
        if celsius < -273.15:
            raise ValueError("Temperature cannot be below absolute zero")
        self._celsius = celsius

# Create temperature object
temp = Temperature(25)
```

```

# Access properties
print(f"Celsius: {temp.celsius}°C")      # Celsius: 25°C
print(f"Fahrenheit: {temp.fahrenheit}°F") # Fahrenheit: 77.0°F

# Set temperature using Celsius
temp.celsius = 30
print(f"Celsius: {temp.celsius}°C")      # Celsius: 30°C
print(f"Fahrenheit: {temp.fahrenheit}°F") # Fahrenheit: 86.0°F

# Set temperature using Fahrenheit
temp.fahrenheit = 68
print(f"Celsius: {temp.celsius}°C")      # Celsius: 20.0°C
print(f"Fahrenheit: {temp.fahrenheit}°F") # Fahrenheit: 68.0°F

# Validation works
try:
    temp.celsius = -300 # Below absolute zero
except ValueError as e:
    print(e) # Temperature cannot be below absolute zero

# Property with only getter (read-only)
class Circle:
    """A class representing a circle."""

    def __init__(self, radius):
        """Initialize with radius."""
        self._radius = radius

    @property
    def radius(self):
        """Get radius."""
        return self._radius

    @radius.setter
    def radius(self, value):
        """Set radius with validation."""
        if value <= 0:
            raise ValueError("Radius must be positive")
        self._radius = value

    @property
    def area(self):
        """Calculate area (read-only)."""
        import math
        return math.pi * self._radius ** 2

```

```

    @property
    def circumference(self):
        """Calculate circumference (read-only)."""
        import math
        return 2 * math.pi * self._radius

# Create circle
circle = Circle(5)

# Access properties
print(f"Radius: {circle.radius}")          # Radius: 5
print(f"Area: {circle.area:.2f}")          # Area: 78.54
print(f"Circumference: {circle.circumference:.2f}") # Circumference: 31.42

# Modify radius
circle.radius = 7
print(f"Radius: {circle.radius}")          # Radius: 7
print(f"Area: {circle.area:.2f}")          # Area: 153.94

# Cannot set read-only properties
try:
    circle.area = 100
except AttributeError as e:
    print(e) # can't set attribute

```

Properties allow for attribute-like access to methods, making it possible to hide implementation details and add behavior like validation or computation.

Property Decorators Property decorators provide a cleaner syntax for defining properties.

```

class Person:
    """A class with properties for name and age."""

    def __init__(self, name, age):
        """Initialize with name and age."""
        self._name = name
        self._age = age

    @property
    def name(self):
        """Get name."""
        return self._name

    @name.setter
    def name(self, value):

```

```

        """Set name with validation."""
        if not value or not isinstance(value, str):
            raise ValueError("Name must be a non-empty string")
        self._name = value

    @property
    def age(self):
        """Get age."""
        return self._age

    @age.setter
    def age(self, value):
        """Set age with validation."""
        if not isinstance(value, int) or value < 0 or value > 120:
            raise ValueError("Age must be an integer between 0 and 120")
        self._age = value

    @property
    def is_adult(self):
        """Check if person is an adult (read-only)."""
        return self._age >= 18

# Create person
person = Person("Alice", 30)

# Access properties
print(f"Name: {person.name}")          # Name: Alice
print(f"Age: {person.age}")            # Age: 30
print(f"Is adult: {person.is_adult}")  # Is adult: True

# Modify properties
person.name = "Bob"
person.age = 15
print(f"Name: {person.name}")          # Name: Bob
print(f"Age: {person.age}")            # Age: 15
print(f"Is adult: {person.is_adult}")  # Is adult: False

# Validation
try:
    person.name = "" # Empty string
except ValueError as e:
    print(e) # Name must be a non-empty string

try:
    person.age = 150 # Too high
except ValueError as e:

```

```

        print(e) # Age must be an integer between 0 and 120

# Property defined without decorators (old style)
class OldStylePerson:
    def __init__(self, name):
        self._name = name

    def get_name(self):
        return self._name

    def set_name(self, value):
        if not value:
            raise ValueError("Name cannot be empty")
        self._name = value

    name = property(get_name, set_name, None, "Person's name")

# Both styles work the same way
old_person = OldStylePerson("Charlie")
print(old_person.name) # Charlie
old_person.name = "David"
print(old_person.name) # David

```

Property decorators offer a more readable and maintainable way to define properties compared to the traditional approach.

Custom Descriptors Descriptors provide a lower-level mechanism for controlling attribute access. They are classes that define the `__get__`, `__set__`, and/or `__delete__` methods.

```

class TypedProperty:
    """A descriptor that enforces a specific type."""

    def __init__(self, name, expected_type):
        """Initialize with attribute name and expected type."""
        self.name = name
        self.expected_type = expected_type

    def __get__(self, instance, owner):
        """Get value from instance."""
        if instance is None:
            return self
        return instance.__dict__.get(self.name, None)

    def __set__(self, instance, value):
        """Validate type and set value in instance."""

```

```

        if not isinstance(value, self.expected_type):
            raise TypeError(f"Expected {self.expected_type}, got {type(value)}")
        instance.__dict__[self.name] = value

    def __delete__(self, instance):
        """Delete value from instance."""
        if self.name in instance.__dict__:
            del instance.__dict__[self.name]

class RangeProperty:
    """A descriptor that enforces a minimum and maximum value."""

    def __init__(self, name, min_value=None, max_value=None):
        """Initialize with attribute name and optional range limits."""
        self.name = name
        self.min_value = min_value
        self.max_value = max_value

    def __get__(self, instance, owner):
        """Get value from instance."""
        if instance is None:
            return self
        return instance.__dict__.get(self.name, None)

    def __set__(self, instance, value):
        """Validate range and set value in instance."""
        if self.min_value is not None and value < self.min_value:
            raise ValueError(f"Value must be at least {self.min_value}")
        if self.max_value is not None and value > self.max_value:
            raise ValueError(f"Value must be at most {self.max_value}")
        instance.__dict__[self.name] = value

class Person:
    """A class using custom descriptors for validation."""

    name = TypedProperty('name', str)
    age = TypedProperty('age', int)
    age_range = RangeProperty('age_range', 0, 120)

    def __init__(self, name, age):
        """Initialize with name and age."""
        self.name = name
        self.age = age
        self.age_range = age # Uses range validation

# Create person

```

```

person = Person("Alice", 30)
print(f"Name: {person.name}, Age: {person.age}") # Name: Alice, Age: 30

# Type validation
try:
    person.name = 123 # Not a string
except TypeError as e:
    print(e) # Expected <class 'str'>, got <class 'int'>

# Range validation
try:
    person.age_range = -1 # Below minimum
except ValueError as e:
    print(e) # Value must be at least 0

try:
    person.age_range = 150 # Above maximum
except ValueError as e:
    print(e) # Value must be at most 120

# Valid assignments
person.name = "Bob"
person.age = 25
person.age_range = 25
print(f"Name: {person.name}, Age: {person.age}, Age range: {person.age_range}")
# Name: Bob, Age: 25, Age range: 25

# Data vs non-data descriptors
class NonDataDesc:
    """A non-data descriptor (only defines __get__)."""

    def __get__(self, instance, owner):
        return "NonDataDesc.__get__"

class DataDesc:
    """A data descriptor (defines __get__ and __set__)."""

    def __get__(self, instance, owner):
        return "DataDesc.__get__"

    def __set__(self, instance, value):
        print(f"DataDesc.__set__: {value}")

class Demo:
    """A class demonstrating descriptor lookup order."""

```

```

non_data = NonDataDesc()
data = DataDesc()

def __init__(self):
    # Instance attribute with same name as non-data descriptor
    self.non_data = "instance.non_data"
    # Instance attribute with same name as data descriptor
    self.data = "instance.data"

demo = Demo()
# Data descriptors take precedence over instance attributes
print(demo.data) # DataDesc.__get__
# Instance attributes take precedence over non-data descriptors
print(demo.non_data) # instance.non_data

```

Descriptors provide a powerful mechanism for controlling attribute access, enabling features like type checking, range validation, and computed attributes. They form the underlying implementation for properties, methods, and class-methods.

@classmethod, @staticmethod, and @property

Python provides several built-in decorators for changing the behavior of methods.

@classmethod Class methods are bound to the class rather than an instance. They receive the class as their first argument (conventionally named `cls`).

```

class Date:
    """A simple date class."""

    def __init__(self, year, month, day):
        """Initialize with year, month, and day."""
        self.year = year
        self.month = month
        self.day = day

    def __str__(self):
        """String representation."""
        return f"{self.year:04d}-{self.month:02d}-{self.day:02d}"

    @classmethod
    def from_string(cls, date_string):
        """Create a Date from a string in format YYYY-MM-DD."""
        year, month, day = map(int, date_string.split('-'))
        return cls(year, month, day)

    @classmethod

```



```

def today(cls):
    """Create a Date for today."""
    import datetime
    d = datetime.datetime.now()
    return cls(d.year, d.month, d.day)

@classmethod
def create_default(cls):
    """Create a default Date (2000-01-01)."""
    return cls(2000, 1, 1)

# Creating dates using different methods
date1 = Date(2023, 4, 15) # Regular initialization
date2 = Date.from_string("2023-05-20") # Using class method
date3 = Date.today() # Using class method to get current date
date4 = Date.create_default() # Using class method for default value

print(date1) # 2023-04-15
print(date2) # 2023-05-20
print(date3) # Current date (e.g., 2023-10-31)
print(date4) # 2000-01-01

# Inheritance with class methods
class EuropeanDate(Date):
    """A Date subclass that displays dates in DD/MM/YYYY format."""

    def __str__(self):
        """String representation."""
        return f"{self.day:02d}/{self.month:02d}/{self.year:04d}"

# The from_string class method works with the derived class
euro_date = EuropeanDate.from_string("2023-06-25")
print(euro_date) # 25/06/2023 (note the format change)

# Factory pattern with class methods
class Document:
    """Base document class."""

    @classmethod
    def create(cls, doc_type, *args, **kwargs):
        """Factory method to create document of specified type."""
        doc_types = {
            'text': TextDocument,
            'spreadsheet': SpreadsheetDocument,
            'presentation': PresentationDocument
        }

```

```

        if doc_type not in doc_types:
            raise ValueError(f"Unknown document type: {doc_type}")
        return doc_types[doc_type](*args, **kwargs)

class TextDocument(Document):
    """Text document class."""

    def __init__(self, content=""):
        self.content = content

    def __str__(self):
        return f"TextDocument({len(self.content)} chars)"

class SpreadsheetDocument(Document):
    """Spreadsheet document class."""

    def __init__(self, rows=0, cols=0):
        self.rows = rows
        self.cols = cols

    def __str__(self):
        return f"SpreadsheetDocument({self.rows}x{self.cols})"

class PresentationDocument(Document):
    """Presentation document class."""

    def __init__(self, slides=0):
        self.slides = slides

    def __str__(self):
        return f"PresentationDocument({self.slides} slides)"

# Create documents using factory method
doc1 = Document.create('text', "Hello, world!")
doc2 = Document.create('spreadsheet', 10, 5)
doc3 = Document.create('presentation', 20)

print(doc1)  # TextDocument(13 chars)
print(doc2)  # SpreadsheetDocument(10x5)
print(doc3)  # PresentationDocument(20 slides)

```

Class methods are commonly used for alternative constructors, factory methods, and operations that apply to the class as a whole rather than a specific instance.

@staticmethod Static methods don't operate on either the class or an instance. They're included in a class because they're related to the class conceptually.

```

class MathUtils:
    """A class containing math utility methods."""

    @staticmethod
    def is_prime(n):
        """Check if a number is prime."""
        if n <= 1:
            return False
        if n <= 3:
            return True
        if n % 2 == 0 or n % 3 == 0:
            return False
        i = 5
        while i * i <= n:
            if n % i == 0 or n % (i + 2) == 0:
                return False
            i += 6
        return True

    @staticmethod
    def gcd(a, b):
        """Calculate greatest common divisor."""
        while b:
            a, b = b, a % b
        return a

    @staticmethod
    def factorial(n):
        """Calculate factorial."""
        if n < 0:
            raise ValueError("Factorial is not defined for negative numbers")
        result = 1
        for i in range(2, n + 1):
            result *= i
        return result

# Using static methods
print(MathUtils.is_prime(17))    # True
print(MathUtils.is_prime(15))    # False
print(MathUtils.gcd(48, 18))     # 6
print(MathUtils.factorial(5))    # 120

# Static methods can also be called from instances
math_utils = MathUtils()
print(math_utils.is_prime(23))   # True
print(math_utils.gcd(56, 48))    # 8

```

```

# Static method in a class hierarchy
class Shape:
    """Base shape class."""

    @staticmethod
    def calculate_area(shape_type, *dimensions):
        """Calculate area for different shape types."""
        if shape_type == "circle":
            import math
            radius = dimensions[0]
            return math.pi * radius ** 2
        elif shape_type == "rectangle":
            length, width = dimensions
            return length * width
        elif shape_type == "triangle":
            base, height = dimensions
            return 0.5 * base * height
        else:
            raise ValueError(f"Unknown shape type: {shape_type}")

class Circle(Shape):
    """Circle class."""

    def __init__(self, radius):
        self.radius = radius

    def area(self):
        return Shape.calculate_area("circle", self.radius)

class Rectangle(Shape):
    """Rectangle class."""

    def __init__(self, length, width):
        self.length = length
        self.width = width

    def area(self):
        return Shape.calculate_area("rectangle", self.length, self.width)

# Using the static method directly and via instances
print(Shape.calculate_area("circle", 5))      # 78.53981633974483
print(Shape.calculate_area("rectangle", 4, 6)) # 24

circle = Circle(5)
rectangle = Rectangle(4, 6)

```

```
print(circle.area())      # 78.53981633974483
print(rectangle.area())  # 24
```

Static methods are useful for utility functions that are conceptually related to a class but don't need to access class or instance data. They improve code organization by keeping related functionality together.

@property (Covered Earlier) The `@property` decorator transforms a method into a read-only property (covered in detail in the “Properties and Descriptors” section).

```
class Circle:
    def __init__(self, radius):
        self._radius = radius

    @property
    def radius(self):
        return self._radius

    @radius.setter
    def radius(self, value):
        if value <= 0:
            raise ValueError("Radius must be positive")
        self._radius = value

    @property
    def area(self):
        import math
        return math.pi * self._radius ** 2

    @property
    def circumference(self):
        import math
        return 2 * math.pi * self._radius

# Using properties
circle = Circle(5)
print(circle.radius)      # 5
print(circle.area)        # 78.53981633974483
print(circle.circumference) # 31.41592653589793

# Setting radius
circle.radius = 7
print(circle.radius)      # 7
print(circle.area)        # 153.93804002589985
```

Properties allow for attribute-like access to methods, making it possible to add

validation or computation to attribute access.

Combining Decorators Decorators can be combined to create powerful patterns. Here's a class that uses `@classmethod`, `@staticmethod`, and `@property`:

```
class Temperature:
    """A class for temperature conversions with various constructors."""

    def __init__(self, celsius):
        """Initialize with temperature in Celsius."""
        self._celsius = celsius

    @property
    def celsius(self):
        """Get temperature in Celsius."""
        return self._celsius

    @celsius.setter
    def celsius(self, value):
        """Set temperature in Celsius, with validation."""
        if value < -273.15:
            raise ValueError("Temperature cannot be below absolute zero")
        self._celsius = value

    @property
    def fahrenheit(self):
        """Get temperature in Fahrenheit."""
        return self._celsius * 9/5 + 32

    @fahrenheit.setter
    def fahrenheit(self, value):
        """Set temperature in Fahrenheit, converting to Celsius."""
        self.celsius = (value - 32) * 5/9

    @property
    def kelvin(self):
        """Get temperature in Kelvin."""
        return self._celsius + 273.15

    @kelvin.setter
    def kelvin(self, value):
        """Set temperature in Kelvin, converting to Celsius."""
        self.celsius = value - 273.15

    @classmethod
    def from_fahrenheit(cls, fahrenheit):
```

```

        """Create a Temperature from a Fahrenheit value."""
        celsius = (fahrenheit - 32) * 5/9
        return cls(celsius)

    @classmethod
    def from_kelvin(cls, kelvin):
        """Create a Temperature from a Kelvin value."""
        celsius = kelvin - 273.15
        return cls(celsius)

    @staticmethod
    def is_valid_celsius(celsius):
        """Check if a Celsius temperature is physically possible."""
        return celsius >= -273.15

    @staticmethod
    def celsius_to_fahrenheit(celsius):
        """Convert Celsius to Fahrenheit."""
        return celsius * 9/5 + 32

    @staticmethod
    def fahrenheit_to_celsius(fahrenheit):
        """Convert Fahrenheit to Celsius."""
        return (fahrenheit - 32) * 5/9

# Create temperatures using different constructors
temp1 = Temperature(25)           # Regular constructor
temp2 = Temperature.from_fahrenheit(77) # Class method
temp3 = Temperature.from_kelvin(300)    # Class method

# Check values
print(f"temp1: {temp1.celsius}°C, {temp1.fahrenheit}°F, {temp1.kelvin}K")
# temp1: 25°C, 77.0°F, 298.15K
print(f"temp2: {temp2.celsius}°C, {temp2.fahrenheit}°F, {temp2.kelvin}K")
# temp2: 25.0°C, 77.0°F, 298.15K
print(f"temp3: {temp3.celsius}°C, {temp3.fahrenheit}°F, {temp3.kelvin}K")
# temp3: 26.850000000000023°C, 80.33000000000004°F, 300K

# Using properties to change values
temp1.fahrenheit = 68
print(f"temp1: {temp1.celsius}°C, {temp1.fahrenheit}°F, {temp1.kelvin}K")
# temp1: 20.0°C, 68.0°F, 293.15K

temp1.kelvin = 273.15
print(f"temp1: {temp1.celsius}°C, {temp1.fahrenheit}°F, {temp1.kelvin}K")
# temp1: 0°C, 32.0°F, 273.15K

```

```
# Using static methods
print(Temperature.is_valid_celsius(25))    # True
print(Temperature.is_valid_celsius(-300))  # False
print(Temperature.celsius_to_fahrenheit(0)) # 32.0
print(Temperature.fahrenheit_to_celsius(212)) # 100.0
```

Combining decorators allows for creating classes with rich functionality, including alternative constructors, utility methods, and controlled attribute access. This makes code more expressive, maintainable, and encapsulated.

Part II: Intermediate Python

4. Advanced Functional Programming

Decorators: Function and Class Decorators

Decorators are a powerful feature in Python that allow modifying the behavior of functions or classes without changing their source code. They follow the principle of “open for extension, closed for modification.”

Function Decorators Function decorators are functions that take another function as an argument and return a modified function.

```
# Basic decorator
def simple_decorator(func):
    """A simple decorator that prints messages before and after function execution."""
    def wrapper(*args, **kwargs):
        print("Before function call")
        result = func(*args, **kwargs)
        print("After function call")
        return result
    return wrapper

# Using the decorator with the @ syntax
@simple_decorator
def say_hello(name):
    """Say hello to someone."""
    print(f"Hello, {name}!")
    return name

# This is equivalent to: say_hello = simple_decorator(say_hello)

# Call the decorated function
say_hello("Alice")
# Output:
# Before function call
```



```

# Hello, Alice!
# After function call

# Decorator with arguments
def repeat(n=2):
    """A decorator that repeats the function call n times."""
    def decorator(func):
        def wrapper(*args, **kwargs):
            results = []
            for _ in range(n):
                results.append(func(*args, **kwargs))
            return results
        return wrapper
    return decorator

@repeat(3)
def greet(name):
    """Greet someone."""
    return f"Hello, {name}!"

# This is equivalent to: greet = repeat(3)(greet)

# Call the decorated function
print(greet("Bob"))
# Output: ['Hello, Bob!', 'Hello, Bob!', 'Hello, Bob!']

# Preserving function metadata with functools.wraps
from functools import wraps

def logging_decorator(func):
    """A decorator that logs function calls."""
    @wraps(func) # This preserves the original function's metadata
    def wrapper(*args, **kwargs):
        print(f"Calling {func.__name__} with args={args}, kwargs={kwargs}")
        result = func(*args, **kwargs)
        print(f"{func.__name__} returned {result}")
        return result
    return wrapper

@logging_decorator
def add(a, b):
    """Add two numbers."""
    return a + b

# Call the decorated function
add(2, 3)

```

```

# Output:
# Calling add with args=(2, 3), kwargs={}
# add returned 5

# Check that metadata is preserved
print(add.__name__) # add
print(add.__doc__)  # Add two numbers.

# Without @wraps, it would print:
# wrapper
# None

# Stacking decorators
def bold(func):
    """Wrap result in HTML bold tags."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        return f"<b>{func(*args, **kwargs)}</b>"
    return wrapper

def italic(func):
    """Wrap result in HTML italic tags."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        return f"<i>{func(*args, **kwargs)}</i>"
    return wrapper

@bold
@italic
def format_text(text):
    """Format text."""
    return text

# This is equivalent to: format_text = bold(italic(format_text))

# Call the decorated function
print(format_text("Hello, World!"))
# Output: <b><i>Hello, World!</i></b>

# Creating a parameterized decorator with or without arguments
def timer(func=None, *, log_level=None):
    """
    A decorator that times function execution.
    Can be used with or without arguments.
    """

```

```

def decorator(func):
    @wraps(func)
    def wrapper(*args, **kwargs):
        import time
        start_time = time.time()
        result = func(*args, **kwargs)
        end_time = time.time()
        elapsed = end_time - start_time

        message = f"{func.__name__} took {elapsed:.6f} seconds to run"
        if log_level == "debug":
            print(f"DEBUG: {message}")
        elif log_level == "info":
            print(f"INFO: {message}")
        else:
            print(message)

        return result
    return wrapper

# If called without arguments, func is provided directly
if func is not None:
    return decorator(func)

# If called with arguments, return the decorator function
return decorator

# Using the decorator without arguments
@timer
def slow_function():
    """A slow function."""
    import time
    time.sleep(0.5)
    return "Done"

# Using the decorator with arguments
@timer(log_level="debug")
def slower_function():
    """An even slower function."""
    import time
    time.sleep(1)
    return "Done"

# Call the decorated functions
slow_function()      # slow_function took 0.500610 seconds to run
slower_function()    # DEBUG: slower_function took 1.001287 seconds to run

```

```

# Real-world examples of decorators

# Caching/memoization
def memoize(func):
    """Cache the results of the function for specific arguments."""
    cache = {}

    @wraps(func)
    def wrapper(*args, **kwargs):
        # Create a key from the arguments
        # Note: kwargs need to be sorted for consistent keys
        key = str(args) + str(sorted(kwargs.items()))

        if key not in cache:
            cache[key] = func(*args, **kwargs)

        return cache[key]

    return wrapper

@memoize
def fibonacci(n):
    """Calculate the nth Fibonacci number."""
    if n <= 1:
        return n
    return fibonacci(n-1) + fibonacci(n-2)

# Fast even for large n
print(fibonacci(35)) # 9227465

# Retry decorator
def retry(max_attempts=3, delay=1):
    """Retry a function if it raises an exception."""
    def decorator(func):
        @wraps(func)
        def wrapper(*args, **kwargs):
            import time
            attempts = 0
            while attempts < max_attempts:
                try:
                    return func(*args, **kwargs)
                except Exception as e:
                    attempts += 1
                    if attempts == max_attempts:
                        raise e

```

```

        print(f"Attempt {attempts} failed: {e}. Retrying in {delay} seconds...")
        time.sleep(delay)

    return wrapper
return decorator

@retry(max_attempts=3, delay=0.5)
def unreliable_function():
    """A function that sometimes fails."""
    import random
    if random.random() < 0.7: # 70% chance of failing
        raise ValueError("Random failure")
    return "Success!"

# May succeed or may show retries
try:
    print(unreliable_function())
except ValueError as e:
    print(f"Final failure: {e}")

# Authentication decorator
def requires_auth(func):
    """A decorator that checks authentication."""
    @wraps(func)
    def wrapper(*args, **kwargs):
        # In a real app, you'd check a user session or token
        # This is just a simplified example
        auth_token = kwargs.pop('auth_token', None)
        if auth_token != "valid_token":
            raise PermissionError("Authentication required")
        return func(*args, **kwargs)
    return wrapper

@requires_auth
def get_sensitive_data():
    """Return sensitive data that requires authentication."""
    return "Sensitive data"

# This would fail
try:
    print(get_sensitive_data())
except PermissionError as e:
    print(e) # Authentication required

# This would succeed
try:
    print(get_sensitive_data(auth_token="valid_token")) # Sensitive data

```

```
except PermissionError as e:
    print(e)
```

Function decorators provide a powerful way to modify or enhance functions without changing their source code. They are widely used for cross-cutting concerns like logging, timing, authentication, and memoization.

Class Decorators Class decorators are functions that take a class as an argument and return a modified class.

```
# Basic class decorator
def add_greeting(cls):
    """Add a greet method to the class."""
    def greet(self, name):
        return f"Hello, {name}! I'm {self.__class__.__name__}."

    cls.greet = greet
    return cls

@add_greeting
class Person:
    def __init__(self, name, age):
        self.name = name
        self.age = age

# Create an instance
person = Person("Alice", 30)
print(person.greet("Bob")) # Hello, Bob! I'm Person.

# Decorator that adds class properties
def add_properties(cls):
    """Add properties to the class based on the __init__ parameters."""
    old_init = cls.__init__

    def new_init(self, *args, **kwargs):
        self._properties = {}
        old_init(self, *args, **kwargs)

    def get_prop(self, name):
        return self._properties.get(name)

    def set_prop(self, name, value):
        self._properties[name] = value

    cls.__init__ = new_init
    cls.get_property = get_prop
```

```

        cls.set_property = set_prop

    return cls

@add_properties
class User:
    def __init__(self, username, email):
        self.username = username
        self.email = email

    # Create an instance
    user = User("johndoe", "john@example.com")
    user.set_property("role", "admin")
    user.set_property("active", True)

    print(user.get_property("role"))    # admin
    print(user.get_property("active"))  # True

    # Singleton decorator
    def singleton(cls):
        """Make a class a Singleton class (only one instance)."""
        instances = {}

        def get_instance(*args, **kwargs):
            if cls not in instances:
                instances[cls] = cls(*args, **kwargs)
            return instances[cls]

        return get_instance

@singleton
class Logger:
    def __init__(self, name="default"):
        self.name = name
        self.logs = []

    def log(self, message):
        self.logs.append(message)
        print(f"[{self.name}] {message}")

    def get_logs(self):
        return self.logs

    # Create loggers
    logger1 = Logger("app")
    logger2 = Logger("system")

```

```

# They are the same instance
print(logger1 is logger2) # True
logger1.log("Test message")
print(logger2.get_logs()) # ['Test message']

# Registration decorator
registry = {}

def register(cls):
    """Register a class in the registry."""
    registry[cls.__name__] = cls
    return cls

@register
class Service1:
    """A service."""
    pass

@register
class Service2:
    """Another service."""
    pass

# Check the registry
print(registry) # {'Service1': <class '...Service1'>, 'Service2': <class '...Service2'>}

# Create a class from the registry
service = registry["Service1"]()
print(isinstance(service, Service1)) # True

# Validation decorator
def validate_attributes(**validators):
    """
    Validate attributes of a class instance.
    Each validator is a function that takes a value and returns a boolean.
    """
    def decorator(cls):
        old_init = cls.__init__

        def new_init(self, *args, **kwargs):
            old_init(self, *args, **kwargs)

            for attr_name, validator in validators.items():
                if hasattr(self, attr_name):
                    value = getattr(self, attr_name)

```



```

        if not validator(value):
            raise ValueError(f"Invalid value for {attr_name}: {value}")

    cls.__init__ = new_init
    return cls

    return decorator

# Define validators
def is_positive(value):
    return isinstance(value, (int, float)) and value > 0

def is_valid_email(value):
    return isinstance(value, str) and "@" in value

@validate_attributes(age=is_positive, email=is_valid_email)
class Employee:
    def __init__(self, name, age, email):
        self.name = name
        self.age = age
        self.email = email

# This should work
employee = Employee("Alice", 30, "alice@example.com")

# These should raise ValueError
try:
    employee = Employee("Bob", -5, "bob@example.com")
except ValueError as e:
    print(e) # Invalid value for age: -5

try:
    employee = Employee("Charlie", 40, "invalid-email")
except ValueError as e:
    print(e) # Invalid value for email: invalid-email

```

Class decorators provide a way to modify or enhance classes without subclassing or changing their source code. They are useful for adding functionality, validating attributes, implementing design patterns, and more.

Context Managers: `__enter__` and `__exit__`, `contextlib`, and `contextmanager`

Context managers provide a way to allocate and release resources when needed. They are typically used with the `with` statement.

Creating Context Managers with Class Methods

```
# Basic context manager using __enter__ and __exit__
class FileManager:
    """A context manager for file operations."""

    def __init__(self, filename, mode='r'):
        """Initialize with filename and mode."""
        self.filename = filename
        self.mode = mode
        self.file = None

    def __enter__(self):
        """Open the file and return it."""
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Close the file."""
        if self.file:
            self.file.close()
        # Return False to propagate exceptions, True to suppress
        return False

# Using the context manager
try:
    with FileManager('example.txt', 'w') as f:
        f.write('Hello, World!')

    with FileManager('example.txt', 'r') as f:
        content = f.read()
        print(content) # Hello, World!
except FileNotFoundError:
    print("File operation skipped (example.txt might not exist)")

# Context manager for database connections
class DatabaseConnection:
    """A context manager for database connections."""

    def __init__(self, connection_string):
        """Initialize with connection string."""
        self.connection_string = connection_string
        self.connection = None

    def __enter__(self):
        """Connect to the database and return the connection."""
```

```

        # This is a simplified example; in real code, you would use a database library
        print(f"Connecting to database: {self.connection_string}")
        self.connection = {"connection": "object", "status": "open"}
        return self.connection

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Close the database connection."""
        if self.connection:
            print("Closing database connection")
            self.connection["status"] = "closed"
            self.connection = None

        # If an exception occurred, log it
        if exc_type is not None:
            print(f"Exception occurred: {exc_type.__name__}: {exc_val}")

        # Return False to propagate the exception, True to suppress
        return False

# Using the context manager
with DatabaseConnection("postgresql://localhost/mydb") as conn:
    print(f"Connection status: {conn['status']}")
    # Perform database operations
    # ...

# Connection status: open
# Closing database connection

# Context manager with exception handling
class TimingContext:
    """A context manager for timing code execution."""

    def __init__(self, name="Code block"):
        """Initialize with an optional name."""
        self.name = name
        self.start_time = None

    def __enter__(self):
        """Start the timer."""
        import time
        self.start_time = time.time()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Calculate and print the elapsed time."""
        import time

```

```

        elapsed = time.time() - self.start_time

        if exc_type is not None:
            # An exception occurred
            print(f"{self.name} failed after {elapsed:.6f} seconds with {exc_type.__name__}")
        else:
            # No exception
            print(f"{self.name} completed in {elapsed:.6f} seconds")

        # Don't suppress exceptions
        return False

    def get_elapsed(self):
        """Get elapsed time so far."""
        import time
        return time.time() - self.start_time

# Using the context manager for successful execution
with TimingContext("Sleep operation") as timer:
    import time
    time.sleep(1)
    print(f"Elapsed so far: {timer.get_elapsed():.6f} seconds")
    time.sleep(0.5)

# Elapsed so far: 1.000544 seconds
# Sleep operation completed in 1.501106 seconds

# Using the context manager with an exception
try:
    with TimingContext("Division operation"):
        result = 1 / 0 # This will raise a ZeroDivisionError
except ZeroDivisionError:
    print("Exception was propagated")

# Division operation failed after 0.000012 seconds with ZeroDivisionError: division by zero
# Exception was propagated

# Nesting context managers
try:
    with FileManager('input.txt', 'r') as input_file:
        with FileManager('output.txt', 'w') as output_file:
            content = input_file.read()
            output_file.write(content.upper())
except FileNotFoundError:
    print("File operation skipped (input.txt or output.txt might not exist)")

```

Creating context managers with class methods is the traditional approach, providing full control over the entry and exit behavior.

Creating Context Managers with `contextlib` The `contextlib` module provides utilities for working with context managers, including a decorator for creating them.

```
import contextlib

# Using contextlib.contextmanager to create a context manager from a generator function
@contextlib.contextmanager
def file_manager(filename, mode='r'):
    """A context manager for file operations using contextlib."""
    try:
        # __enter__ equivalent
        file = open(filename, mode)
        yield file
    finally:
        # __exit__ equivalent
        file.close()

# Using the context manager
try:
    with file_manager('example.txt', 'w') as f:
        f.write('Hello from contextlib!')

    with file_manager('example.txt', 'r') as f:
        content = f.read()
        print(content) # Hello from contextlib!
except FileNotFoundError:
    print("File operation skipped (example.txt might not exist)")

# Using contextlib for temporary changes
@contextlib.contextmanager
def temp_attribute(obj, name, value):
    """Temporarily set an attribute on an object."""
    original = getattr(obj, name, None)
    setattr(obj, name, value)
    try:
        yield
    finally:
        if original is None:
            delattr(obj, name)
        else:
            setattr(obj, name, original)
```

```

# Using the context manager
class Person:
    def __init__(self, name):
        self.name = name

person = Person("Alice")
print(person.name) # Alice

with temp_attribute(person, 'name', 'Bob'):
    print(person.name) # Bob

print(person.name) # Alice

with temp_attribute(person, 'age', 30):
    print(person.age) # 30

# print(person.age) # AttributeError: 'Person' object has no attribute 'age'

# Using contextlib.suppress to suppress specific exceptions
import os

# Without contextlib.suppress
try:
    os.remove('nonexistent_file.txt')
except FileNotFoundError:
    pass # Ignore error if file doesn't exist

# With contextlib.suppress
with contextlib.suppress(FileNotFoundError):
    os.remove('nonexistent_file.txt') # Error is suppressed

# Using contextlib.redirect_stdout to redirect standard output
import sys
from io import StringIO

# Redirect stdout to a string buffer
stdout_buffer = StringIO()
with contextlib.redirect_stdout(stdout_buffer):
    print("This goes to the buffer")
    print("So does this")

# Get the buffered output
output = stdout_buffer.getvalue()
print(f"Captured: {output}") # Captured: This goes to the buffer\nSo does this\n

# Using contextlib.redirect_stderr to redirect standard error

```

```

stderr_buffer = StringIO()
with contextlib.redirect_stderr(stderr_buffer):
    sys.stderr.write("This is an error message\n")

error_output = stderr_buffer.getvalue()
print(f"Captured error: {error_output}") # Captured error: This is an error message\n

# Using contextlib.ExitStack to dynamically manage multiple context managers
@contextlib.contextmanager
def create_file(filename, content):
    """Create a file with the given content."""
    with open(filename, 'w') as f:
        f.write(content)
    try:
        yield filename
    finally:
        try:
            os.remove(filename)
        except FileNotFoundError:
            pass

# Using ExitStack to open multiple files based on a condition
def process_files(filenamees, process_all=True):
    with contextlib.ExitStack() as stack:
        files = []

        # dynamically build a list of context managers to enter
        for filename in filenamees:
            try:
                file = stack.enter_context(open(filename, 'r'))
                files.append(file)
            except FileNotFoundError:
                if process_all:
                    # If process_all is True, we need all files to exist
                    print(f"Error: {filename} not found")
                    return
                print(f"Warning: {filename} not found")

        # Process the files that were successfully opened
        for file in files:
            print(f"Processing {file.name}: {file.read().strip()}")

# Create some test files
try:
    with contextlib.ExitStack() as stack:
        stack.enter_context(create_file('test1.txt', 'Content of test1'))

```

```

stack.enter_context(create_file('test2.txt', 'Content of test2'))

# Try to process all files
process_files(['test1.txt', 'test2.txt', 'nonexistent.txt'], process_all=False)
# Warning: nonexistent.txt not found
# Processing test1.txt: Content of test1
# Processing test2.txt: Content of test2

# Try to process all files, requiring all to exist
process_files(['test1.txt', 'test2.txt', 'nonexistent.txt'], process_all=True)
# Error: nonexistent.txt not found
except Exception as e:
    print(f"Error: {e}")

```

The `contextlib` module provides a more concise way to create context managers, especially for simple cases. The `contextmanager` decorator allows you to create a context manager from a generator function with a single `yield` statement, which is often more readable than implementing the full `__enter__` and `__exit__` methods.

Real-World Examples of Context Managers

```

import contextlib
import time
import threading
import logging

# Example 1: Timing execution of code blocks
@contextlib.contextmanager
def timing_context(name=None):
    """Time the execution of a code block."""
    start_time = time.time()
    yield
    elapsed = time.time() - start_time
    name = name or "Code block"
    print(f"{name} took {elapsed:.6f} seconds")

def slow_function():
    """A slow function for demonstration."""
    time.sleep(1)

with timing_context("Slow function"):
    slow_function()
# Slow function took 1.001234 seconds

# Example 2: Thread-safe operations with a lock

```



```

class LockManager:
    """A context manager for thread locks."""

    def __init__(self, lock):
        self.lock = lock

    def __enter__(self):
        self.lock.acquire()
        return self

    def __exit__(self, exc_type, exc_val, exc_tb):
        self.lock.release()
        return False

# Using the lock manager
lock = threading.Lock()

def increment_counter(counter, lock_manager):
    with lock_manager:
        # This section is thread-safe
        current = counter["value"]
        time.sleep(0.1) # Simulate some processing
        counter["value"] = current + 1

# Create a simple counter
counter = {"value": 0}

# Create threads that increment the counter
threads = []
for _ in range(5):
    thread = threading.Thread(target=increment_counter, args=(counter, LockManager(lock)))
    threads.append(thread)
    thread.start()

# Wait for all threads to complete
for thread in threads:
    thread.join()

print(f"Counter value: {counter['value']}") # Should be 5

# Example 3: Logging context
@contextlib.contextmanager
def log_context(logger, level=logging.INFO, message_enter=None, message_exit=None):
    """Log messages at the beginning and end of a block."""
    if message_enter:
        logger.log(level, message_enter)

```

```

    try:
        yield
    except Exception as e:
        logger.exception(f"Exception occurred: {e}")
        raise
    finally:
        if message_exit:
            logger.log(level, message_exit)

# Configure logging
logging.basicConfig(level=logging.INFO, format='%(asctime)s - %(levelname)s - %(message)s')
logger = logging.getLogger(__name__)

# Using the log context
with log_context(logger, message_enter="Starting operation", message_exit="Operation completed"):
    logger.info("Doing something...")
    # Simulated operation
    time.sleep(0.5)

# Example 4: Temporary environment variable
@contextlib.contextmanager
def temp_env_var(name, value):
    """Temporarily set an environment variable."""
    import os
    old_value = os.environ.get(name)
    os.environ[name] = value
    try:
        yield
    finally:
        if old_value is None:
            del os.environ[name]
        else:
            os.environ[name] = old_value

# Using the environment variable context
import os
print(f"DEBUG before: {os.environ.get('DEBUG', 'Not set')}")

with temp_env_var("DEBUG", "True"):
    print(f"DEBUG during: {os.environ.get('DEBUG')}")

print(f"DEBUG after: {os.environ.get('DEBUG', 'Not set')}")

# Example 5: Transaction context for database operations
class Transaction:

```

```

"""A simplified database transaction context manager."""

def __init__(self, connection):
    self.connection = connection

def __enter__(self):
    print("Starting transaction")
    return self

def __exit__(self, exc_type, exc_val, exc_tb):
    if exc_type is not None:
        print(f"Rolling back transaction due to {exc_type.__name__}: {exc_val}")
        return False # Propagate the exception
    else:
        print("Committing transaction")
        return True

# Simulated database connection
db_connection = {"status": "connected"}

# Using the transaction context
with Transaction(db_connection):
    print("Executing query 1")
    print("Executing query 2")
    # All operations successful, transaction will commit

# Using the transaction context with an error
try:
    with Transaction(db_connection):
        print("Executing query 1")
        # Simulate an error
        raise ValueError("Invalid data")
        print("This won't be executed")
except ValueError:
    print("Transaction was rolled back and exception was re-raised")

```

Context managers are a powerful Python feature that helps manage resources correctly, even in the face of exceptions. They are widely used for file handling, database connections, locks, logging, and more.

Generators and Iterator Protocols

Generators are a simple way to create iterators. Instead of returning a single value, generators yield a series of values one at a time.

Generators

```

# Basic generator function with yield
def count_up_to(n):
    """Generate numbers from 1 to n."""
    i = 1
    while i <= n:
        yield i
        i += 1

# Using the generator
for num in count_up_to(5):
    print(num, end=' ') # 1 2 3 4 5

print() # Newline

# Generators are iterators
counter = count_up_to(3)
print(next(counter)) # 1
print(next(counter)) # 2
print(next(counter)) # 3
try:
    print(next(counter)) # StopIteration exception
except StopIteration:
    print("Generator exhausted")

# Generator expressions
squares = (x**2 for x in range(1, 6))
print(list(squares)) # [1, 4, 9, 16, 25]

# Generator function vs list comprehension: memory efficiency
import sys

# List comprehension (creates the entire list in memory)
numbers_list = [x for x in range(10000)]
print(f"List size: {sys.getsizeof(numbers_list)} bytes")

# Generator expression (generates values on demand)
numbers_gen = (x for x in range(10000))
print(f"Generator size: {sys.getsizeof(numbers_gen)} bytes")

# Infinite generators
def infinite_counter(start=0):
    """Generate an infinite sequence of numbers."""
    i = start
    while True:
        yield i
        i += 1

```

```

# Using islice to take a limited number of values from an infinite generator
from itertools import islice

counter = infinite_counter(10)
first_five = list(islice(counter, 5))
print(first_five)  # [10, 11, 12, 13, 14]

# Generators with multiple yield statements
def fibonacci_sequence(n):
    """Generate the first n Fibonacci numbers."""
    a, b = 0, 1
    count = 0
    while count < n:
        yield a
        a, b = b, a + b
        count += 1

fib_nums = list(fibonacci_sequence(10))
print(fib_nums)  # [0, 1, 1, 2, 3, 5, 8, 13, 21, 34]

# Generator with send() method
def echo_generator():
    """Echo back the values sent to the generator."""
    result = yield None  # Initial yield to start the generator
    while True:
        result = yield result  # Yield the received value

echo = echo_generator()
next(echo)  # Prime the generator
print(echo.send("Hello"))  # Hello
print(echo.send(42))      # 42
print(echo.send([1, 2, 3]))  # [1, 2, 3]

# Generator with throw() and close() methods
def resilient_generator():
    """A generator that can handle exceptions thrown into it."""
    try:
        i = 0
        while True:
            try:
                value = yield i
                print(f"Received: {value}")
                i += 1
            except ValueError:
                print("Caught ValueError, continuing...")

```

```

        i = 100 # Reset to a different value
    finally:
        print("Generator closing...")
        yield "Goodbye!" # This value will be returned by close()

gen = resilient_generator()
print(next(gen)) # 0
print(next(gen)) # 1
print(gen.throw(ValueError)) # Caught ValueError, continuing... 100
print(next(gen)) # 101
result = gen.close() # Generator closing...
print(f"Close result: {result}") # Close result: None

# yield from - delegating to another generator
def subgenerator():
    """A generator that yields three values."""
    yield 1
    yield 2
    yield 3
    return "SubGenerator done" # This value is returned, not yielded

def delegating_generator():
    """A generator that delegates to subgenerator()."""
    # Same as:
    # for x in subgenerator():
    #     yield x
    result = yield from subgenerator()
    print(f"Subgenerator returned: {result}")
    yield 4
    yield 5

# Using the delegating generator
for value in delegating_generator():
    print(value, end=' ') # 1 2 3 4 5
# Subgenerator returned: SubGenerator done

print() # Newline

# Complex example: Processing a large file with generators
def read_large_file(file_path, chunk_size=1024):
    """Read a large file in chunks."""
    with open(file_path, 'r') as f:
        while True:
            chunk = f.read(chunk_size)
            if not chunk:
                break

```

```

        yield chunk

def process_lines(chunk_gen):
    """Process chunks into lines."""
    # Buffer for partial lines
    partial_line = ""

    for chunk in chunk_gen:
        lines = (partial_line + chunk).split('\n')
        partial_line = lines.pop() # Last part might be incomplete

        for line in lines:
            yield line

    # Don't forget the last line if there's content in buffer
    if partial_line:
        yield partial_line

def grep(line_gen, pattern):
    """Filter lines that contain a pattern."""
    for line in line_gen:
        if pattern in line:
            yield line

# Example usage (if the file exists)
try:
    file_path = "example.txt"

    # Create a sample file
    with open(file_path, 'w') as f:
        f.write("Line 1: Hello\nLine 2: World\nLine 3: Python\nLine 4: Generators\n")

    # Process the file using generators
    chunks = read_large_file(file_path, chunk_size=10) # Small chunk size for demonstration
    lines = process_lines(chunks)
    matches = grep(lines, "Line")

    for match in matches:
        print(match)

    # Clean up
    import os
    os.remove(file_path)
except Exception as e:
    print(f"Error: {e}")

```

Generators provide a clean and memory-efficient way to work with sequences, especially large or infinite ones. They are particularly useful for processing large files, implementing iterators, and creating pipelines of data transformations.

Iterator Protocols The iterator protocol is the foundation for how loops work in Python. It consists of two special methods: `__iter__()` and `__next__()`.

```
# Custom iterator class
class Countdown:
    """An iterator that counts down from n to 1."""

    def __init__(self, start):
        """Initialize with starting number."""
        self.start = start

    def __iter__(self):
        """Return the iterator object (self)."""
        return self

    def __next__(self):
        """Return the next value or raise StopIteration."""
        if self.start <= 0:
            raise StopIteration

        current = self.start
        self.start -= 1
        return current

# Using the iterator
for num in Countdown(5):
    print(num, end=' ') # 5 4 3 2 1

print() # Newline

# Manual iteration
counter = Countdown(3)
print(next(counter)) # 3
print(next(counter)) # 2
print(next(counter)) # 1
try:
    print(next(counter)) # StopIteration exception
except StopIteration:
    print("Iterator exhausted")

# Making an iterable but not an iterator
class Range:
```



```

"""A simple recreation of Python's range."""

def __init__(self, start, stop=None, step=1):
    """Initialize with start, stop, and step."""
    if stop is None:
        self.start = 0
        self.stop = start
    else:
        self.start = start
        self.stop = stop

    self.step = step

def __iter__(self):
    """Return a new iterator each time."""
    return RangeIterator(self.start, self.stop, self.step)

def __len__(self):
    """Return the number of elements."""
    return max(0, (self.stop - self.start + self.step - 1) // self.step)

def __repr__(self):
    """Return a string representation."""
    if self.start == 0 and self.step == 1:
        return f"Range({self.stop})"
    elif self.step == 1:
        return f"Range({self.start}, {self.stop})"
    else:
        return f"Range({self.start}, {self.stop}, {self.step})"

class RangeIterator:
    """An iterator for the Range class."""

    def __init__(self, start, stop, step):
        """Initialize with start, stop, and step."""
        self.current = start
        self.stop = stop
        self.step = step

    def __iter__(self):
        """Return self as iterator."""
        return self

    def __next__(self):
        """Return the next value or raise StopIteration."""
        if (self.step > 0 and self.current >= self.stop) or \

```

```

        (self.step < 0 and self.current <= self.stop):
            raise StopIteration

        current = self.current
        self.current += self.step
        return current

# Using the custom Range class
for num in Range(5):
    print(num, end=' ') # 0 1 2 3 4

print() # Newline

for num in Range(2, 10, 2):
    print(num, end=' ') # 2 4 6 8

print() # Newline

# Creating multiple iterators from the same iterable
r = Range(3)
it1 = iter(r)
it2 = iter(r)

print(next(it1)) # 0
print(next(it1)) # 1
print(next(it2)) # 0 (independent iterator)

# Sentinel pattern with iter()
def read_until_sentinel(file_obj, sentinel):
    """Read lines from a file until the sentinel value is found."""
    return iter(file_obj.readline, sentinel)

# Example usage (if the file exists)
try:
    file_path = "example.txt"

    # Create a sample file
    with open(file_path, 'w') as f:
        f.write("Line 1\nLine 2\nEND\nLine 3\n")

    # Read until "END"
    with open(file_path, 'r') as f:
        for line in read_until_sentinel(f, "END\n"):
            print(line.strip())

    # Clean up

```

```

import os
os.remove(file_path)
except Exception as e:
    print(f"Error: {e}")

# Combining iterators
def chain_iterators(*iterables):
    """Chain multiple iterables together."""
    for iterable in iterables:
        for item in iterable:
            yield item

# Using the custom chain function
result = chain_iterators([1, 2], [3, 4, 5], [6])
print(list(result)) # [1, 2, 3, 4, 5, 6]

# Lazy evaluation with iterators
def is_prime(n):
    """Check if a number is prime."""
    if n <= 1:
        return False
    if n <= 3:
        return True
    if n % 2 == 0 or n % 3 == 0:
        return False
    i = 5
    while i * i <= n:
        if n % i == 0 or n % (i + 2) == 0:
            return False
        i += 6
    return True

def primes():
    """Generate prime numbers indefinitely."""
    n = 2
    while True:
        if is_prime(n):
            yield n
        n += 1

# Get the first 10 prime numbers
from itertools import islice
first_10_primes = list(islice(primes(), 10))
print(first_10_primes) # [2, 3, 5, 7, 11, 13, 17, 19, 23, 29]

# Implementing the iterable protocol for a binary tree

```

```

class Node:
    """A node in a binary tree."""

    def __init__(self, value, left=None, right=None):
        """Initialize with a value and optional left and right children."""
        self.value = value
        self.left = left
        self.right = right

class BinaryTree:
    """A binary tree that supports iteration in different orders."""

    def __init__(self, root=None):
        """Initialize with an optional root node."""
        self.root = root

    def __iter__(self):
        """Default to in-order traversal."""
        return self.inorder()

    def inorder(self):
        """In-order traversal: left, root, right."""
        def _inorder(node):
            if node:
                yield from _inorder(node.left)
                yield node.value
                yield from _inorder(node.right)

        return _inorder(self.root)

    def preorder(self):
        """Pre-order traversal: root, left, right."""
        def _preorder(node):
            if node:
                yield node.value
                yield from _preorder(node.left)
                yield from _preorder(node.right)

        return _preorder(self.root)

    def postorder(self):
        """Post-order traversal: left, right, root."""
        def _postorder(node):
            if node:
                yield from _postorder(node.left)
                yield from _postorder(node.right)

```

```

        yield node.value

    return _postorder(self.root)

def levelorder(self):
    """Level-order traversal (breadth-first)."""
    if not self.root:
        return

    queue = [self.root]
    while queue:
        node = queue.pop(0)
        yield node.value

        if node.left:
            queue.append(node.left)
        if node.right:
            queue.append(node.right)

# Create a sample binary tree
#
#      1
#     /\
#    2  3
#   /\  \
#  4  5  6
root = Node(1,
            Node(2,
                Node(4),
                Node(5)
            ),
            Node(3,
                None,
                Node(6)
            )
        )

tree = BinaryTree(root)

# Try different traversal orders
print("In-order:    ", list(tree.inorder()))    # [4, 2, 5, 1, 3, 6]
print("Pre-order:   ", list(tree.preorder()))   # [1, 2, 4, 5, 3, 6]
print("Post-order:  ", list(tree.postorder()))  # [4, 5, 2, 6, 3, 1]
print("Level-order:", list(tree.levelorder()))  # [1, 2, 3, 4, 5, 6]

# Default iteration uses in-order traversal
print("Default:     ", list(tree))              # [4, 2, 5, 1, 3, 6]

```

The iterator protocol is a fundamental part of Python, enabling a consistent way to iterate over different types of collections. By implementing `__iter__()` and `__next__()`, you can make custom classes work seamlessly with Python's iteration tools, including for loops, list comprehensions, and the various functions in the `itertools` module.

5. Data Classes and Validation

@dataclass, field, and Default Values

Data classes, introduced in Python 3.7, provide a concise way to create classes that are primarily used to store data. They automatically generate special methods like `__init__`, `__repr__`, and `__eq__`.

Basic Data Classes

```
# Basic data class usage
from dataclasses import dataclass

@dataclass
class Point:
    """A simple 2D point."""
    x: float
    y: float

# Create an instance
p = Point(1.0, 2.0)
print(p)  # Point(x=1.0, y=2.0)

# Auto-generated equality comparison
p1 = Point(1.0, 2.0)
p2 = Point(1.0, 2.0)
p3 = Point(3.0, 4.0)
print(p1 == p2)  # True
print(p1 == p3)  # False

# Mutable data class
@dataclass
class Rectangle:
    """A rectangle defined by width and height."""
    width: float
    height: float

    def area(self):
        """Calculate the area."""
        return self.width * self.height
```

```

# Create and use instance
rect = Rectangle(3.0, 4.0)
print(rect.area()) # 12.0

# Changing attributes
rect.width = 5.0
print(rect.area()) # 20.0

# Immutable data class
@dataclass(frozen=True)
class ImmutablePoint:
    """An immutable 2D point."""
    x: float
    y: float

# Create instance
ip = ImmutablePoint(1.0, 2.0)
try:
    ip.x = 3.0 # Raises FrozenInstanceError
except Exception as e:
    print(f"Error: {e}")

```

Data classes simplify the creation of classes that primarily store values, automatically generating common special methods.

Default Values and field

```

from dataclasses import dataclass, field, fields
import datetime

# Default values
@dataclass
class Product:
    """A product with name, price, and inventory."""
    name: str
    price: float = 0.0
    inventory: int = 0

    def total_value(self):
        """Calculate the total value of the inventory."""
        return self.price * self.inventory

# Create instances
p1 = Product("Widget")
p2 = Product("Gadget", 19.99, 100)
print(p1) # Product(name='Widget', price=0.0, inventory=0)

```

```

print(p2) # Product(name='Gadget', price=19.99, inventory=100)
print(p2.total_value()) # 1999.0

# Using field() for more control
@dataclass
class TaskInfo:
    """Information about a task."""
    id: int
    name: str
    description: str = ""
    created_at: datetime.datetime = field(default_factory=datetime.datetime.now)
    tags: list = field(default_factory=list)
    completed: bool = False
    metadata: dict = field(default_factory=dict)

# Create instances
task = TaskInfo(1, "Setup", "Initial setup")
print(task) # TaskInfo(id=1, name='Setup', description='Initial setup', created_at=..., ta

# Each instance gets its own list for tags
task1 = TaskInfo(1, "Task 1")
task2 = TaskInfo(2, "Task 2")
task1.tags.append("important")
print(task1.tags) # ['important']
print(task2.tags) # [] (not affected by task1's tags)

# Advanced field usage
@dataclass
class AdvancedExample:
    """Demonstrating advanced field options."""
    # Regular field
    name: str

    # Field with default value
    value: int = 0

    # Field with a factory function for default
    timestamp: datetime.datetime = field(default_factory=datetime.datetime.now)

    # Field excluded from __init__
    derived_value: int = field(init=False, default=0)

    # Field excluded from __repr__
    secret: str = field(repr=False, default="")

    # Field excluded from comparison

```



```

random_data: list = field(compare=False, default_factory=list)

# Field with custom metadata
metadata: dict = field(metadata={"description": "Additional metadata"}, default_factory=...)

def __post_init__(self):
    """Called after __init__ to set derived values."""
    self.derived_value = self.value * 2

# Create an instance
example = AdvancedExample(name="Example", value=10, secret="password123")
print(example) # AdvancedExample(name='Example', value=10, timestamp=..., derived_value=20)
# Note that 'secret' is not shown in the representation

# Examining fields
for f in fields(example):
    print(f"{f.name}: {f.type}, default: {f.default}, metadata: {f.metadata}")

# Inheritance with data classes
@dataclass
class Animal:
    """Base animal class."""
    name: str
    species: str

    def description(self):
        """Return a description."""
        return f"{self.name} is a {self.species}"

@dataclass
class Dog(Animal):
    """A dog class inheriting from Animal."""
    breed: str
    # Inherits name and species from Animal

    def __post_init__(self):
        """Called after __init__."""
        self.species = "Dog" # Always set species to "Dog"

# Create an instance
dog = Dog(name="Rex", species="ignored", breed="German Shepherd")
print(dog) # Dog(name='Rex', species='Dog', breed='German Shepherd')
print(dog.description()) # Rex is a Dog

```

The `field()` function provides fine-grained control over how fields behave in data classes, including default values, exclusion from various auto-generated

methods, and custom metadata.

Advanced Data Class Features

```
from dataclasses import dataclass, field, asdict, astuple, replace, InitVar, make_dataclass
import json
from typing import List, Dict, ClassVar, Optional

# Post-initialization processing
@dataclass
class Circle:
    """A circle with radius."""
    radius: float
    diameter: float = field(init=False)
    area: float = field(init=False)

    def __post_init__(self):
        """Called after __init__ to set derived values."""
        import math
        self.diameter = self.radius * 2
        self.area = math.pi * self.radius ** 2

# Create an instance
circle = Circle(5.0)
print(circle) # Circle(radius=5.0, diameter=10.0, area=78.53981633974483)

# InitVar fields (passed to __init__ and __post_init__ but not stored)
@dataclass
class Database:
    """A simple database connection."""
    name: str
    password: InitVar[str] # Only used during initialization
    user: str = "admin"
    connection_string: str = field(init=False)

    def __post_init__(self, password):
        """Called after __init__ with password."""
        # In a real app, you'd use a secure method for this
        self.connection_string = f"db://{self.user}:{password}@{self.name}"

# Create an instance
db = Database("mydb", "secret")
print(db) # Database(name='mydb', user='admin', connection_string='db://admin:secret@mydb')
# Note that 'password' is not stored as an instance attribute

# Class variables
```

```

@dataclass
class Config:
    """Configuration settings."""
    # Class variables (shared by all instances)
    VERSION: ClassVar[str] = "1.0.0"
    DEBUG: ClassVar[bool] = False

    # Instance variables
    name: str
    settings: Dict[str, str] = field(default_factory=dict)

    def get_version_info(self):
        """Return version information."""
        return f"{self.name} v{self.VERSION} (Debug: {self.DEBUG})"

# Create instances
config1 = Config("App1")
config2 = Config("App2")

# Class variables are shared
Config.DEBUG = True
print(config1.get_version_info()) # App1 v1.0.0 (Debug: True)
print(config2.get_version_info()) # App2 v1.0.0 (Debug: True)

# Instance variables are not shared
config1.settings["theme"] = "dark"
print(config1.settings) # {'theme': 'dark'}
print(config2.settings) # {}

# Converting to dictionary and tuple
@dataclass
class User:
    """A user with name, email, and roles."""
    name: str
    email: str
    roles: List[str] = field(default_factory=list)
    active: bool = True

# Create an instance
user = User("Alice", "alice@example.com", ["admin", "editor"])

# Convert to dictionary
user_dict = asdict(user)
print(user_dict) # {'name': 'Alice', 'email': 'alice@example.com', 'roles': ['admin', 'editor'], 'active': True}

# Convert to tuple

```

```

user_tuple = astuple(user)
print(user_tuple)  # ('Alice', 'alice@example.com', ['admin', 'editor'], True)

# Convert to JSON
user_json = json.dumps(asdict(user))
print(user_json)  # {"name": "Alice", "email": "alice@example.com", "roles": ["admin", "editor"]}

# Creating a new instance with changes
updated_user = replace(user, active=False, roles=["viewer"])
print(user)        # User(name='Alice', email='alice@example.com', roles=['admin', 'editor'], active=True)
print(updated_user) # User(name='Alice', email='alice@example.com', roles=['viewer'], active=True)

# Creating data classes at runtime
Person = make_dataclass('Person',
                        [('name', str),
                         ('age', int, field(default=0)),
                         ('email', Optional[str], None)],
                        namespace={'greeting': lambda self: f"Hello, my name is {self.name}"})

# Create an instance of the dynamically created class
person = Person("Bob", 30)
print(person)        # Person(name='Bob', age=30, email=None)
print(person.greeting()) # Hello, my name is Bob

# Using slots for memory efficiency
@dataclass(slots=True) # Python 3.10+
class Point3D:
    """A 3D point with memory-efficient storage."""
    x: float
    y: float
    z: float

    def distance_from_origin(self):
        """Calculate distance from origin."""
        return (self.x**2 + self.y**2 + self.z**2) ** 0.5

# Create an instance
p3d = Point3D(1.0, 2.0, 3.0)
print(p3d.distance_from_origin()) # 3.7416573867739413

# Memory usage comparison (for demonstration)
import sys

@dataclass
class RegularPoint:
    x: float

```

```

        y: float
        z: float

regular = RegularPoint(1.0, 2.0, 3.0)
slotted = Point3D(1.0, 2.0, 3.0)

print(f"Regular size: {sys.getsizeof(regular)} bytes")
print(f"Slotted size: {sys.getsizeof(slotted)} bytes")

```

Data classes provide a rich set of features for creating classes that store data, with support for customization, inheritance, and various utility functions.

Pydantic.BaseModel for Data Validation

Pydantic is a third-party library that provides data validation and settings management using Python type annotations. It's particularly popular for data validation in web applications.

Basic Pydantic Models

```

# Basic pydantic model
from pydantic import BaseModel, ValidationError, Field
from typing import List, Dict, Optional, Union
from datetime import datetime, date
import uuid

class User(BaseModel):
    """A user model with validation."""
    id: int
    name: str
    email: str
    age: Optional[int] = None
    is_active: bool = True
    tags: List[str] = []

# Create valid instances
try:
    user1 = User(id=1, name="Alice", email="alice@example.com")
    user2 = User(id=2, name="Bob", email="bob@example.com", age=30, tags=["customer", "premium"])

    print(user1) # id=1 name='Alice' email='alice@example.com' age=None is_active=True tags=[]
    print(user2) # id=2 name='Bob' email='bob@example.com' age=30 is_active=True tags=['customer', 'premium']
except ValidationError as e:
    print(f"Validation error: {e}")

# Invalid data raises ValidationError
try:

```

```

        # String for id instead of int
        User(id="1", name="Charlie", email="charlie@example.com")
except ValidationError as e:
    print(f"Validation error: {e}")

# Accessing data
print(user2.name)           # Bob
print(user2.dict())         # Full dictionary of values
print(user2.json())         # JSON string representation
print(user2.dict(exclude={"tags"})) # Dictionary excluding tags

# Using Field for more control
class Product(BaseModel):
    """A product model with Field constraints."""
    id: int = Field(..., gt=0, description="Unique product identifier")
    name: str = Field(..., min_length=1, max_length=100)
    price: float = Field(..., gt=0)
    description: Optional[str] = Field(None, max_length=1000)
    sku: str = Field(..., regex=r"^[A-Z]{2}-\d{6}$") # Format: XX-123456
    in_stock: bool = True
    created_at: datetime = Field(default_factory=datetime.now)

# Create a valid product
try:
    product = Product(
        id=1,
        name="Laptop",
        price=999.99,
        description="A high-performance laptop",
        sku="LP-123456"
    )
    print(product)
except ValidationError as e:
    print(f"Validation error: {e}")

# Invalid data
try:
    Product(
        id=0, # Must be > 0
        name="", # Too short
        price=-10, # Must be > 0
        sku="invalid" # Doesn't match regex
    )
except ValidationError as e:
    print(f"Validation error: {e}")

```

Pydantic provides robust data validation based on Python type annotations, making it easy to ensure data integrity in applications.

Advanced Pydantic Features

```
# Nested models
from pydantic import BaseModel, Field, EmailStr, validator, root_validator
from enum import Enum
from typing import List, Dict, Optional, Set
from datetime import datetime, date

class Role(str, Enum):
    """User roles as an enum."""
    ADMIN = "admin"
    EDITOR = "editor"
    VIEWER = "viewer"

class Address(BaseModel):
    """A postal address."""
    street: str
    city: str
    state: str
    zip_code: str
    country: str = "USA"

class User(BaseModel):
    """A user with nested address and validation."""
    id: int
    name: str
    email: EmailStr # Validates email format
    roles: Set[Role] = set() # Using enum for roles
    address: Optional[Address] = None
    created_at: datetime = Field(default_factory=datetime.now)

# Create a user with nested address
try:
    user = User(
        id=1,
        name="Alice",
        email="alice@example.com",
        roles=["admin", "editor"], # Automatically converted to enum values
        address={
            "street": "123 Main St",
            "city": "Springfield",
            "state": "IL",
            "zip_code": "62704"
        }
    )
```

```

    }
)
print(user)
except ValidationError as e:
    print(f"Validation error: {e}")

# Custom validators
class Order(BaseModel):
    """An order with custom validation."""
    id: str = Field(..., regex=r"~ORD-\d{8}$") # Format: ORD-12345678
    items: List[Dict[str, Union[str, float, int]]]
    customer_id: int
    total: float = 0.0
    status: str = "pending"

    @validator("status")
    def status_must_be_valid(cls, v):
        """Validate that status is one of the allowed values."""
        allowed = ["pending", "processing", "shipped", "delivered", "cancelled"]
        if v not in allowed:
            raise ValueError(f"Status must be one of {allowed}")
        return v

    @validator("items")
    def items_must_be_valid(cls, v):
        """Validate that each item has the required fields."""
        for item in v:
            if "product_id" not in item:
                raise ValueError("Each item must have a product_id")
            if "quantity" not in item:
                raise ValueError("Each item must have a quantity")
            if "price" not in item:
                raise ValueError("Each item must have a price")
        return v

    @root_validator
    def calculate_total(cls, values):
        """Calculate the total based on items."""
        items = values.get("items", [])
        total = sum(item.get("price", 0) * item.get("quantity", 0) for item in items)
        values["total"] = total
        return values

# Create a valid order
try:
    order = Order(

```



```

        id="ORD-12345678",
        customer_id=123,
        items=[
            {"product_id": "PROD-001", "quantity": 2, "price": 29.99},
            {"product_id": "PROD-002", "quantity": 1, "price": 49.99}
        ]
    )
    print(order)  # total will be calculated automatically
except ValidationError as e:
    print(f"Validation error: {e}")

# Invalid order (missing required item fields)
try:
    Order(
        id="ORD-12345678",
        customer_id=123,
        items=[
            {"product_id": "PROD-001"}  # Missing quantity and price
        ]
    )
except ValidationError as e:
    print(f"Validation error: {e}")

# Data type conversion
class Conversion(BaseModel):
    """Demonstrating automatic type conversion."""
    integer: int
    floating: float
    boolean: bool
    string_list: List[str]
    date_value: date
    datetime_value: datetime

# Pydantic tries to convert types when possible
try:
    conversion = Conversion(
        integer="123",  # String to int
        floating="3.14",  # String to float
        boolean="true",  # String to bool
        string_list="a,b,c",  # String to list (not always works as expected)
        date_value="2023-01-15",  # String to date
        datetime_value="2023-01-15T12:30:45"  # String to datetime
    )
    print(conversion)
except ValidationError as e:
    print(f"Validation error: {e}")

```

```

# Config and schema customization
class ConfigExample(BaseModel):
    """Example of config customization."""
    id: int
    name: str
    password: str

    class Config:
        # Schema customization
        title = "Configuration Example"
        description = "A model showing config options"

        # Behavior customization
        validate_assignment = True # Validate when attributes are assigned
        arbitrary_types_allowed = True # Allow arbitrary types
        extra = "forbid" # Forbid extra attributes

        # JSON Schema customization
        schema_extra = {
            "examples": [
                {
                    "id": 1,
                    "name": "Example User",
                    "password": "password123"
                }
            ]
        }

        # Serialization options
        json_encoders = {
            datetime: lambda dt: dt.isoformat()
        }

        # Security
        fields = {
            "password": {"exclude": True} # Exclude password from serialization
        }

# Create an instance
config_example = ConfigExample(id=1, name="Test", password="secret")
print(config_example.json()) # password is excluded
print(config_example.schema()) # Get JSON schema

# Try to set invalid value
try:

```

```

        config_example.id = "not an integer" # Should fail due to validate_assignment
    except ValidationError as e:
        print(f"Validation error: {e}")

# Try to add extra attributes
try:
    ConfigExample(id=1, name="Test", password="secret", extra_field="not allowed")
except ValidationError as e:
    print(f"Validation error: {e}")

```

Pydantic offers a comprehensive set of features for data validation, including nested models, custom validators, automatic type conversion, and flexible configuration options. It's particularly useful for API development, configuration management, and data processing pipelines.

6. Concurrency and Parallelism

Python provides several approaches to concurrent and parallel programming, each with its own strengths and use cases.

threading and multiprocessing

Threading The `threading` module allows for concurrent execution of code through threads, which are lighter weight than processes but limited by the Global Interpreter Lock (GIL) for CPU-bound tasks.

```

import threading
import time
import queue
import random

# Basic threading example
def worker(name):
    """A simple worker function."""
    print(f"Worker {name} starting")
    time.sleep(2) # Simulate work
    print(f"Worker {name} finished")

# Create and start threads
threads = []
for i in range(5):
    t = threading.Thread(target=worker, args=(i,))
    threads.append(t)
    t.start()

# Wait for all threads to complete
for t in threads:

```

```

t.join()

print("All workers finished")

# Thread with a class
class WorkerThread(threading.Thread):
    """A thread class."""

    def __init__(self, name):
        """Initialize with name."""
        super().__init__()
        self.name = name

    def run(self):
        """Override the run method for thread logic."""
        print(f"Thread {self.name} starting")
        time.sleep(random.uniform(1, 3)) # Simulate varied work time
        print(f"Thread {self.name} finished")

# Create and start thread objects
thread_objects = []
for i in range(3):
    t = WorkerThread(f"T{i}")
    thread_objects.append(t)
    t.start()

# Wait for all thread objects to complete
for t in thread_objects:
    t.join()

print("All thread objects finished")

# Passing data between threads with a Queue
def producer(q):
    """Produce items and put them in the queue."""
    for i in range(5):
        item = random.randint(1, 100)
        q.put(item)
        print(f"Produced: {item}")
        time.sleep(random.uniform(0.1, 0.5))

def consumer(q):
    """Consume items from the queue."""
    while True:
        item = q.get()
        if item is None: # Sentinel value to signal end

```

```

        break
    print(f"Consumed: {item}")
    q.task_done()
    time.sleep(random.uniform(0.5, 1))

# Create a queue
q = queue.Queue()

# Start producer and consumer threads
producer_thread = threading.Thread(target=producer, args=(q,))
consumer_thread = threading.Thread(target=consumer, args=(q,))

producer_thread.start()
consumer_thread.start()

# Wait for producer to finish
producer_thread.join()

# Signal consumer to exit
q.put(None)

# Wait for consumer to finish
consumer_thread.join()

print("Producer and consumer finished")

# Thread synchronization with Lock
counter = 0
counter_lock = threading.Lock()

def increment_counter(lock):
    """Increment a global counter with synchronization."""
    global counter
    for _ in range(100000):
        with lock: # Acquire and release the lock automatically
            counter += 1

# Create threads
threads = []
for _ in range(10):
    t = threading.Thread(target=increment_counter, args=(counter_lock,))
    threads.append(t)
    t.start()

# Wait for all threads
for t in threads:

```

```

t.join()

print(f"Counter value: {counter}") # Should be 1000000

# Other synchronization primitives

# RLock (Reentrant Lock)
rlock = threading.RLock()

def reentrant_function():
    """Demonstrate reentrant lock."""
    with rlock:
        print("Acquired lock once")
        # With a regular Lock, this would deadlock
        with rlock:
            print("Acquired lock twice")

reentrant_thread = threading.Thread(target=reentrant_function)
reentrant_thread.start()
reentrant_thread.join()

# Semaphore (limiting concurrent access)
semaphore = threading.Semaphore(3) # Allow 3 concurrent threads

def access_resource(thread_id):
    """Access a resource with limited concurrency."""
    with semaphore:
        print(f"Thread {thread_id} accessing resource")
        time.sleep(1)
        print(f"Thread {thread_id} releasing resource")

# Create threads
threads = []
for i in range(10):
    t = threading.Thread(target=access_resource, args=(i,))
    threads.append(t)
    t.start()

# Wait for all threads
for t in threads:
    t.join()

# Event (signaling between threads)
event = threading.Event()

def waiter(event):

```

```

        """Wait for an event to be set."""
        print("Waiter: Waiting for event")
        event.wait() # Block until event is set
        print("Waiter: Event received!")

def setter(event):
    """Set an event after a delay."""
    print("Setter: I'll set the event soon")
    time.sleep(2)
    print("Setter: Setting event now")
    event.set()

# Create threads
waiter_thread = threading.Thread(target=waiter, args=(event,))
setter_thread = threading.Thread(target=setter, args=(event,))

waiter_thread.start()
setter_thread.start()

waiter_thread.join()
setter_thread.join()

# Condition (more complex signaling)
condition = threading.Condition()
data = []

def consumer_condition():
    """Consume data when available."""
    with condition:
        print("Consumer: Waiting for data")
        condition.wait() # Wait for notification
        print(f"Consumer: Received data {data}")

def producer_condition():
    """Produce data and notify consumers."""
    with condition:
        print("Producer: Making data")
        time.sleep(1)
        data.append(random.randint(1, 100))
        print("Producer: Notifying")
        condition.notify() # Wake up a waiting thread

# Create threads
consumer_thread = threading.Thread(target=consumer_condition)
producer_thread = threading.Thread(target=producer_condition)

```

```

consumer_thread.start()
producer_thread.start()

consumer_thread.join()
producer_thread.join()

# Barrier (synchronization point for multiple threads)
barrier = threading.Barrier(3) # Wait for 3 threads

def barrier_worker(thread_id, barrier):
    """Worker that synchronizes at a barrier."""
    print(f"Thread {thread_id} before barrier")
    time.sleep(random.uniform(0.1, 1))

    # All threads wait here until 3 have arrived
    barrier.wait()

    print(f"Thread {thread_id} after barrier")

# Create threads
threads = []
for i in range(5): # Note: Barrier needs exactly 3, but we start 5
    t = threading.Thread(target=barrier_worker, args=(i, barrier))
    threads.append(t)
    t.start()

# Wait for all threads
for t in threads:
    t.join()

# Timer thread (delayed execution)
def delayed_task():
    """Task to run after a delay."""
    print("Delayed task executed")

timer = threading.Timer(2.0, delayed_task)
timer.start()
print("Timer started, waiting for execution...")
timer.join()

# Thread local storage
thread_local = threading.local()

def local_worker(thread_id):
    """Worker using thread-local storage."""
    thread_local.x = thread_id

```



```

        time.sleep(random.uniform(0.1, 0.5))
        print(f"Thread {thread_id}, local value: {thread_local.x}")

# Create threads
threads = []
for i in range(5):
    t = threading.Thread(target=local_worker, args=(i,))
    threads.append(t)
    t.start()

# Wait for all threads
for t in threads:
    t.join()

```

Threading is useful for I/O-bound tasks or when you need to maintain responsiveness in an application while performing background work. However, due to the Global Interpreter Lock (GIL) in CPython, threads are not suitable for CPU-intensive tasks that need true parallelism.

Multiprocessing The multiprocessing module allows for true parallel execution by using separate processes instead of threads, bypassing the GIL but with higher overhead for communication.

```

import multiprocessing as mp
import time
import random
import os

# Basic multiprocessing example
def worker(name):
    """A simple worker function."""
    print(f"Worker {name} (PID: {os.getpid()}) starting")
    time.sleep(1)
    print(f"Worker {name} finished")

# Using Process directly
processes = []
for i in range(5):
    p = mp.Process(target=worker, args=(i,))
    processes.append(p)
    p.start()

# Wait for all processes to complete
for p in processes:
    p.join()

```

```

print("All workers finished")

# Process with a class
class WorkerProcess(mp.Process):
    """A process class."""

    def __init__(self, name):
        """Initialize with name."""
        super().__init__()
        self.name = name

    def run(self):
        """Override the run method for process logic."""
        print(f"Process {self.name} (PID: {os.getpid()}) starting")
        time.sleep(random.uniform(1, 2))
        print(f"Process {self.name} finished")

# Create and start process objects
process_objects = []
for i in range(3):
    p = WorkerProcess(f"P{i}")
    process_objects.append(p)
    p.start()

# Wait for all process objects to complete
for p in process_objects:
    p.join()

print("All process objects finished")

# Passing data between processes with Queue
def producer_process(q):
    """Produce items and put them in the queue."""
    for i in range(5):
        item = random.randint(1, 100)
        q.put(item)
        print(f"Produced: {item}")
        time.sleep(random.uniform(0.1, 0.5))

def consumer_process(q):
    """Consume items from the queue."""
    while True:
        try:
            item = q.get(timeout=3) # Wait up to 3 seconds
            print(f"Consumed: {item}")
            time.sleep(random.uniform(0.5, 1))

```

```

        except Exception:
            # Queue.Empty or other error
            print("Consumer exiting")
            break

# Create a multiprocessing Queue
q = mp.Queue()

# Start producer and consumer processes
producer_proc = mp.Process(target=producer_process, args=(q,))
consumer_proc = mp.Process(target=consumer_process, args=(q,))

producer_proc.start()
consumer_proc.start()

# Wait for producer to finish
producer_proc.join()

# Wait for consumer to finish (or timeout)
consumer_proc.join(timeout=5)
if consumer_proc.is_alive():
    print("Terminating consumer")
    consumer_proc.terminate()

print("Producer and consumer processes finished")

# Sharing data with shared memory
def increment_value(value, lock):
    """Increment a shared value with synchronization."""
    for _ in range(100):
        with lock:
            value.value += 1
            time.sleep(0.01)

# Create shared memory objects
shared_value = mp.Value('i', 0) # 'i' for integer
lock = mp.Lock()

# Start processes
processes = []
for _ in range(5):
    p = mp.Process(target=increment_value, args=(shared_value, lock))
    processes.append(p)
    p.start()

# Wait for all processes

```

```

for p in processes:
    p.join()

print(f"Final shared value: {shared_value.value}") # Should be 500

# Shared array
def modify_array(array, index, value, lock):
    """Modify a specific index in a shared array."""
    with lock:
        array[index] = value

# Create a shared array
shared_array = mp.Array('i', [0] * 5) # Array of 5 integers
lock = mp.Lock()

# Start processes to modify the array
processes = []
for i in range(5):
    p = mp.Process(target=modify_array, args=(shared_array, i, i+10, lock))
    processes.append(p)
    p.start()

# Wait for all processes
for p in processes:
    p.join()

print(f"Final shared array: {list(shared_array)}") # Should be [10, 11, 12, 13, 14]

# Process Pool
def calculate_square(n):
    """Calculate the square of a number."""
    time.sleep(0.1) # Simulate computation
    return n * n

# Create a pool with 4 processes
with mp.Pool(processes=4) as pool:
    # Map function over data
    numbers = list(range(1, 11))
    results = pool.map(calculate_square, numbers)
    print(f"Squares: {results}")

    # Apply function to a single value
    result = pool.apply(calculate_square, (12,))
    print(f"12 squared: {result}")

    # Apply function asynchronously

```

```

async_result = pool.apply_async(calculate_square, (15,))
print(f"15 squared (async): {async_result.get()}")

# Map function asynchronously
async_map = pool.map_async(calculate_square, range(5))
print(f"Async map results: {async_map.get()}")

# Pool with imap (lazy evaluation)
def process_item(item):
    """Process a single item."""
    time.sleep(random.uniform(0.1, 0.5)) # Simulate varied processing time
    return item * 2

with mp.Pool(processes=3) as pool:
    # imap returns results as they're ready in original order
    for result in pool.imap(process_item, range(10)):
        print(f"imap result: {result}")

    # imap_unordered returns results as they're ready in any order
    for result in pool.imap_unordered(process_item, range(10)):
        print(f"imap_unordered result: {result}")

# Pipes for two-way communication
def ping(conn):
    """Send messages through a pipe."""
    conn.send("Hello")
    time.sleep(1)
    conn.send("World")
    time.sleep(1)
    conn.close()

def pong(conn):
    """Receive messages from a pipe."""
    while True:
        try:
            msg = conn.recv()
            print(f"Received: {msg}")
        except EOFError:
            print("Connection closed")
            break

# Create a pipe
parent_conn, child_conn = mp.Pipe()

# Start processes
ping_proc = mp.Process(target=ping, args=(parent_conn,))

```

```

pong_proc = mp.Process(target=pong, args=(child_conn,))

ping_proc.start()
pong_proc.start()

ping_proc.join()
pong_proc.join()

# Manager for more complex shared objects
def update_dict(d, key, value):
    """Update a shared dictionary."""
    d[key] = value
    print(f"Dict updated: {dict(d)}")

def update_list(lst, index, value):
    """Update a shared list."""
    lst[index] = value
    print(f"List updated: {list(lst)}")

# Create a manager
with mp.Manager() as manager:
    # Create shared objects
    shared_dict = manager.dict()
    shared_list = manager.list([0] * 5)

    # Start processes
    processes = []

    for i in range(5):
        p1 = mp.Process(target=update_dict, args=(shared_dict, f"key{i}", i*10))
        p2 = mp.Process(target=update_list, args=(shared_list, i, i*10))
        processes.extend([p1, p2])
        p1.start()
        p2.start()

    # Wait for all processes
    for p in processes:
        p.join()

    print(f"Final shared dict: {dict(shared_dict)}")
    print(f"Final shared list: {list(shared_list)}")

# Event for synchronization between processes
def waiter_process(event):
    """Wait for an event to be set."""
    print("Waiter: Waiting for event")

```

```

        event.wait() # Block until event is set
        print("Waiter: Event received!")

def setter_process(event):
    """Set an event after a delay."""
    print("Setter: I'll set the event soon")
    time.sleep(2)
    print("Setter: Setting event now")
    event.set()

# Create an event
event = mp.Event()

# Start processes
waiter_proc = mp.Process(target=waiter_process, args=(event,))
setter_proc = mp.Process(target=setter_process, args=(event,))

waiter_proc.start()
setter_proc.start()

waiter_proc.join()
setter_proc.join()

```

Multiprocessing provides true parallelism by using separate processes, making it suitable for CPU-bound tasks. However, it has higher overhead than threading due to the cost of creating processes and the more expensive communication between them.

concurrent.futures API

The `concurrent.futures` module provides a high-level interface for asynchronously executing functions using either threads or processes.

```

import concurrent.futures
import time
import random
import requests
import math
import os

# Basic ThreadPoolExecutor example
def task(n):
    """A simple task that takes n seconds."""
    print(f"Starting task {n}")
    time.sleep(n)
    print(f"Finished task {n}")
    return n * 2

```

```

# Using ThreadPoolExecutor to run tasks concurrently
with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
    # Submit individual tasks
    future1 = executor.submit(task, 1)
    future2 = executor.submit(task, 2)
    future3 = executor.submit(task, 3)

    # Get results as they complete
    for future in concurrent.futures.as_completed([future1, future2, future3]):
        try:
            result = future.result()
            print(f"Task returned: {result}")
        except Exception as e:
            print(f"Task raised an exception: {e}")

# Using map to apply a function to an iterable
def square(n):
    """Calculate the square of n."""
    time.sleep(0.5) # Simulate computation
    return n * n

with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
    numbers = [1, 2, 3, 4, 5]
    results = executor.map(square, numbers)

    # Results are returned in the same order as the input
    for num, result in zip(numbers, results):
        print(f"{num} squared is {result}")

# Downloading URLs with ThreadPoolExecutor
def download_url(url):
    """Download a URL and return its content length."""
    try:
        response = requests.get(url, timeout=10)
        print(f"Downloaded {url}: {len(response.content)} bytes")
        return len(response.content)
    except Exception as e:
        print(f"Error downloading {url}: {e}")
        return 0

urls = [
    "https://www.python.org/",
    "https://docs.python.org/",
    "https://pypi.org/",
    "https://peps.python.org/"

```



```
]
```

```
with concurrent.futures.ThreadPoolExecutor(max_workers=4) as executor:
    # Start the download tasks and get Future objects
    future_to_url = {executor.submit(download_url, url): url for url in urls}

    # Process results as they complete
    for future in concurrent.futures.as_completed(future_to_url):
        url = future_to_url[future]
        try:
            size = future.result()
            print(f"Size of {url}: {size} bytes")
        except Exception as e:
            print(f"Error processing {url}: {e}")

# ProcessPoolExecutor for CPU-bound tasks
def compute_factorial(n):
    """Compute the factorial of n."""
    print(f"Computing factorial of {n} in process {os.getpid()}")
    time.sleep(1) # Simulate computation
    return math.factorial(n)

with concurrent.futures.ProcessPoolExecutor(max_workers=4) as executor:
    numbers = [5, 10, 15, 20]

    # Submit tasks and keep track of futures
    futures = []
    for num in numbers:
        future = executor.submit(compute_factorial, num)
        futures.append((num, future))

    # Wait for all futures to complete and process results
    for num, future in futures:
        try:
            result = future.result()
            print(f"Factorial of {num}: {result}")
        except Exception as e:
            print(f"Error computing factorial of {num}: {e}")

# Mixing ThreadPoolExecutor and ProcessPoolExecutor
def io_bound_task(n):
    """Simulate an I/O-bound task."""
    print(f"Starting I/O task {n}")
    time.sleep(random.uniform(0.5, 2))
    print(f"Finished I/O task {n}")
    return f"I/O result {n}"
```

```

def cpu_bound_task(n):
    """Simulate a CPU-bound task."""
    print(f"Starting CPU task {n} in process {os.getpid()}")
    result = 0
    for i in range(10**7): # Perform a lot of calculations
        result += i
    print(f"Finished CPU task {n}")
    return f"CPU result {n}: {result}"

# Run I/O-bound tasks in threads
print("Running I/O-bound tasks in ThreadPoolExecutor")
with concurrent.futures.ThreadPoolExecutor(max_workers=5) as executor:
    io_futures = [executor.submit(io_bound_task, i) for i in range(5)]

    for future in concurrent.futures.as_completed(io_futures):
        print(future.result())

# Run CPU-bound tasks in processes
print("Running CPU-bound tasks in ProcessPoolExecutor")
with concurrent.futures.ProcessPoolExecutor(max_workers=3) as executor:
    cpu_futures = [executor.submit(cpu_bound_task, i) for i in range(3)]

    for future in concurrent.futures.as_completed(cpu_futures):
        print(future.result())

# Advanced features with futures
def long_running_task(name, duration, fail=False):
    """A task that runs for a specified duration."""
    print(f"Task {name} started")
    time.sleep(duration)

    if fail:
        raise ValueError(f"Task {name} failed")

    print(f"Task {name} completed")
    return f"Result from {name}"

with concurrent.futures.ThreadPoolExecutor(max_workers=3) as executor:
    # Submit tasks
    future_a = executor.submit(long_running_task, "A", 2)
    future_b = executor.submit(long_running_task, "B", 1)
    future_c = executor.submit(long_running_task, "C", 3, fail=True)

    # Add a callback function to a future
    def future_callback(future):

```

```

    try:
        result = future.result()
        print(f"Callback got result: {result}")
    except Exception as e:
        print(f"Callback caught exception: {e}")

future_a.add_done_callback(future_callback)
future_c.add_done_callback(future_callback)

# Wait for tasks with timeout
try:
    # Wait for all tasks to complete with a timeout
    done, not_done = concurrent.futures.wait(
        [future_a, future_b, future_c],
        timeout=2.5,
        return_when=concurrent.futures.ALL_COMPLETED
    )

    print(f"Completed tasks: {len(done)}")
    print(f"Pending tasks: {len(not_done)}")

    # Cancel any tasks that didn't complete
    for future in not_done:
        future.cancel()
        print(f"Cancelled task: {future}")

except KeyboardInterrupt:
    # Handle Ctrl+C
    print("Caught KeyboardInterrupt, cancelling tasks...")
    executor.shutdown(wait=False)
    for future in [future_a, future_b, future_c]:
        future.cancel()

```

The `concurrent.futures` module provides a higher-level interface for working with threads and processes, making it easier to run functions asynchronously and handle their results. It's particularly useful for tasks like parallel downloads, parallel computation, and managing task dependencies.

Comparison of Concurrency Models

```

import threading
import multiprocessing
import concurrent.futures
import time
import math
import os

```

```

# Define a CPU-bound task
def cpu_task(n):
    """A CPU-intensive task that calculates primes up to n."""
    count = 0
    for num in range(2, n + 1):
        is_prime = True
        for i in range(2, int(math.sqrt(num)) + 1):
            if num % i == 0:
                is_prime = False
                break
        if is_prime:
            count += 1
    return count

# Define an I/O-bound task
def io_task(n):
    """A task that simulates I/O operations."""
    print(f"Task {n} started")
    time.sleep(1) # Simulate I/O
    print(f"Task {n} completed")
    return n

# Run CPU-bound tasks sequentially
def run_sequential(func, tasks):
    """Run tasks sequentially."""
    start = time.time()
    results = [func(task) for task in tasks]
    end = time.time()
    return results, end - start

# Run CPU-bound tasks with threading
def run_threading(func, tasks):
    """Run tasks using threads."""
    start = time.time()

    threads = []
    results = [None] * len(tasks)

    def worker(i, task):
        results[i] = func(task)

    for i, task in enumerate(tasks):
        t = threading.Thread(target=worker, args=(i, task))
        threads.append(t)
        t.start()

```

```

        for t in threads:
            t.join()

        end = time.time()
        return results, end - start

# Run CPU-bound tasks with multiprocessing
def run_multiprocessing(func, tasks):
    """Run tasks using processes."""
    start = time.time()

    with multiprocessing.Pool() as pool:
        results = pool.map(func, tasks)

    end = time.time()
    return results, end - start

# Run CPU-bound tasks with concurrent.futures ThreadPoolExecutor
def run_thread_executor(func, tasks):
    """Run tasks using a thread pool executor."""
    start = time.time()

    with concurrent.futures.ThreadPoolExecutor() as executor:
        results = list(executor.map(func, tasks))

    end = time.time()
    return results, end - start

# Run CPU-bound tasks with concurrent.futures ProcessPoolExecutor
def run_process_executor(func, tasks):
    """Run tasks using a process pool executor."""
    start = time.time()

    with concurrent.futures.ProcessPoolExecutor() as executor:
        results = list(executor.map(func, tasks))

    end = time.time()
    return results, end - start

# Compare all models with CPU-bound tasks
def compare_cpu_bound():
    """Compare concurrency models with CPU-bound tasks."""
    print("\nComparing models with CPU-bound tasks:")

    tasks = [100000] * 8 # 8 tasks to find primes up to 100000

```

```

print("\nSequential execution:")
results, duration = run_sequential(cpu_task, tasks)
print(f"Time: {duration:.2f} seconds, Results: {results}")

print("\nThreading:")
results, duration = run_threading(cpu_task, tasks)
print(f"Time: {duration:.2f} seconds, Results: {results}")

print("\nMultiprocessing:")
results, duration = run_multiprocessing(cpu_task, tasks)
print(f"Time: {duration:.2f} seconds, Results: {results}")

print("\nThreadPoolExecutor:")
results, duration = run_thread_executor(cpu_task, tasks)
print(f"Time: {duration:.2f} seconds, Results: {results}")

print("\nProcessPoolExecutor:")
results, duration = run_process_executor(cpu_task, tasks)
print(f"Time: {duration:.2f} seconds, Results: {results}")

# Compare all models with I/O-bound tasks
def compare_io_bound():
    """Compare concurrency models with I/O-bound tasks."""
    print("\nComparing models with I/O-bound tasks:")

    tasks = list(range(8)) # 8 tasks with 1 second sleep each

    print("\nSequential execution:")
    results, duration = run_sequential(io_task, tasks)
    print(f"Time: {duration:.2f} seconds, Results: {results}")

    print("\nThreading:")
    results, duration = run_threading(io_task, tasks)
    print(f"Time: {duration:.2f} seconds, Results: {results}")

    print("\nMultiprocessing:")
    results, duration = run_multiprocessing(io_task, tasks)
    print(f"Time: {duration:.2f} seconds, Results: {results}")

    print("\nThreadPoolExecutor:")
    results, duration = run_thread_executor(io_task, tasks)
    print(f"Time: {duration:.2f} seconds, Results: {results}")

    print("\nProcessPoolExecutor:")
    results, duration = run_process_executor(io_task, tasks)

```

```

    print(f"Time: {duration:.2f} seconds, Results: {results}")

# Run the comparisons
if __name__ == "__main__":
    print("System Information:")
    print(f"CPU Count: {os.cpu_count()}")
    print(f"Python Version: {'.'.join(map(str, list(sys.version_info)[:3]))}")

    compare_cpu_bound()
    compare_io_bound()

```

The comparison of concurrency models illustrates the strengths and weaknesses of each approach:

1. **Threading:**

- Good for I/O-bound tasks where tasks spend time waiting for external operations.
- Limited by the Global Interpreter Lock (GIL) for CPU-bound tasks.
- Lower overhead compared to multiprocessing.

2. **Multiprocessing:**

- Excellent for CPU-bound tasks that need true parallelism.
- Bypasses the GIL by using separate processes.
- Higher overhead due to process creation and inter-process communication.

3. **concurrent.futures:**

- Provides a higher-level interface for both threading and multiprocessing.
- ThreadPoolExecutor is similar to threading for I/O-bound tasks.
- ProcessPoolExecutor is similar to multiprocessing for CPU-bound tasks.
- Easier to use with a more consistent API for both models.

The appropriate model depends on the nature of your tasks: - For I/O-bound tasks (network operations, file I/O), use threading or ThreadPoolExecutor. - For CPU-bound tasks (computation, data processing), use multiprocessing or ProcessPoolExecutor. - For a mix of tasks or when you need a consistent API for both types, use concurrent.futures and choose the appropriate executor.

Understanding these tradeoffs is crucial for writing efficient concurrent and parallel code in Python.

7. Type Hints and Annotations

Python 3.5+ introduced type hints, which provide a way to indicate the types of variables, function parameters, and return values. While Python remains dynamically typed, type hints enable static analysis tools to catch type-related bugs before runtime.

Basic Type Annotations

```
# Basic variable annotations
name: str = "Alice"
age: int = 30
height: float = 5.8
is_student: bool = False

# Function parameter and return annotations
def greet(name: str) -> str:
    """Greet a person by name."""
    return f"Hello, {name}!"

# Function with multiple parameters
def calculate_rectangle_area(length: float, width: float) -> float:
    """Calculate the area of a rectangle."""
    return length * width

# Functions without return values
def log_message(message: str) -> None:
    """Log a message to console."""
    print(f"LOG: {message}")

# Optional parameters with default values
def create_user(name: str, age: int = 18, email: str = None) -> dict:
    """Create a user dictionary."""
    user = {"name": name, "age": age}
    if email:
        user["email"] = email
    return user

# Type annotations with nested data structures
def process_user_data(user_id: int, data: dict) -> list:
    """Process user data and return results."""
    results = []
    for key, value in data.items():
        results.append(f"{user_id}: {key}={value}")
    return results

# Using the annotated functions
message = greet("Bob")
print(message) # Hello, Bob!

area = calculate_rectangle_area(3.5, 2.0)
print(area) # 7.0
```



```

log_message("Operation completed") # LOG: Operation completed

user = create_user("Charlie", 25, "charlie@example.com")
print(user) # {'name': 'Charlie', 'age': 25, 'email': 'charlie@example.com'}

results = process_user_data(1, {"name": "Alice", "score": 95})
print(results) # ['1: name=Alice', '1: score=95']

```

Basic type annotations provide a simple way to document the expected types of variables and function parameters/returns, which can make code more readable and help catch type-related errors early.

List, Dict, Union, Optional, TypeVar, Callable

The `typing` module provides more advanced type annotations for complex types.

```

from typing import List, Dict, Tuple, Set, Union, Optional, Any, Callable, TypeVar, Generic

# Container types
names: List[str] = ["Alice", "Bob", "Charlie"]
ages: Dict[str, int] = {"Alice": 30, "Bob": 25, "Charlie": 35}
coordinates: Tuple[float, float] = (3.5, 2.0)
tags: Set[str] = {"python", "programming", "tutorial"}

# Nested containers
matrix: List[List[int]] = [[1, 2, 3], [4, 5, 6], [7, 8, 9]]
student_grades: Dict[str, List[int]] = {
    "Alice": [85, 90, 95],
    "Bob": [75, 80, 85]
}

# Union types (variables that could be one of several types)
user_id: Union[int, str] = 123 # Could be int or str
user_id = "ABC123" # Still valid

# Optional types (variables that could be None)
email: Optional[str] = "user@example.com" # Same as Union[str, None]
email = None # Valid

# Any type (can be any type, disables type checking)
dynamic_value: Any = 42
dynamic_value = "Hello" # Valid
dynamic_value = [1, 2, 3] # Valid

# Functions with complex type annotations
def get_user_by_id(user_id: Union[int, str]) -> Optional[Dict[str, Any]]:
    """Get a user by ID, which could be an integer or string."""

```

```

    users = {
        123: {"name": "Alice", "email": "alice@example.com"},
        "ABC123": {"name": "Bob", "email": "bob@example.com"}
    }
    return users.get(user_id)

# Function with a callable parameter
def apply_operation(x: int, y: int, operation: Callable[[int, int], int]) -> int:
    """Apply a binary operation to two integers."""
    return operation(x, y)

def add(a: int, b: int) -> int:
    """Add two numbers."""
    return a + b

def multiply(a: int, b: int) -> int:
    """Multiply two numbers."""
    return a * b

# Using the function with different callables
result1 = apply_operation(3, 4, add)
print(result1) # 7

result2 = apply_operation(3, 4, multiply)
print(result2) # 12

result3 = apply_operation(3, 4, lambda a, b: a - b)
print(result3) # -1

# Using TypeVar for generic functions
T = TypeVar('T') # Define a type variable

def first_element(items: List[T]) -> Optional[T]:
    """Return the first element of a list, or None if empty."""
    return items[0] if items else None

# The function works with any type
first_str = first_element(["Alice", "Bob"]) # Type: Optional[str]
first_int = first_element([1, 2, 3]) # Type: Optional[int]

print(first_str) # Alice
print(first_int) # 1

# Generic classes
class Box(Generic[T]):
    """A generic box that can hold any type of value."""

```

```

def __init__(self, value: T) -> None:
    self.value = value

def get_value(self) -> T:
    return self.value

def set_value(self, value: T) -> None:
    self.value = value

# Create boxes with different types
int_box = Box[int](42)
str_box = Box[str]("Hello")

print(int_box.get_value()) # 42
print(str_box.get_value()) # Hello

# Type casting (for cases where you know more than the type checker)
def legacy_function(data):
    """Legacy function without type annotations."""
    return data

# Use cast to tell the type checker about the return type
data = legacy_function({1, 2, 3})
set_data = cast(Set[int], data) # No runtime effect, just for the type checker

# Complex function signature
def process_items(
    items: List[Union[int, str]],
    processors: Dict[str, Callable[[Union[int, str]], Any]],
    default_processor: Optional[Callable[[Union[int, str]], Any]] = None
) -> Dict[str, List[Any]]:
    """Process items using different processors based on type."""
    results: Dict[str, List[Any]] = {"int": [], "str": [], "other": []}

    for item in items:
        if isinstance(item, int) and "int" in processors:
            results["int"].append(processors["int"](item))
        elif isinstance(item, str) and "str" in processors:
            results["str"].append(processors["str"](item))
        elif default_processor:
            results["other"].append(default_processor(item))

    return results

# Using the complex function

```

```

processors = {
    "int": lambda x: x * 2,
    "str": lambda x: x.upper()
}

results = process_items([1, "hello", 3, "world"], processors)
print(results) # {'int': [2, 6], 'str': ['HELLO', 'WORLD'], 'other': []}

```

The `typing` module provides powerful tools for annotating complex types, which can help document your code, catch type errors early, and enable better IDE support.

Type Checking with mypy

`mypy` is a static type checker for Python that can catch type errors before runtime based on type annotations.

```

# Example file: example.py

def add(a: int, b: int) -> int:
    """Add two integers."""
    return a + b

def greet(name: str) -> str:
    """Greet a person by name."""
    return f"Hello, {name}!"

# Correct usage
x = add(1, 2)
greeting = greet("Alice")

# Type errors that mypy would catch
y = add("1", 2) # Error: "1" is not an int
z = add(1, "2") # Error: "2" is not an int
name = greet(123) # Error: 123 is not a str

# Function with more complex typing
from typing import List, Dict, Union, Optional

def process_data(
    data: List[Union[int, str]],
    filter_nulls: bool = True
) -> Dict[str, List[Union[int, str]]]:
    """Process a list of data items."""
    result = {"integers": [], "strings": []}

    for item in data:

```

```

        if item is None and filter_nulls:
            continue

        if isinstance(item, int):
            result["integers"].append(item)
        elif isinstance(item, str):
            result["strings"].append(item)

    return result

# Correct usage
result1 = process_data([1, "hello", None, 42, "world"])

# Type error that mypy would catch
result2 = process_data(123) # Error: 123 is not a List
result3 = process_data([1, 2, 3], filter_nulls="yes") # Error: "yes" is not a bool

```

To check a file with mypy, you'd run:

```
mypy example.py
```

And it would output something like:

```

example.py:17: error: Argument 1 to "add" has incompatible type "str"; expected "int"
example.py:18: error: Argument 2 to "add" has incompatible type "str"; expected "int"
example.py:19: error: Argument 1 to "greet" has incompatible type "int"; expected "str"
example.py:47: error: Argument 1 to "process_data" has incompatible type "int"; expected "List"
example.py:48: error: Argument "filter_nulls" to "process_data" has incompatible type "str"; expected "bool"

```

Advanced mypy Configuration For more complex projects, you can configure mypy using a mypy.ini file:

```

# mypy.ini
[mypy]
python_version = 3.9
warn_return_any = True
warn_unused_configs = True
disallow_untyped_defs = True
disallow_incomplete_defs = True
check_untyped_defs = True
disallow_untyped_decorators = True
no_implicit_optional = True
strict_optional = True

[mypy.plugins.numpy.*]
disallow_untyped_defs = False

[mypy-module_to_ignore.*]

```

```
ignore_missing_imports = True
disallow_untyped_defs = False
```

Type Stubs and stub Files For libraries without type annotations, you can use stub files (with a `.pyi` extension) to add type information. For example, if you have a module `legacy_module.py` without type annotations:

```
# legacy_module.py
def calculate_total(items):
    return sum(items)

def get_user_info(user_id):
    users = {1: {"name": "Alice", "email": "alice@example.com"}}
    return users.get(user_id)
```

You can create a stub file `legacy_module.pyi`:

```
# legacy_module.pyi
from typing import List, Dict, Any, Optional

def calculate_total(items: List[float]) -> float: ...
def get_user_info(user_id: int) -> Optional[Dict[str, str]]: ...
```

Third-Party Type Packages Many third-party libraries provide type stubs as separate packages. For example:

```
pip install types-requests # Type stubs for requests
pip install types-PyYAML   # Type stubs for PyYAML
```

Runtime Type Checking While mypy performs static type checking during development, you can also perform runtime type checking using libraries like `typeguard`:

```
from typeguard import typechecked
from typing import List, Dict

@typechecked
def sum_values(values: List[int]) -> int:
    return sum(values)

# This will work
print(sum_values([1, 2, 3])) # 6

# This will raise a TypeError at runtime
try:
    print(sum_values(["1", "2", "3"]))
except TypeError as e:
    print(f"Type error: {e}")
```

Benefits of Type Checking

1. **Catch errors early:** Find type-related bugs before running your code.
2. **Better documentation:** Type annotations serve as documentation of expected types.
3. **IDE support:** Modern IDEs use type hints for better autocompletion and error detection.
4. **Safer refactoring:** Makes it easier to change code without introducing type errors.
5. **Gradual adoption:** You can add type hints gradually to your codebase.

Type hints and static type checking have become an important part of Python development, especially for larger projects and teams, helping to improve code quality and maintainability.

Part III: Advanced Python

8. Asynchronous Programming

Asynchronous programming allows for concurrent execution without using threads or processes, making it ideal for I/O-bound operations like network requests and file operations.

`async def`, `await`, `async for`, and `async with`

Python's asynchronous features use the `asyncio` module along with keywords like `async` and `await` to define and work with coroutines.

```
import asyncio
import aiohttp
import time
import random
from contextlib import asynccontextmanager

# Basic coroutine definition with async def
async def hello_world():
    """A simple coroutine that returns a greeting."""
    return "Hello, World!"

# Using await to get the result of a coroutine
async def main1():
    """Run the hello_world coroutine and print the result."""
    result = await hello_world()
    print(result) # Hello, World!

# Running a coroutine with asyncio.run()
asyncio.run(main1())
```

```

# Coroutines with delay
async def delayed_greeting(name, delay):
    """Return a greeting after a delay."""
    print(f"Waiting for {delay} seconds before greeting {name}...")
    await asyncio.sleep(delay) # Non-blocking sleep
    return f"Hello, {name}!"

# Running multiple coroutines concurrently
async def main2():
    """Run multiple coroutines concurrently and gather their results."""
    start = time.time()

    # Create coroutine objects
    alice = delayed_greeting("Alice", 2)
    bob = delayed_greeting("Bob", 1)
    charlie = delayed_greeting("Charlie", 3)

    # Wait for all coroutines to complete
    results = await asyncio.gather(alice, bob, charlie)

    end = time.time()

    # Print results
    for result in results:
        print(result)

    print(f"Total time: {end - start:.2f} seconds") # ~3 seconds, not 6

# Running the second main coroutine
asyncio.run(main2())

# Asynchronous HTTP requests with aiohttp
async def fetch_url(session, url):
    """Fetch the content of a URL asynchronously."""
    print(f"Fetching {url}...")
    async with session.get(url) as response:
        return await response.text()

async def main3():
    """Fetch multiple URLs concurrently."""
    urls = [
        "https://www.python.org",
        "https://docs.python.org",
        "https://pypi.org"
    ]

```



```

start = time.time()

async with aiohttp.ClientSession() as session:
    tasks = [fetch_url(session, url) for url in urls]
    results = await asyncio.gather(*tasks)

    for i, result in enumerate(results):
        print(f"URL {urls[i]}: {len(result)} bytes")

end = time.time()
print(f"Total time: {end - start:.2f} seconds")

# Running the third main coroutine
try:
    asyncio.run(main3())
except ImportError:
    print("aiohttp not available. Skipping URL fetching example.")

# Creating an asynchronous context manager
@asynccontextmanager
async def timer():
    """An asynchronous context manager that times a block of code."""
    start = time.time()
    try:
        yield # Control returns to the user code
    finally:
        end = time.time()
        print(f"Elapsed time: {end - start:.2f} seconds")

# Using async with
async def main4():
    """Use an asynchronous context manager."""
    async with timer():
        # Simulate some asynchronous work
        await asyncio.sleep(2)
        print("Work completed inside async with")

# Running the fourth main coroutine
asyncio.run(main4())

# Creating an asynchronous iterator
class AsyncCounter:
    """An asynchronous iterator that counts up to n."""

    def __init__(self, n):
        self.n = n

```

```

        self.i = 0

    def __aiter__(self):
        return self

    async def __anext__(self):
        if self.i >= self.n:
            raise StopAsyncIteration

        self.i += 1
        await asyncio.sleep(random.random()) # Simulate async work
        return self.i - 1

# Using async for
async def main5():
    """Use an asynchronous iterator with async for."""
    print("Counting asynchronously:")
    async for i in AsyncCounter(5):
        print(i)

# Running the fifth main coroutine
asyncio.run(main5())

# Working with asyncio.TaskGroup (Python 3.11+)
async def main6():
    """Use TaskGroup to run multiple coroutines concurrently."""
    async def worker(name, delay):
        await asyncio.sleep(delay)
        return f"Worker {name} done after {delay} seconds"

    try:
        # Try to use TaskGroup if available (Python 3.11+)
        if hasattr(asyncio, "TaskGroup"):
            async with asyncio.TaskGroup() as tg:
                task1 = tg.create_task(worker("A", 2))
                task2 = tg.create_task(worker("B", 1))
                task3 = tg.create_task(worker("C", 3))

            print(task1.result())
            print(task2.result())
            print(task3.result())
        else:
            # Fallback for earlier Python versions
            tasks = [
                asyncio.create_task(worker("A", 2)),
                asyncio.create_task(worker("B", 1)),

```

```

        asyncio.create_task(worker("C", 3))
    ]

    for task in tasks:
        result = await task
        print(result)
    except Exception as e:
        print(f"Error: {e}")

# Running the sixth main coroutine
asyncio.run(main6())

```

Asynchronous programming in Python allows for efficient handling of I/O-bound tasks by enabling concurrent execution without the overhead of threads or processes. It uses a cooperative multitasking model where tasks yield control explicitly.

Asynchronous Context Managers and Generators

Python provides mechanisms for creating asynchronous context managers and generators, enabling efficient resource management in asynchronous code.

```

import asyncio
import aiofiles
import contextlib
import time
import random

# Asynchronous context manager with __aenter__ and __aexit__
class AsyncResource:
    """A class that implements the asynchronous context manager protocol."""

    def __init__(self, name):
        self.name = name

    async def __aenter__(self):
        """Asynchronously acquire the resource."""
        print(f"Acquiring {self.name}...")
        await asyncio.sleep(1) # Simulate async acquisition
        print(f"{self.name} acquired")
        return self

    async def __aexit__(self, exc_type, exc_val, exc_tb):
        """Asynchronously release the resource."""
        print(f"Releasing {self.name}...")
        await asyncio.sleep(0.5) # Simulate async release
        print(f"{self.name} released")

```

```

        return False # Don't suppress exceptions

# Using the asynchronous context manager
async def use_resource():
    """Use an asynchronous context manager."""
    async with AsyncResource("Database") as db:
        print(f"Using {db.name}")
        await asyncio.sleep(2) # Simulate async work
        print(f"Done with {db.name}")

# Asynchronous context manager with contextlib.asynccontextmanager
@contextlib.asynccontextmanager
async def async_timer(name):
    """An asynchronous context manager for timing code blocks."""
    start = time.time()
    try:
        print(f"Starting {name}...")
        yield # Control returns to the user code
    finally:
        end = time.time()
        print(f"{name} completed in {end - start:.2f} seconds")

# Using the asynccontextmanager-based context manager
async def timed_operation():
    """Perform a timed operation using an async context manager."""
    async with async_timer("Long operation"):
        await asyncio.sleep(3)
        print("Operation in progress...")
        await asyncio.sleep(2)

# Asynchronous file I/O with aiofiles
async def read_file(filename):
    """Read a file asynchronously."""
    try:
        async with aiofiles.open(filename, 'r') as f:
            content = await f.read()
            return content
    except FileNotFoundError:
        print(f"File {filename} not found")
        return None

async def write_file(filename, content):
    """Write to a file asynchronously."""
    try:
        async with aiofiles.open(filename, 'w') as f:
            await f.write(content)

```

```

        return True
    except Exception as e:
        print(f"Error writing to {filename}: {e}")
        return False

# Asynchronous generator
async def async_range(start, stop):
    """An asynchronous generator that yields numbers with random delays."""
    for i in range(start, stop):
        await asyncio.sleep(random.random()) # Random delay
        yield i

# Using an asynchronous generator
async def use_async_generator():
    """Use an asynchronous generator with async for."""
    print("Starting async iteration")
    async for i in async_range(1, 6):
        print(f"Got value: {i}")
    print("Async iteration complete")

# Asynchronous generator with try/finally
async def fetch_with_retry(urls, max_retries=3):
    """Fetch URLs with retry logic, yielding results as they complete."""
    import aiohttp

    try:
        async with aiohttp.ClientSession() as session:
            for url in urls:
                retries = 0
                while retries < max_retries:
                    try:
                        print(f"Fetching {url}, attempt {retries + 1}...")
                        async with session.get(url, timeout=5) as response:
                            content = await response.text()
                            yield url, content
                            break # Success, break retry loop
                    except (aiohttp.ClientError, asyncio.TimeoutError) as e:
                        retries += 1
                        if retries < max_retries:
                            delay = 0.5 * (2 ** retries) # Exponential backoff
                            print(f"Error fetching {url}: {e}. Retrying in {delay:.2f} seconds")
                            await asyncio.sleep(delay)
                        else:
                            print(f"Failed to fetch {url} after {max_retries} attempts")
                            yield url, None # Yield None for failed URLs
    except ImportError:

```

```

        print("aiohttp not available")
        yield None, None

# Using the retry generator
async def download_urls():
    """Download URLs using an asynchronous generator with retry logic."""
    urls = [
        "https://www.python.org",
        "https://invalid.example.com", # Will fail
        "https://docs.python.org"
    ]

    try:
        async for url, content in fetch_with_retry(urls):
            if content:
                print(f"Successfully downloaded {url}: {len(content)} bytes")
            else:
                print(f"Failed to download {url}")
    except ImportError:
        print("URL downloading example skipped due to missing aiohttp")

# Nesting asynchronous context managers
async def nested_async_contexts():
    """Demonstrate nesting multiple asynchronous context managers."""
    async with async_timer("Outer operation"):
        print("In the outer context")

        # Nested context
        async with async_timer("Inner operation"):
            print("In the inner context")
            await asyncio.sleep(1)

        print("Back to the outer context")
        await asyncio.sleep(2)

# Example combining multiple asynchronous features
async def advanced_example():
    """Combine various asynchronous features."""
    # Create a temporary file
    filename = "temp_async_test.txt"

    # Write to the file
    try:
        print(f"Writing to {filename}")
        success = await write_file(filename, "Hello, async world!\n" * 100)

```

```

    if success:
        # Use context manager and generator together
        async with async_timer("File processing"):
            content = await read_file(filename)

            if content:
                lines = content.split('\n')
                print(f"File has {len(lines)} lines")

                # Process lines with async generator
                async for i in async_range(0, min(5, len(lines))):
                    print(f"Line {i}: {lines[i][:30]}...")
                    await asyncio.sleep(0.5)
    except ImportError:
        print("aiofiles not available, skipping file operations")
    finally:
        # Clean up (in a real app, you would use async file operations)
        import os
        if os.path.exists(filename):
            os.remove(filename)
            print(f"Removed {filename}")

# Main function to run all examples
async def main():
    """Run all the asynchronous examples."""
    # Use a basic async context manager
    print("\n--- Using AsyncResource ---")
    await use_resource()

    # Use a timer context manager
    print("\n--- Using async_timer ---")
    await timed_operation()

    # Use an async generator
    print("\n--- Using async_range generator ---")
    await use_async_generator()

    # Download URLs with retry
    print("\n--- Downloading URLs with retry ---")
    await download_urls()

    # Test nested contexts
    print("\n--- Testing nested async contexts ---")
    await nested_async_contexts()

    # Run the advanced example

```

```

    print("\n--- Running advanced example ---")
    await advanced_example()

# Run everything
try:
    asyncio.run(main())
except ImportError as e:
    print(f"Some examples skipped due to missing dependencies: {e}")

```

Asynchronous context managers and generators are powerful tools for resource management and data generation in asynchronous code. They allow for efficient handling of resources and sequential data production without blocking the event loop.

Additional Asynchronous Patterns

Here are more examples of advanced asynchronous patterns:

```

import asyncio
import random
from typing import List, Dict, Any, Optional
import time

# Pattern 1: Task cancellation
async def background_task(name: str) -> None:
    """A long-running background task that can be cancelled."""
    try:
        print(f"Task {name} started")
        iteration = 0
        while True:
            iteration += 1
            await asyncio.sleep(1)
            print(f"Task {name}: iteration {iteration}")
    except asyncio.CancelledError:
        print(f"Task {name} was cancelled")
        raise # Re-raise to properly handle cancellation

async def task_cancellation_demo() -> None:
    """Demonstrate how to cancel tasks."""
    # Create and start a background task
    task = asyncio.create_task(background_task("Demo"))

    # Let it run for a while
    await asyncio.sleep(3)

    # Cancel the task
    print("Cancelling the task...")

```



```

task.cancel()

try:
    await task # Wait for the task to be cancelled
except asyncio.CancelledError:
    print("Task cancellation confirmed")

# Pattern 2: Timeouts
async def slow_operation() -> str:
    """A slow operation that might take too long."""
    delay = random.uniform(1, 5)
    print(f"Slow operation starting, will take {delay:.2f} seconds")
    await asyncio.sleep(delay)
    return f"Completed after {delay:.2f} seconds"

async def timeout_demo() -> None:
    """Demonstrate how to handle timeouts."""
    try:
        # Wait for at most 2 seconds
        result = await asyncio.wait_for(slow_operation(), timeout=2.0)
        print(f"Operation successful: {result}")
    except asyncio.TimeoutError:
        print("Operation timed out")

# Pattern 3: Gathering results as they complete
async def fetch_data(id: int) -> Dict[str, Any]:
    """Simulate fetching data from a remote source."""
    delay = random.uniform(0.5, 3.0)
    await asyncio.sleep(delay)
    return {"id": id, "value": f"Data-{id}", "delay": delay}

async def as_completed_demo() -> None:
    """Demonstrate processing results as they complete."""
    # Create tasks for multiple data fetches
    tasks = [fetch_data(i) for i in range(1, 6)]

    # Process results as they complete
    for future in asyncio.as_completed(tasks):
        result = await future
        print(f"Received result for id={result['id']} after {result['delay']:.2f}s: {result}")

# Pattern 4: Limiting concurrency with a semaphore
async def limited_concurrency_worker(id: int, semaphore: asyncio.Semaphore) -> str:
    """A worker that respects a concurrency limit."""
    async with semaphore:
        print(f"Worker {id} acquired semaphore")

```

```

        await asyncio.sleep(random.uniform(1, 3))
        print(f"Worker {id} releasing semaphore")
        return f"Result from worker {id}"

async def semaphore_demo() -> None:
    """Demonstrate limiting concurrency with a semaphore."""
    # Allow only 2 workers at a time
    semaphore = asyncio.Semaphore(2)

    # Create tasks for 5 workers
    tasks = [limited_concurrency_worker(i, semaphore) for i in range(1, 6)]

    # Wait for all tasks to complete
    results = await asyncio.gather(*tasks)
    print(f"All workers completed: {results}")

# Pattern 5: Producer-consumer with queue
async def producer(queue: asyncio.Queue, items: int) -> None:
    """Produce items and put them in the queue."""
    for i in range(items):
        item = f"Item-{i}"
        await queue.put(item)
        print(f"Produced {item}")
        await asyncio.sleep(random.uniform(0.1, 0.5))

    # Signal end of production
    await queue.put(None)
    print("Producer finished")

async def consumer(queue: asyncio.Queue, name: str) -> int:
    """Consume items from the queue until receiving None."""
    count = 0
    while True:
        item = await queue.get()
        if item is None:
            # Re-insert None for other consumers
            await queue.put(None)
            queue.task_done()
            break

        print(f"Consumer {name} got {item}")
        await asyncio.sleep(random.uniform(0.3, 1.0))
        queue.task_done()
        count += 1

    print(f"Consumer {name} finished, processed {count} items")

```

```

    return count

async def queue_demo() -> None:
    """Demonstrate producer-consumer pattern with a queue."""
    queue = asyncio.Queue()

    # Start one producer and multiple consumers
    producer_task = asyncio.create_task(producer(queue, 10))
    consumer_tasks = [
        asyncio.create_task(consumer(queue, f"Consumer-{i}"))
        for i in range(3)
    ]

    # Wait for the producer to finish
    await producer_task

    # Wait for consumers to finish
    results = await asyncio.gather(*consumer_tasks)
    print(f"Consumers processed {sum(results)} items in total")

# Pattern 6: Streaming processing with async generators
async def data_source(count: int) -> None:
    """Generate data asynchronously."""
    for i in range(count):
        await asyncio.sleep(random.uniform(0.1, 0.5))
        yield f"Data-{i}"

async def process_stream(stream):
    """Process a stream of data as it arrives."""
    async for item in stream:
        print(f"Processing {item}")
        await asyncio.sleep(random.uniform(0.2, 0.7))
        yield f"Processed-{item}"

async def streaming_demo() -> None:
    """Demonstrate streaming data processing."""
    source = data_source(5)
    processor = process_stream(source)

    print("Starting streaming demo")
    results = []
    async for result in processor:
        print(f"Received {result}")
        results.append(result)

    print(f"All results: {results}")

```

```

# Pattern 7: Periodic tasks
async def periodic_task(interval: float, name: str, count: int = 5) -> None:
    """Run a task periodically at specified intervals."""
    for i in range(count):
        print(f"{name}: Execution {i+1}/{count} at {time.time():.2f}")
        await asyncio.sleep(interval)

    print(f"{name}: Completed all executions")

async def periodic_demo() -> None:
    """Demonstrate running periodic tasks."""
    # Start multiple periodic tasks with different intervals
    task1 = asyncio.create_task(periodic_task(1.0, "Fast Task", 5))
    task2 = asyncio.create_task(periodic_task(2.0, "Medium Task", 3))
    task3 = asyncio.create_task(periodic_task(3.0, "Slow Task", 2))

    # Wait for all tasks to complete
    await asyncio.gather(task1, task2, task3)
    print("All periodic tasks completed")

# Pattern 8: Rate limiting
class RateLimiter:
    """A rate limiter for async tasks."""

    def __init__(self, rate: int, interval: float = 1.0):
        """
        Initialize with maximum number of operations per interval.

        Args:
            rate: Maximum number of operations per interval
            interval: Time interval in seconds
        """
        self.rate = rate
        self.interval = interval
        self.tokens = rate
        self.updated_at = time.time()
        self.lock = asyncio.Lock()

    async def acquire(self) -> None:
        """Acquire a token, waiting if necessary."""
        async with self.lock:
            while True:
                # Update tokens based on elapsed time
                now = time.time()
                elapsed = now - self.updated_at

```

```

        if elapsed > self.interval:
            # Refill tokens
            self.tokens = self.rate
            self.updated_at = now

        if self.tokens > 0:
            # Token available
            self.tokens -= 1
            return

        # No tokens available, calculate wait time
        wait_time = self.interval - elapsed
        await asyncio.sleep(wait_time)

    async def rate_limited_task(id: int, limiter: RateLimiter) -> None:
        """A task that respects rate limiting."""
        await limiter.acquire()
        print(f"Task {id} running at {time.time():.2f}")
        await asyncio.sleep(random.uniform(0.1, 0.5))

    async def rate_limiter_demo() -> None:
        """Demonstrate rate limiting async tasks."""
        # Allow 3 operations per second
        limiter = RateLimiter(rate=3, interval=1.0)

        # Start many tasks that will be rate limited
        tasks = [rate_limited_task(i, limiter) for i in range(10)]
        await asyncio.gather(*tasks)
        print("All rate-limited tasks completed")

    # Pattern 9: Retry with exponential backoff
    async def unreliable_operation(id: int, fail_probability: float = 0.5) -> str:
        """An operation that might fail randomly."""
        await asyncio.sleep(random.uniform(0.1, 0.5))

        if random.random() < fail_probability:
            raise Exception(f"Operation {id} failed randomly")

        return f"Result from operation {id}"

    async def with_retries(coroutine, max_retries: int = 3, base_delay: float = 1.0) -> Any:
        """Execute a coroutine with retries and exponential backoff."""
        retries = 0
        while True:
            try:

```

```

        return await coro
    except Exception as e:
        retries += 1
        if retries > max_retries:
            print(f"Failed after {max_retries} retries: {e}")
            raise

        delay = base_delay * (2 ** (retries - 1)) * (0.5 + random.random())
        print(f"Retry {retries}/{max_retries} after {delay:.2f}s due to: {e}")
        await asyncio.sleep(delay)

async def retry_demo() -> None:
    """Demonstrate retry logic with exponential backoff."""
    tasks = [
        with_retries(unreliable_operation(i, fail_probability=0.7))
        for i in range(5)
    ]

    results = await asyncio.gather(*tasks, return_exceptions=True)

    for i, result in enumerate(results):
        if isinstance(result, Exception):
            print(f"Task {i} failed with: {result}")
        else:
            print(f"Task {i} succeeded with: {result}")

# Pattern 10: Graceful shutdown
class AsyncService:
    """A service that needs graceful shutdown."""

    def __init__(self, name: str):
        self.name = name
        self.is_running = False
        self.tasks = []

    async def start(self) -> None:
        """Start the service and its background tasks."""
        print(f"Starting {self.name}...")
        self.is_running = True

        # Start background tasks
        self.tasks = [
            asyncio.create_task(self._background_job(f"Job-{i}"))
            for i in range(3)
        ]

```

```

        print(f"{self.name} started with {len(self.tasks)} background tasks")

    async def _background_job(self, job_id: str) -> None:
        """A background job that runs until the service stops."""
        print(f"{self.name} - {job_id}: Starting")
        try:
            while self.is_running:
                print(f"{self.name} - {job_id}: Working...")
                await asyncio.sleep(1)
        except asyncio.CancelledError:
            print(f"{self.name} - {job_id}: Cancelled")
            raise
        finally:
            print(f"{self.name} - {job_id}: Cleaning up")

    async def stop(self) -> None:
        """Stop the service gracefully."""
        print(f"Stopping {self.name}...")
        self.is_running = False

        # Cancel all background tasks
        for task in self.tasks:
            task.cancel()

        # Wait for all tasks to finish with a timeout
        if self.tasks:
            await asyncio.wait(self.tasks, timeout=5)

        print(f"{self.name} stopped")

    async def shutdown_demo() -> None:
        """Demonstrate graceful service shutdown."""
        service = AsyncService("DemoService")

        # Start the service
        await service.start()

        # Let it run for a while
        print("Service running... (waiting)")
        await asyncio.sleep(3)

        # Request graceful shutdown
        print("Requesting service shutdown")
        await service.stop()
        print("Service shutdown completed")

```

```

# Main function to run all demos
async def main() -> None:
    """Run all the asynchronous pattern demos."""
    # Set up a list of demos
    demos = [
        ("Task Cancellation", task_cancellation_demo),
        ("Timeouts", timeout_demo),
        ("As Completed", as_completed_demo),
        ("Semaphore for Concurrency Limiting", semaphore_demo),
        ("Producer-Consumer Queue", queue_demo),
        ("Streaming Processing", streaming_demo),
        ("Periodic Tasks", periodic_demo),
        ("Rate Limiting", rate_limiter_demo),
        ("Retry with Backoff", retry_demo),
        ("Graceful Shutdown", shutdown_demo)
    ]

    # Run each demo with a header
    for title, demo in demos:
        print(f"\n{'=' * 50}")
        print(f"DEMO: {title}")
        print(f"{'=' * 50}\n")

        try:
            await demo()
        except Exception as e:
            print(f"Demo failed with error: {e}")

        print(f"\n{'=' * 50}")
        print(f"End of {title} demo")
        print(f"{'=' * 50}\n")

        # Short pause between demos
        await asyncio.sleep(1)

    print("\nAll demos completed!")

# Run everything
if __name__ == "__main__":
    asyncio.run(main())

```

9. CPython Internals

Understanding CPython internals provides valuable insight into why Python behaves the way it does, aids in writing more efficient code, and helps diagnose performance issues. This chapter explores the theoretical foundations and

implementation details of key CPython components.

Bytecode and the Python Virtual Machine

Python is a compiled language—but unlike C or Java, it compiles to bytecode rather than machine code. Understanding this compilation process and how the Python Virtual Machine (PVM) executes bytecode provides insight into Python’s execution model.

Compilation Process When Python code is executed, it undergoes several transformation phases:

1. **Parsing:** Source code is parsed into a parse tree
2. **AST Generation:** The parse tree is converted to an Abstract Syntax Tree (AST)
3. **Symbol Table Construction:** Variable scopes and symbol references are resolved
4. **Code Generation:** The AST is compiled into bytecode
5. **Execution:** The bytecode is executed by the Python Virtual Machine

Bytecode Structure Python bytecode consists of a series of instructions, each with an opcode and potentially one or more operands. These instructions operate on the stack and the local/global namespaces.

```
import dis

def example_function(x, y):
    z = x + y
    return z * 2

# Disassemble the function to see its bytecode
dis.dis(example_function)
```

Output:

2	0 LOAD_FAST	0 (x)
	2 LOAD_FAST	1 (y)
	4 BINARY_ADD	
	6 STORE_FAST	2 (z)
3	8 LOAD_FAST	2 (z)
	10 LOAD_CONST	1 (2)
	12 BINARY_MULTIPLY	
	14 RETURN_VALUE	

Each line in the disassembly shows: - Line number in the original source code - Bytecode offset - Instruction name (opcode) - Operand(s), if any - Interpretation of the operand in parentheses

Code Objects Bytecode is stored in code objects, which contain all the information needed to execute a block of code. The structure of a code object includes:

```
import inspect

def example():
    x = 1
    y = 2
    return x + y

# Inspect the code object
code = example.__code__
print(f"Argument count: {code.co_argcount}")
print(f"Local variables: {code.co_varnames}")
print(f"Bytecode: {code.co_code.hex()}")
print(f"Constants: {code.co_consts}")
print(f"Names: {code.co_names}")
```

Code objects are immutable, ensuring execution integrity, and are stored in .pyc files for modules to enable faster subsequent loading.

The Python Virtual Machine The CPython VM is stack-based, meaning operations pop operands from the stack and push results back onto it. The VM's main components are:

1. **Frame Objects:** Execution frames containing the local namespace, stack, and bytecode
2. **Evaluation Loop:** The core loop that fetches, decodes, and executes instructions
3. **Thread State:** Containing the call stack and exception state
4. **Interpreter State:** Global state including modules and configuration

The evaluation loop is implemented in CPython's `ceval.c` file, using a giant switch statement to handle each opcode. This is the heart of Python's execution model.

Bytecode Optimization CPython performs several optimizations at the bytecode level:

1. **Constant Folding:** Expressions with constants are pre-computed
2. **Peephole Optimization:** Removing redundant instructions
3. **Jump Optimization:** Simplifying and shortening jump chains

However, CPython does not perform many traditional compiler optimizations like just-in-time compilation (with some exceptions like the specialized eval loop for certain operations in modern CPython versions).

Bytecode Cache For imported modules, Python caches the compiled bytecode in `.pyc` files. This cache speeds up subsequent module imports since the compilation step can be skipped.

The bytecode format is version-specific—.pyc files are not guaranteed to be compatible between Python versions. They include a header with a magic number indicating the Python version and a timestamp or hash to check if recompilation is needed.

Python’s Object Model: Everything is an Object

In Python, the phrase “everything is an object” is a fundamental design principle. Unlike some languages that distinguish between primitive types and objects, Python treats everything—integers, strings, functions, classes, modules—as first-class objects.

Object System Architecture Every Python object consists of three essential components: 1. **Identity**: A unique identifier (memory address) returned by the `id()` function 2. **Type**: The object’s type, determining valid operations and attributes 3. **Value**: The data represented by the object

This design is implemented in CPython’s C source code with the fundamental `PyObject` structure:

```
typedef struct _object {
    Py_ssize_t ob_refcnt;           /* Reference count */
    PyTypeObject *ob_type;         /* Object type */
} PyObject;
```

All Python objects derive from this base structure, with specific types extending it to include their value representation. For example, an integer object’s C structure looks like:

```
typedef struct {
    PyObject_HEAD                 /* Standard object header */
    long ob_ival;                 /* Integer value */
} PyIntObject;
```

Type System and Type Objects Python’s type system is hierarchical. Every object has a type, and types themselves are objects (instances of `type`). This metaprogramming capability enables powerful techniques like metaclasses.

Types define: - What operations can be performed on objects - How memory is allocated and managed - What attributes are available - How objects interact with the interpreter

Let’s examine this concept:

```
# Everything has a type
x = 42
```

```

y = "hello"
z = [1, 2, 3]

print(type(x)) # <class 'int'>
print(type(y)) # <class 'str'>
print(type(z)) # <class 'list'>

# Types are objects too
print(type(type(x))) # <class 'type'>

# The type of type is type (the metaclass)
print(type(type)) # <class 'type'>

```

Object Lifetime and Memory Management CPython manages object lifetimes through reference counting, supplemented by a cycle-detecting garbage collector. Each object maintains a count of references to it, and when this count reaches zero, the object is deallocated.

Memory Management and Garbage Collection

CPython uses a two-tiered approach to memory management: reference counting as the primary mechanism, supplemented by a cycle-detecting garbage collector.

Reference Counting Reference counting is the foundation of CPython's memory management system. Each object maintains a count of references pointing to it:

1. When a reference is created, the count increases
2. When a reference is destroyed, the count decreases
3. When the count reaches zero, the object is deallocated

```

import sys

# Create an object
x = [1, 2, 3]
# Reference count = 1 (variable x)

# Create another reference
y = x
# Reference count = 2 (variables x and y)

# Check reference count (add 1 for temporary reference in getrefcount)
print(sys.getrefcount(x) - 1) # 2

# Remove a reference
y = None
# Reference count = 1 (just variable x)

```

When x goes out of scope, count becomes 0 and the object is deallocated

Reference counting provides immediate reclamation of memory when an object is no longer needed. However, it has two main limitations:

1. **Overhead:** Each reference operation must update the counter
2. **Cyclic References:** Cannot handle circular references

Cyclic Garbage Collection To handle circular references that reference counting can't detect, CPython implements a supplementary cycle-detecting garbage collector:

```
import gc

# Create a cycle
def create_cycle():
    """Create objects with circular references."""
    lst1 = []
    lst2 = []
    # Create cycle
    lst1.append(lst2)
    lst2.append(lst1)

    # Both lst1 and lst2 have reference count of 1 outside this function
    # But they reference each other, forming a cycle

# Cyclic references demonstration
create_cycle() # After this function returns, lst1 and lst2 are unreachable
               # but their reference counts are not zero

# Force garbage collection
collected = gc.collect()
print(f"Collected {collected} objects")
```

The cycle detector works by:

1. Tracking all container objects (lists, dictionaries, etc.)
2. Periodically searching for cycles in the object graph
3. Breaking cycles and freeing objects when detected

Generational Garbage Collection CPython's garbage collector is generational, dividing objects into three generations based on how many collection cycles they've survived:

1. **Generation 0:** New objects
2. **Generation 1:** Objects that survived one collection
3. **Generation 2:** Objects that survived two or more collections

The generational hypothesis assumes most objects die young, so the collector runs more frequently on younger generations:

```
import gc

# Check current thresholds for each generation
print(f"GC thresholds: {gc.get_threshold()}") # (700, 10, 10) by default

# Check current counts
print(f"GC counts: {gc.get_count()}") # (n0, n1, n2) - counters for each generation

# Generation 0 is collected when its counter reaches 700
# Generation 1 is collected when its counter reaches 10
# Generation 2 is collected when its counter reaches 10
```

Memory Allocation Strategies CPython employs specialized memory allocators to manage different types of objects:

1. **PyMalloc:** A custom allocator for small objects (≤ 512 bytes)
2. **Block Allocator:** Groups objects of similar sizes for efficiency
3. **Arena Allocator:** Allocates large chunks of memory (arenas) that are subdivided

This multi-level approach reduces fragmentation and improves performance compared to using the system’s general-purpose allocator.

Dictionary Implementation

Python dictionaries (dict) are fundamental to the language’s performance. They’re used for variable lookups, object attribute storage, and countless other operations.

Hash Tables and Dictionary Structure At their core, Python dictionaries are implemented as hash tables—an array of slots where each entry contains a hash, key, and value. This implementation provides $O(1)$ average time complexity for lookups, insertions, and deletions.

The dictionary’s internal structure consists of:

- A sparse array of entries (the hash table)
- Each entry contains a hash value, key reference, and value reference
- A “fill factor” that triggers resizing

Hash Functions and Collisions For any object to be a dictionary key, it must be hashable—it must implement a `__hash__()` method that returns a consistent integer throughout its lifetime. Immutable types like strings, numbers, and tuples (containing only immutable items) are hashable by default.

The hash function aims to distribute keys uniformly across the hash table to minimize collisions. When collisions occur (when different keys hash to the same

location), CPython uses a collision resolution strategy.

Open Addressing with Probing CPython uses open addressing to handle collisions. When a collision occurs, it probes for the next available slot using a pseudorandom sequence based on the hash value. This approach is known as “open addressing with random probing.”

```
# Simplified pseudocode of dictionary lookup process
def lookup(dict, key):
    hash_value = hash(key)
    index = hash_value % dict.size

    while True:
        entry = dict.entries[index]

        if entry is empty:
            # Key not found
            return KeyError

        if entry.hash == hash_value and entry.key == key:
            # Key found
            return entry.value

        # Collision occurred, probe next location
        index = (index + 1) % dict.size
```

Dictionary Resizing As dictionaries grow, they need to be resized to maintain performance. CPython triggers resizing when the dictionary reaches approximately 2/3 capacity. Resizing involves:

1. Allocating a new, larger table (typically 2-4 times larger)
2. Recomputing the index for each key-value pair based on the new table size
3. Inserting them into the new table

This operation, while expensive, preserves the $O(1)$ average-case complexity for lookups.

Ordered Dictionaries (Python 3.6+) Since Python 3.6, dictionaries preserve insertion order. This was achieved by adding another data structure—a doubly-linked list—that maintains the insertion order of keys. This design combines the performance benefits of hash tables with ordered iteration.

```
# Demonstrating ordered dictionaries
d = {'first': 1, 'second': 2, 'third': 3}
print(list(d.keys())) # ['first', 'second', 'third'] - Guaranteed order since Python 3.7
```

List Implementation

Python lists are versatile data structures that provide dynamic arrays with amortized constant-time appends and $O(1)$ indexing.

Dynamic Array Implementation Under the hood, a Python list is implemented as a dynamic array (vector) of pointers to Python objects. The key features of this implementation include:

1. **Overallocation:** Lists allocate more memory than immediately needed
2. **Contiguous memory:** List elements are stored in contiguous memory blocks
3. **Variable resizing:** Lists grow and shrink as elements are added or removed

The C structure underlying a list looks like:

```
typedef struct {
    PyObject_VAR_HEAD
    PyObject **ob_item;    /* Vector of pointers to items */
    Py_ssize_t allocated;  /* Size of allocated memory */
} PyListObject;
```

Growth Strategy and Memory Management When a list needs to grow beyond its allocated space, CPython uses an overallocation strategy:

1. For small lists ($n < 8$), allocated size = $n + 4$
2. For larger lists, allocated size = $n + n/2 + 1$

This overallocation reduces the frequency of resizing operations, resulting in amortized $O(1)$ complexity for appends.

Analyzing list growth

```
import sys

sizes = []
for i in range(64):
    lst = []
    for j in range(i):
        lst.append(j)
    sizes.append((i, sys.getsizeof(lst), len(lst)))

for i, (size, memory, length) in enumerate(sizes):
    if i > 0:
        prev_size, prev_memory, _ = sizes[i-1]
        if memory > prev_memory:
            print(f"List with {length} elements uses {memory} bytes (increased from {prev_m
```

Time Complexity Analysis Understanding the implementation helps explain the time complexity of common list operations:

- **Indexing:** $O(1)$ - Direct access via memory offset
- **Append:** $O(1)$ amortized - Occasional resizing, but infrequent
- **Insert/Delete at beginning:** $O(n)$ - Requires shifting all elements
- **Insert/Delete in middle:** $O(n)$ - Requires shifting subsequent elements
- **Pop from end:** $O(1)$ - Simple pointer adjustment
- **Pop from beginning:** $O(n)$ - Requires shifting all elements

Timsort Implementation Python’s sorting algorithm, Timsort, is a hybrid of merge sort and insertion sort. It was specifically designed for Python but has been adopted by other languages.

Key features of Timsort: - Exploits natural runs (already sorted sequences) in the data - Uses insertion sort for small runs - Merges runs using merge sort techniques - Maintains stability (preserving relative order of equal elements) - Achieves $O(n \log n)$ worst-case time complexity - Approaches $O(n)$ time complexity for partially sorted data

Integer Implementation and Arbitrary Precision

Python’s integers are implemented with arbitrary precision, allowing them to hold values of any magnitude, limited only by available memory.

Fixed-Width vs. Arbitrary Precision Integers Most programming languages use fixed-width integers (typically 32 or 64 bits), which have a limited range and overflow when that range is exceeded. Python, in contrast, uses arbitrary precision integers that automatically expand to accommodate larger values.

CPython Integer Implementation In CPython, integers are implemented using the following structure:

```
struct _PyLongObject {
    PyObject_VAR_HEAD
    digit ob_digit[1]; /* Variable-length array of digits */
};
```

The key aspects of this implementation are:

1. **Variable-length Array:** The `ob_digit` array stores the integer value in a series of “digits” (typically 30-bit values in modern CPython)
2. **Base-2³⁰ Representation:** Each “digit” represents a power of 2³⁰, allowing efficient storage and arithmetic
3. **Dynamic Memory Allocation:** The structure dynamically allocates memory as needed for larger integers

Small Integer Cache For efficiency, CPython preallocates integers in a commonly used range (typically -5 to 256). This means small integers are

singletons—the same object is reused, improving memory usage and comparison performance.

```
# Demonstrating small integer caching
a = 42
b = 42
print(a is b)  # True - same object

x = 1000
y = 1000
print(x is y)  # False - different objects for larger integers
```

Arbitrary Precision Arithmetic Arbitrary precision allows Python to handle integers of any size, implementing fundamental mathematical operations regardless of value magnitude:

1. **Addition and Subtraction:** Performed digit-by-digit with carry/borrow
2. **Multiplication:** Uses Karatsuba algorithm for larger integers, improving on classical $O(n^2)$ multiplication
3. **Division:** Implements a variant of long division
4. **Exponentiation:** Uses binary exponentiation for efficiency

This capability enables Python to handle calculations that would cause overflow in languages with fixed-width integers:

```
# Factorial of a large number
import math

n = 100
factorial_100 = math.factorial(n)
print(f"Factorial of 100 has {len(str(factorial_100))} digits")

# Extremely large exponentiation
large_power = 2**1000
print(f"2^1000 has {len(str(large_power))} digits")
```

Performance Considerations While arbitrary precision is powerful, it comes with overhead:

1. **Memory Management:** Dynamic allocation and deallocation of memory
2. **Computational Complexity:** Operations on large integers have higher complexity
3. **Object Overhead:** Each integer carries PyObject overhead

For performance-critical code working with values in the normal integer range, specialized libraries like NumPy provide more efficient fixed-width integer types.

The Global Interpreter Lock (GIL)

The Global Interpreter Lock (GIL) is one of the most misunderstood aspects of CPython. It's a mutex that protects access to Python objects, preventing multiple native threads from executing Python bytecode simultaneously.

Theoretical Foundation and Purpose The GIL exists primarily for two reasons:

1. **Memory Management Simplification:** CPython's reference counting system is not thread-safe by design. Without the GIL, reference counts would need thread synchronization mechanisms, adding complexity and overhead.
2. **C Extension Compatibility:** Many C extensions rely on the GIL for thread safety, allowing them to manipulate Python objects without additional synchronization.

How the GIL Works The GIL is a single lock that must be acquired before executing Python bytecode. The execution process follows this pattern:

1. Thread acquires the GIL
2. Thread executes for a fixed number of bytecode instructions or duration
3. Thread releases the GIL (even if it's not finished)
4. Other threads can acquire the GIL
5. Original thread competes to reacquire the GIL

This cooperative multitasking system allows multiple threads to exist but prevents true parallelism for CPU-bound tasks within a single Python process.

CPU-bound Tasks For CPU-bound tasks (calculations, data processing), threading provides no performance benefit due to the GIL. Only one thread can execute Python code at a time, so multiple threads simply take turns without parallelism.

```
import threading
import time
import multiprocessing

def cpu_bound_task(n):
    """A CPU-intensive function that performs many calculations."""
    count = 0
    for i in range(n):
        count += i
    return count

# With threading (GIL prevents parallelism)
def run_with_threads(n_threads, n_calculations):
```

```

threads = []
for _ in range(n_threads):
    t = threading.Thread(target=cpu_bound_task, args=(n_calculations//n_threads,))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

# With multiprocessing (bypasses GIL)
def run_with_processes(n_processes, n_calculations):
    processes = []
    for _ in range(n_processes):
        p = multiprocessing.Process(target=cpu_bound_task, args=(n_calculations//n_processes,))
        processes.append(p)
        p.start()

    for p in processes:
        p.join()

```

In CPU-bound tasks, multiprocessing outperforms threading because each process has its own Python interpreter and GIL.

I/O-bound Tasks For I/O-bound tasks (file operations, network requests), threading provides benefits despite the GIL. This is because the GIL is released during I/O operations, allowing other threads to execute.

```

import threading
import time
import requests

def io_bound_task(url):
    """An I/O-bound function that makes a network request."""
    response = requests.get(url)
    return response.text[:100] # Return first 100 chars

def sequential_io():
    """Run I/O tasks sequentially."""
    urls = ["https://example.com" for _ in range(10)]
    results = []
    for url in urls:
        results.append(io_bound_task(url))
    return results

def threaded_io():
    """Run I/O tasks with threading."""

```

```

urls = ["https://example.com" for _ in range(10)]
threads = []
results = []

for url in urls:
    t = threading.Thread(target=lambda u, r: r.append(io_bound_task(u)),
                        args=(url, results))
    threads.append(t)
    t.start()

for t in threads:
    t.join()

return results

```

GIL Impact on System Design Understanding the GIL influences system architecture decisions:

1. For CPU-bound workloads, use multiprocessing or implement critical sections in C extensions that release the GIL
2. For I/O-bound workloads, threading or asyncio works well despite the GIL
3. For mixed workloads, consider a hybrid approach

Extension Modules and the C API

Python's extensibility is one of its key strengths, allowing performance-critical components to be implemented in languages like C or C++. The CPython C API provides the interface for extending Python with compiled code.

Extension Modules Architecture Extension modules are dynamically loadable libraries that implement Python modules in C/C++. Their structure follows a defined pattern:

1. **Module Initialization:** Setting up the module object and its methods
2. **Type Definitions:** Implementing custom Python types in C
3. **Function Definitions:** C functions that can be called from Python
4. **Reference Management:** Handling object references correctly

A simple extension module might look like:

```

#include <Python.h>

static PyObject *
example_add(PyObject *self, PyObject *args)
{
    int a, b;
    if (!PyArg_ParseTuple(args, "ii", &a, &b))
        return NULL;
}

```

```

    return PyLong_FromLong(a + b);
}

static PyMethodDef ExampleMethods[] = {
    {"add", example_add, METH_VARARGS, "Add two integers."},
    {NULL, NULL, 0, NULL} /* Sentinel */
};

static struct PyModuleDef examplemodule = {
    PyModuleDef_HEAD_INIT,
    "example", /* module name */
    NULL, /* module documentation */
    -1, /* size of per-interpreter state or -1 */
    ExampleMethods
};

PyMODINIT_FUNC
PyInit_example(void)
{
    return PyModule_Create(&examplemodule);
}

```

The Python/C API The Python/C API consists of hundreds of functions to interact with the Python interpreter. Key areas include:

1. **Object Management:** Creating, inspecting, and manipulating Python objects
2. **Type System Interaction:** Working with Python's type system
3. **Memory Management:** Reference counting and memory allocation
4. **Exception Handling:** Setting and checking for exceptions
5. **Module and Class Creation:** Building modules and defining new types

Reference Counting in Extensions Extension code must carefully maintain reference counts to avoid memory leaks or use-after-free errors:

1. **Py_INCREF(obj):** Increases an object's reference count
2. **Py_DECREF(obj):** Decreases the count, potentially deallocating the object
3. **Borrowed References:** References that don't increase the count
4. **Owned References:** References that you must eventually Py_DECREF

Modern extensions often use Capsules to safely pass C pointers to Python and back:

```

static PyObject *
create_capsule(PyObject *self, PyObject *args)
{

```

```

    MyStruct *data = malloc(sizeof(MyStruct));
    /* Initialize data... */

    return PyCapsule_New(data, "example.MyStruct", capsule_destructor);
}

static void
capsule_destructor(PyObject *capsule)
{
    MyStruct *data = PyCapsule_GetPointer(capsule, "example.MyStruct");
    free(data);
}

```

GIL Management in Extensions Extensions that perform long-running operations should release the GIL to allow other Python threads to run:

```

static PyObject *
long_running_function(PyObject *self, PyObject *args)
{
    /* Parse arguments... */

    /* Release the GIL */
    Py_BEGIN_ALLOW_THREADS

    /* CPU-intensive operation without Python objects */
    perform_lengthy_computation();

    /* Reacquire the GIL */
    Py_END_ALLOW_THREADS

    /* Return result */
    return Py_BuildValue("i", result);
}

```

C API Best Practices When writing extension modules:

1. **Maintain Reference Counts:** Carefully track object ownership
2. **Handle Exceptions:** Check for errors after API calls and propagate them
3. **Release the GIL:** Allow other threads to run during long operations
4. **Use Typed Objects:** Use functions specific to object types when possible
5. **Consider Alternative Approaches:** Cython, ctypes, or cffi might be easier
6. **Support Multiple Python Versions:** Use macros for version-specific features

Extension modules provide a way to overcome CPython's performance limitations for specific tasks, especially computation-intensive operations where the GIL

would otherwise be a bottleneck.

Conclusion: Implications for Python Programmers

Understanding CPython internals helps programmers write more efficient and effective Python code:

1. **Bytecode Knowledge:** Understanding bytecode generation helps explain Python's execution model and optimize code
2. **Object Model:** Appreciating Python's "everything is an object" design explains language behavior and capabilities
3. **Memory Management:** Understanding reference counting and garbage collection helps prevent memory leaks
4. **Data Structure Choices:** Knowing the implementation details of dictionaries and lists guides appropriate data structure selection
5. **Integer Behavior:** Understanding arbitrary precision integers explains why Python doesn't have integer overflow issues
6. **Concurrency Design:** Recognizing the GIL's impact guides better concurrent programming choices
7. **Performance Optimization:** Knowing when and how to use C extensions allows for addressing performance bottlenecks

By building on these foundational concepts, Python programmers can make informed decisions about algorithms, data structures, and system architecture that lead to more efficient and maintainable code.

10. Metaprogramming and Design Patterns

Dunder Methods: `__init__`, `__repr__`, `__eq__`, `__lt__`, `__hash__`, `__getitem__`, etc.

Python's special methods (a.k.a. "dunder" or double underscore methods) allow you to define how objects of your classes behave in various contexts. They enable you to make custom objects that work seamlessly with Python's built-in functions and operators.

```
# Comprehensive demonstration of dunder methods

# Basic initialization and representation
class Point:
    """A 2D point class with basic dunder methods."""

    def __init__(self, x, y):
        """Initialize with x and y coordinates."""
        self.x = x
        self.y = y

    def __repr__(self):
```



```

        """Return the official string representation."""
        return f"Point({self.x}, {self.y})"

    def __str__(self):
        """Return a user-friendly string representation."""
        return f"Point at ({self.x}, {self.y})"

    def __format__(self, format_spec):
        """Control how the object is formatted with format()."""
        if format_spec == 'c': # Cartesian format
            return f"({self.x}, {self.y})"
        elif format_spec == 'p': # Polar format
            import math
            r = math.sqrt(self.x**2 + self.y**2)
            theta = math.atan2(self.y, self.x)
            return f"({r:.2f}, {theta:.2f})"
        else:
            return str(self)

# Create a Point instance
point = Point(3, 4)
print(repr(point)) # Point(3, 4)
print(str(point)) # Point at (3, 4)
print(f"{point}") # Point at (3, 4)
print(f"{point:c}") # (3, 4)
print(f"{point:p}") # (5.00, 0.93)

# Comparison and hashing
class Vector:
    """A vector class with comparison and hashing."""

    def __init__(self, x, y, z):
        """Initialize with x, y, and z components."""
        self.x = x
        self.y = y
        self.z = z

    def __repr__(self):
        """Return the official string representation."""
        return f"Vector({self.x}, {self.y}, {self.z})"

    def __eq__(self, other):
        """Define equality between vectors."""
        if not isinstance(other, Vector):
            return NotImplemented
        return (self.x == other.x and

```

```

        self.y == other.y and
        self.z == other.z)

def __lt__(self, other):
    """Compare vectors based on their magnitude."""
    if not isinstance(other, Vector):
        return NotImplemented
    return self.magnitude() < other.magnitude()

def __le__(self, other):
    """Less than or equal comparison."""
    if not isinstance(other, Vector):
        return NotImplemented
    return self < other or self == other

def __gt__(self, other):
    """Greater than comparison."""
    if not isinstance(other, Vector):
        return NotImplemented
    return not (self <= other)

def __ge__(self, other):
    """Greater than or equal comparison."""
    if not isinstance(other, Vector):
        return NotImplemented
    return not (self < other)

def __hash__(self):
    """Make vectors hashable (immutable vectors only)."""
    return hash((self.x, self.y, self.z))

def magnitude(self):
    """Calculate the magnitude of the vector."""
    return (self.x**2 + self.y**2 + self.z**2)**0.5

# Create Vector instances
v1 = Vector(1, 2, 3)
v2 = Vector(1, 2, 3)
v3 = Vector(4, 5, 6)

print(v1 == v2) # True
print(v1 == v3) # False
print(v1 < v3)  # True
print(v1 > v3)  # False

# Use vectors in sets and dictionaries (thanks to __hash__)

```

```

vector_set = {v1, v3}
print(len(vector_set)) # 2
vector_set.add(v2) # v2 is equal to v1, so it won't be added
print(len(vector_set)) # Still 2

vector_dict = {v1: "First vector", v3: "Second vector"}
print(vector_dict[v2]) # "First vector" (v2 is equal to v1)

# Numeric operations
class ComplexNumber:
    """A complex number class with numeric operations."""

    def __init__(self, real, imag):
        """Initialize with real and imaginary parts."""
        self.real = real
        self.imag = imag

    def __repr__(self):
        """Return the official string representation."""
        if self.imag >= 0:
            return f"ComplexNumber({self.real}, {self.imag})"
        else:
            return f"ComplexNumber({self.real}, {self.imag})"

    def __str__(self):
        """Return a user-friendly string representation."""
        if self.imag >= 0:
            return f"{self.real} + {self.imag}i"
        else:
            return f"{self.real} - {abs(self.imag)}i"

    def __add__(self, other):
        """Add two complex numbers."""
        if isinstance(other, (int, float)):
            other = ComplexNumber(other, 0)
        if not isinstance(other, ComplexNumber):
            return NotImplemented
        return ComplexNumber(self.real + other.real, self.imag + other.imag)

    def __radd__(self, other):
        """Reverse addition (other + self)."""
        return self.__add__(other)

    def __sub__(self, other):
        """Subtract two complex numbers."""
        if isinstance(other, (int, float)):

```

```

        other = ComplexNumber(other, 0)
    if not isinstance(other, ComplexNumber):
        return NotImplemented
    return ComplexNumber(self.real - other.real, self.imag - other.imag)

def __rsub__(self, other):
    """Reverse subtraction (other - self)."""
    if isinstance(other, (int, float)):
        other = ComplexNumber(other, 0)
    if not isinstance(other, ComplexNumber):
        return NotImplemented
    return ComplexNumber(other.real - self.real, other.imag - self.imag)

def __mul__(self, other):
    """Multiply two complex numbers."""
    if isinstance(other, (int, float)):
        other = ComplexNumber(other, 0)
    if not isinstance(other, ComplexNumber):
        return NotImplemented
    real = self.real * other.real - self.imag * other.imag
    imag = self.real * other.imag + self.imag * other.real
    return ComplexNumber(real, imag)

def __rmul__(self, other):
    """Reverse multiplication (other * self)."""
    return self.__mul__(other)

def __neg__(self):
    """Negation (-self)."""
    return ComplexNumber(-self.real, -self.imag)

def __abs__(self):
    """Absolute value."""
    return (self.real**2 + self.imag**2)**0.5

def __bool__(self):
    """Boolean value."""
    return self.real != 0 or self.imag != 0

# Create ComplexNumber instances
c1 = ComplexNumber(3, 4)
c2 = ComplexNumber(1, 2)

print(c1)           # 3 + 4i
print(c1 + c2)       # 4 + 6i
print(c1 - c2)       # 2 + 2i

```

```

print(c1 * c2)      # -5 + 10i
print(-c1)         # -3 - 4i
print(abs(c1))     # 5.0
print(bool(c1))    # True
print(bool(ComplexNumber(0, 0))) # False

# Also works with regular numbers
print(c1 + 5)      # 8 + 4i
print(5 + c1)      # 8 + 4i
print(2 * c1)      # 6 + 8i

# Container operations
class Matrix:
    """A simple matrix class with container operations."""

    def __init__(self, data):
        """Initialize with a 2D list of data."""
        self.data = list(data)
        self.rows = len(data)
        self.cols = len(data[0]) if self.rows > 0 else 0

    def __repr__(self):
        """Return the official string representation."""
        return f"Matrix({self.data})"

    def __str__(self):
        """Return a user-friendly string representation."""
        result = []
        for row in self.data:
            result.append(" ".join(str(x) for x in row))
        return "\n".join(result)

    def __getitem__(self, key):
        """Get an item using indexing or slicing."""
        if isinstance(key, tuple) and len(key) == 2:
            # Matrix[row, col]
            row, col = key
            return self.data[row][col]
        elif isinstance(key, int):
            # Matrix[row]
            return self.data[key]
        else:
            raise TypeError("Invalid index type")

    def __setitem__(self, key, value):
        """Set an item using indexing."""

```

```

        if isinstance(key, tuple) and len(key) == 2:
            # Matrix[row, col] = value
            row, col = key
            self.data[row][col] = value
        elif isinstance(key, int):
            # Matrix[row] = value
            if not isinstance(value, list) or len(value) != self.cols:
                raise ValueError(f"Row must be a list of length {self.cols}")
            self.data[key] = value
        else:
            raise TypeError("Invalid index type")

    def __len__(self):
        """Return the number of rows."""
        return self.rows

    def __contains__(self, item):
        """Check if a value is in the matrix."""
        for row in self.data:
            if item in row:
                return True
        return False

    def __iter__(self):
        """Iterate over the rows of the matrix."""
        return iter(self.data)

# Create a Matrix instance
matrix = Matrix([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])

print(matrix)           # Prints the matrix in a readable format
print(matrix[1, 2])     # 6
print(matrix[0])        # [1, 2, 3]
matrix[0, 0] = 10       # Update a single element
print(matrix[0, 0])     # 10
print(len(matrix))      # 3
print(5 in matrix)      # True
print(15 in matrix)     # False

# Iterate over the matrix
for row in matrix:
    print(row)

```

```

# Context manager
class FileManager:
    """A file manager that uses context manager protocol."""

    def __init__(self, filename, mode='r'):
        """Initialize with filename and mode."""
        self.filename = filename
        self.mode = mode
        self.file = None

    def __enter__(self):
        """Open the file and return it."""
        self.file = open(self.filename, self.mode)
        return self.file

    def __exit__(self, exc_type, exc_value, traceback):
        """Close the file."""
        if self.file:
            self.file.close()

        # Return False to propagate exceptions, True to suppress
        if exc_type is not None:
            print(f"Exception {exc_type.__name__} occurred: {exc_value}")
        return False

# Using the context manager
try:
    with FileManager('example.txt', 'w') as f:
        f.write('Hello, World!')

    with FileManager('example.txt', 'r') as f:
        content = f.read()
        print(f"File content: {content}")

    with FileManager('nonexistent.txt', 'r') as f:
        content = f.read() # This will raise FileNotFoundError
except FileNotFoundError:
    print("File not found error was correctly propagated")

# Resource management
class Resource:
    """A class that demonstrates resource initialization and cleanup."""

    def __init__(self, name):
        """Initialize the resource."""

```

```

        self.name = name
        print(f"Resource {name} initialized")

    def __del__(self):
        """Cleanup when the object is about to be destroyed."""
        print(f"Resource {self.name} is being destroyed")

    def __enter__(self):
        """Context manager enter."""
        print(f"Entering context with {self.name}")
        return self

    def __exit__(self, exc_type, exc_value, traceback):
        """Context manager exit."""
        print(f"Exiting context with {self.name}")
        return False

# Using the resource
resource = Resource("example")
# Later, when the resource goes out of scope, __del__ will be called

# Using as a context manager
with Resource("context_example") as r:
    print(f"Using {r.name} within context")

# Callable objects
class Counter:
    """A callable counter class."""

    def __init__(self, start=0, step=1):
        """Initialize with start value and step size."""
        self.value = start
        self.step = step

    def __call__(self, increment=None):
        """Make the object callable."""
        if increment is None:
            increment = self.step

        self.value += increment
        return self.value

# Create a Counter instance
counter = Counter(10, 2)
print(counter())    # 12
print(counter())    # 14

```



```

print(counter(5))    # 19

# Descriptors
class Positive:
    """A descriptor that ensures a value is positive."""

    def __init__(self, name):
        """Initialize with attribute name."""
        self.name = name
        self.private_name = f"_{name}"

    def __get__(self, instance, owner):
        """Get the attribute value."""
        if instance is None:
            return self
        return getattr(instance, self.private_name, 0)

    def __set__(self, instance, value):
        """Set the attribute value, ensuring it's positive."""
        if value <= 0:
            raise ValueError(f"{self.name} must be positive")
        setattr(instance, self.private_name, value)

```python
class Product:
 """A product class that uses the Positive descriptor."""

 price = Positive("price")
 quantity = Positive("quantity")

 def __init__(self, name, price, quantity):
 """Initialize with name, price, and quantity."""
 self.name = name
 self.price = price
 self.quantity = quantity

 @property
 def total(self):
 """Calculate the total value."""
 return self.price * self.quantity

Create a Product instance
try:
 product = Product("Widget", 10.0, 5)
 print(f"{product.name}: {product.price} × {product.quantity} = {product.total}")

```

```

 # Try to set a negative price
 product.price = -5.0
 except ValueError as e:
 print(f"Error: {e}")

Attribute access customization
class AttrDict:
 """A dictionary-like class that allows attribute-style access."""

 def __init__(self, **kwargs):
 """Initialize with keyword arguments."""
 self.__dict__.update(kwargs)

 def __getattr__(self, name):
 """Called when an attribute lookup fails."""
 raise AttributeError(f"'{self.__class__.__name__}' object has no attribute '{name}'")

 def __setattr__(self, name, value):
 """Called when setting an attribute."""
 self.__dict__[name] = value

 def __delattr__(self, name):
 """Called when deleting an attribute."""
 if name in self.__dict__:
 del self.__dict__[name]
 else:
 raise AttributeError(f"'{self.__class__.__name__}' object has no attribute '{name}'")

Create an AttrDict instance
attr_dict = AttrDict(name="John", age=30)
print(attr_dict.name) # John
print(attr_dict.age) # 30

attr_dict.email = "john@example.com"
print(attr_dict.email) # john@example.com

try:
 print(attr_dict.address) # Will raise AttributeError
except AttributeError as e:
 print(f"Error: {e}")

Comprehensive example bringing it all together
class Polynomial:
 """A polynomial class with various dunder methods."""

 def __init__(self, *coefficients):

```

```

 """Initialize with coefficients (highest power first)."""
 self.coefficients = list(coefficients)

 def __repr__(self):
 """Return the official string representation."""
 return f"Polynomial({' ', ' '.join(map(str, self.coefficients))})"

 def __str__(self):
 """Return a user-friendly string representation."""
 if not self.coefficients:
 return "0"

 terms = []
 degree = len(self.coefficients) - 1

 for i, coef in enumerate(self.coefficients):
 power = degree - i

 if coef == 0:
 continue

 term = ""
 if i > 0 and coef > 0:
 term += "+"

 if coef != 1 or power == 0:
 term += str(coef)

 if power > 0:
 term += "x"
 if power > 1:
 term += f"^{power}"

 terms.append(term)

 return " ".join(terms)

 def __add__(self, other):
 """Add two polynomials."""
 if isinstance(other, (int, float)):
 other = Polynomial(other)
 if not isinstance(other, Polynomial):
 return NotImplemented

 # Ensure both polynomials have the same length by padding with zeros
 result_coefs = [0] * max(len(self.coefficients), len(other.coefficients))

```

```

 for i, coef in enumerate(self.coefficients):
 result_coefs[i + (len(result_coefs) - len(self.coefficients))] += coef

 for i, coef in enumerate(other.coefficients):
 result_coefs[i + (len(result_coefs) - len(other.coefficients))] += coef

 return Polynomial(*result_coefs)

def __radd__(self, other):
 """Reverse addition."""
 return self.__add__(other)

def __sub__(self, other):
 """Subtract two polynomials."""
 if isinstance(other, (int, float)):
 other = Polynomial(other)
 if not isinstance(other, Polynomial):
 return NotImplemented

 # Negate coefficients of other and add
 neg_other = Polynomial(*[-c for c in other.coefficients])
 return self.__add__(neg_other)

def __rsub__(self, other):
 """Reverse subtraction."""
 if isinstance(other, (int, float)):
 other = Polynomial(other)
 if not isinstance(other, Polynomial):
 return NotImplemented

 # Negate coefficients of self and add to other
 neg_self = Polynomial(*[-c for c in self.coefficients])
 return other.__add__(neg_self)

def __mul__(self, other):
 """Multiply two polynomials."""
 if isinstance(other, (int, float)):
 return Polynomial(*[c * other for c in self.coefficients])
 if not isinstance(other, Polynomial):
 return NotImplemented

 # Multiply two polynomials
 result_degree = len(self.coefficients) + len(other.coefficients) - 2
 result_coefs = [0] * (result_degree + 1)

```

```

 for i, c1 in enumerate(self.coefficients):
 for j, c2 in enumerate(other.coefficients):
 power = (len(self.coefficients) - 1 - i) + (len(other.coefficients) - 1 - j)
 result_coefs[result_degree - power] += c1 * c2

 return Polynomial(*result_coefs)

def __rmul__(self, other):
 """Reverse multiplication."""
 return self.__mul__(other)

def __call__(self, x):
 """Evaluate the polynomial at x."""
 result = 0
 for i, coef in enumerate(self.coefficients):
 power = len(self.coefficients) - 1 - i
 result += coef * (x ** power)
 return result

def __len__(self):
 """Return the number of terms (non-zero coefficients)."""
 return sum(1 for c in self.coefficients if c != 0)

def __getitem__(self, power):
 """Get the coefficient of x^{power} ."""
 degree = len(self.coefficients) - 1
 if power < 0 or power > degree:
 raise IndexError("Power out of range")
 return self.coefficients[degree - power]

def __setitem__(self, power, value):
 """Set the coefficient of x^{power} ."""
 degree = len(self.coefficients) - 1
 if power < 0 or power > degree:
 raise IndexError("Power out of range")
 self.coefficients[degree - power] = value

def __eq__(self, other):
 """Check if two polynomials are equal."""
 if isinstance(other, (int, float)):
 other = Polynomial(other)
 if not isinstance(other, Polynomial):
 return NotImplemented

 # Remove leading zeros
 self_coefs = self.coefficients.copy()

```

```

 while len(self_coeffs) > 1 and self_coeffs[0] == 0:
 self_coeffs.pop(0)

 other_coeffs = other.coefficients.copy()
 while len(other_coeffs) > 1 and other_coeffs[0] == 0:
 other_coeffs.pop(0)

 return self_coeffs == other_coeffs

 def derivative(self):
 """Calculate the derivative of the polynomial."""
 if len(self.coefficients) <= 1:
 return Polynomial(0)

 result_coeffs = []
 degree = len(self.coefficients) - 1

 for i, coef in enumerate(self.coefficients[:-1]):
 power = degree - i
 result_coeffs.append(coef * power)

 return Polynomial(*result_coeffs)

Create polynomial instances
p1 = Polynomial(1, -2, 1) # x^2 - 2x + 1
p2 = Polynomial(1, 1) # x + 1

print(f"p1 = {p1}")
print(f"p2 = {p2}")
print(f"p1 + p2 = {p1 + p2}")
print(f"p1 - p2 = {p1 - p2}")
print(f"p1 * p2 = {p1 * p2}")
print(f"p1(2) = {p1(2)}") # Evaluate at x=2
print(f"Coefficient of x^1 in p1: {p1[1]}")
print(f"Derivative of p1: {p1.derivative()}")

```

Dunder methods provide a powerful way to make your classes integrate seamlessly with Python's syntax and built-in functions. By implementing these methods, you can create objects that behave like built-in types but with custom semantics tailored to your application's needs.

## Design Patterns: Singleton, Factory, Builder, Adapter, Proxy, Command, Template Method

Design patterns are reusable solutions to common problems in software design. Python's dynamic nature and flexible syntax make implementing these patterns

more concise than in other languages.

```
Singleton Pattern
class Singleton:
 """
 A singleton class that ensures only one instance exists.

 The singleton pattern restricts the instantiation of a class to one object.
 """

 _instance = None

 def __new__(cls, *args, **kwargs):
 """Create a new instance only if none exists."""
 if cls._instance is None:
 cls._instance = super().__new__(cls)
 return cls._instance

 def __init__(self, name="Singleton"):
 """Initialize the singleton (only runs on the first instantiation)."""
 # This simple approach has an issue: __init__ is called on every instantiation
 # A more robust approach would track whether initialization has occurred
 self.name = name

Test the Singleton pattern
singleton1 = Singleton("First")
singleton2 = Singleton("Second") # The name won't change

print(singleton1 is singleton2) # True (same instance)
print(singleton1.name) # "First"
print(singleton2.name) # "First" (not "Second")

Improved Singleton Pattern with proper initialization
class BetterSingleton:
 """A better singleton implementation that only initializes once."""

 _instance = None
 _initialized = False

 def __new__(cls, *args, **kwargs):
 """Creates a new instance only if none exists."""
 if cls._instance is None:
 cls._instance = super().__new__(cls)
 return cls._instance

 def __init__(self, name="Singleton"):
```

```

 """Initialize only if not already initialized."""
 if not self.__class__._initialized:
 self.name = name
 self.__class__._initialized = True

Test the Better Singleton pattern
better1 = BetterSingleton("First")
better2 = BetterSingleton("Second") # Won't change the name

print(better1 is better2) # True
print(better1.name) # "First"
print(better2.name) # "First"

Singleton using a decorator
def singleton(cls):
 """
 A decorator to make a class follow the singleton pattern.

 This approach is more Pythonic and less error-prone.
 """
 instances = {}

 def get_instance(*args, **kwargs):
 if cls not in instances:
 instances[cls] = cls(*args, **kwargs)
 return instances[cls]

 return get_instance

@singleton
class Logger:
 """A simple logger class."""

 def __init__(self, name="Main"):
 self.name = name
 self.logs = []

 def log(self, message):
 """Log a message."""
 self.logs.append(message)
 print(f"[{self.name}] {message}")

Test the Singleton decorator
logger1 = Logger("App")
logger2 = Logger("System") # Won't change the name

```



```

print(logger1 is logger2) # True
logger1.log("Test message")
print(len(logger2.logs)) # 1

Factory Pattern
class Shape:
 """Base shape class."""

 def draw(self):
 """Draw the shape."""
 raise NotImplementedError("Subclass must implement abstract method")

class Circle(Shape):
 """A circle shape."""

 def __init__(self, radius):
 self.radius = radius

 def draw(self):
 """Draw a circle."""
 return f"Drawing Circle with radius {self.radius}"

class Rectangle(Shape):
 """A rectangle shape."""

 def __init__(self, width, height):
 self.width = width
 self.height = height

 def draw(self):
 """Draw a rectangle."""
 return f"Drawing Rectangle with width {self.width} and height {self.height}"

class Triangle(Shape):
 """A triangle shape."""

 def __init__(self, a, b, c):
 self.a = a
 self.b = b
 self.c = c

 def draw(self):
 """Draw a triangle."""
 return f"Drawing Triangle with sides {self.a}, {self.b}, {self.c}"

class ShapeFactory:

```

```

"""
A factory class for creating shapes.

The factory pattern creates objects without exposing the instantiation
logic to the client and refers to the newly created object through a
common interface.
"""

@staticmethod
def create_shape(shape_type, **kwargs):
 """Create a shape based on type and parameters."""
 shape_type = shape_type.lower()

 if shape_type == "circle":
 radius = kwargs.get("radius", 1)
 return Circle(radius)
 elif shape_type == "rectangle":
 width = kwargs.get("width", 1)
 height = kwargs.get("height", 1)
 return Rectangle(width, height)
 elif shape_type == "triangle":
 a = kwargs.get("a", 1)
 b = kwargs.get("b", 1)
 c = kwargs.get("c", 1)
 return Triangle(a, b, c)
 else:
 raise ValueError(f"Unknown shape type: {shape_type}")

Test the Factory pattern
factory = ShapeFactory()

circle = factory.create_shape("circle", radius=5)
rectangle = factory.create_shape("rectangle", width=10, height=20)
triangle = factory.create_shape("triangle", a=3, b=4, c=5)

print(circle.draw()) # Drawing Circle with radius 5
print(rectangle.draw()) # Drawing Rectangle with width 10 and height 20
print(triangle.draw()) # Drawing Triangle with sides 3, 4, 5

Builder Pattern
class Computer:
 """
 A computer class with many configurable components.

 The builder pattern separates the construction of a complex object
 from its representation, allowing the same construction process to

```

```

 create different representations.
 """

 def __init__(self):
 self.cpu = None
 self.memory = None
 self.storage = None
 self.gpu = None
 self.os = None

 def __str__(self):
 """Return a string representation of the computer."""
 components = []
 if self.cpu:
 components.append(f"CPU: {self.cpu}")
 if self.memory:
 components.append(f"Memory: {self.memory}GB")
 if self.storage:
 components.append(f"Storage: {self.storage}GB")
 if self.gpu:
 components.append(f"GPU: {self.gpu}")
 if self.os:
 components.append(f"OS: {self.os}")

 return "Computer: " + ", ".join(components)

class ComputerBuilder:
 """Builder class for creating Computer objects."""

 def __init__(self):
 self.computer = Computer()

 def with_cpu(self, cpu):
 """Add a CPU to the computer."""
 self.computer.cpu = cpu
 return self # Return self for method chaining

 def with_memory(self, memory):
 """Add memory to the computer."""
 self.computer.memory = memory
 return self

 def with_storage(self, storage):
 """Add storage to the computer."""
 self.computer.storage = storage
 return self

```

```

def with_gpu(self, gpu):
 """Add a GPU to the computer."""
 self.computer.gpu = gpu
 return self

def with_os(self, os):
 """Add an OS to the computer."""
 self.computer.os = os
 return self

def build(self):
 """Return the built computer."""
 return self.computer

Test the Builder pattern
gaming_pc = (ComputerBuilder()
 .with_cpu("Intel i9")
 .with_memory(32)
 .with_storage(2000)
 .with_gpu("NVIDIA RTX 3080")
 .with_os("Windows 11")
 .build())

office_pc = (ComputerBuilder()
 .with_cpu("Intel i5")
 .with_memory(16)
 .with_storage(512)
 .with_os("Ubuntu 22.04")
 .build()) # No GPU

print(gaming_pc) # Computer: CPU: Intel i9, Memory: 32GB, Storage: 2000GB, GPU: NVIDIA RTX
print(office_pc) # Computer: CPU: Intel i5, Memory: 16GB, Storage: 512GB, OS: Ubuntu 22.04

Adapter Pattern
class EuropeanSocket:
 """European power socket interface."""

 def voltage(self):
 """Return the voltage."""
 return 230

 def live(self):
 """Return the live pin."""
 return 1

```

```

def neutral(self):
 """Return the neutral pin."""
 return -1

def earth(self):
 """Return the earth pin."""
 return 0

class USSocket:
 """US power socket interface."""

 def voltage(self):
 """Return the voltage."""
 return 120

 def live(self):
 """Return the live pin."""
 return 1

 def neutral(self):
 """Return the neutral pin."""
 return -1

class USToEuropeAdapter:
 """
 Adapter for using US devices with European sockets.

 The adapter pattern allows incompatible interfaces to work together.
 It wraps an object of the incompatible class with a new adapter class
 that matches the expected interface.
 """

 def __init__(self, us_socket):
 """Initialize with a US socket."""
 self.us_socket = us_socket
 self.transformer = "Transformer: 120V to 230V"

 def voltage(self):
 """Return the adapted voltage."""
 return 230

 def live(self):
 """Return the live pin."""
 return self.us_socket.live()

 def neutral(self):

```

```

 """Return the neutral pin."""
 return self.us_socket.neutral()

 def earth(self):
 """Return the earth pin."""
 return 0 # Adding earth pin that US socket doesn't have

class ElectronicDevice:
 """A device that needs a power socket."""

 def __init__(self, name, socket):
 """Initialize with a name and power socket."""
 self.name = name
 self.socket = socket
 self.powered = False

 def power_up(self):
 """Power up the device using the socket."""
 voltage = self.socket.voltage()
 if voltage > 110 and voltage < 240:
 print(f"{self.name} is powered up with {voltage}V")
 self.powered = True
 else:
 print(f"Incompatible voltage: {voltage}V for {self.name}")

Test the Adapter pattern
european_socket = EuropeanSocket()
us_socket = USSocket()

European device with European socket (works fine)
european_device = ElectronicDevice("EU TV", european_socket)
european_device.power_up() # EU TV is powered up with 230V

US device with US socket (works fine)
us_device = ElectronicDevice("US Laptop", us_socket)
us_device.power_up() # US Laptop is powered up with 120V

US device with European socket (needs adapter)
adapter = USToEuropeAdapter(us_socket)
us_device_in_europe = ElectronicDevice("US Laptop in EU", adapter)
us_device_in_europe.power_up() # US Laptop in EU is powered up with 230V

Proxy Pattern
class Image:
 """Image interface."""

```

```

def display(self):
 """Display the image."""
 pass

class RealImage(Image):
 """
 Real image that is expensive to load.

 This is the actual object that the proxy represents.
 """

 def __init__(self, filename):
 """Initialize with a filename and load the image."""
 self.filename = filename
 self.load_from_disk()

 def load_from_disk(self):
 """Load the image from disk (expensive operation)."""
 print(f>Loading {self.filename} from disk")

 def display(self):
 """Display the image."""
 print(f>Displaying {self.filename}")

class ImageProxy(Image):
 """
 Proxy for the RealImage class.

 The proxy pattern provides a surrogate or placeholder for another object
 to control access to it. It can be used for lazy loading, access control,
 logging, etc.
 """

 def __init__(self, filename):
 """Initialize with a filename."""
 self.filename = filename
 self.real_image = None

 def display(self):
 """Display the image, loading it if necessary."""
 if self.real_image is None:
 # Lazy loading: only create the real image when needed
 self.real_image = RealImage(self.filename)

 self.real_image.display()

```

```

Test the Proxy pattern
Without proxy, the image is loaded immediately
print("Creating real image:")
real_image = RealImage("sample.jpg")
real_image.display()

With proxy, the image is loaded only when display() is called
print("\nCreating image proxy:")
proxy_image = ImageProxy("sample.jpg")
print("Image proxy created, but image not loaded yet")

print("\nDisplaying image through proxy:")
proxy_image.display() # Now the image is loaded

print("\nDisplaying image through proxy again:")
proxy_image.display() # Image already loaded, no loading message

Command Pattern
class Light:
 """A light that can be turned on and off."""

 def __init__(self, location):
 """Initialize with a location."""
 self.location = location
 self.is_on = False

 def turn_on(self):
 """Turn the light on."""
 self.is_on = True
 print(f"{self.location} light is now ON")

 def turn_off(self):
 """Turn the light off."""
 self.is_on = False
 print(f"{self.location} light is now OFF")

class Command:
 """
 Command interface.

 The command pattern encapsulates a request as an object, thereby letting
 you parameterize clients with different requests, queue or log requests,
 and support undoable operations.
 """

 def execute(self):

```



```

 """Execute the command."""
 pass

 def undo(self):
 """Undo the command."""
 pass

class LightOnCommand(Command):
 """Command to turn a light on."""

 def __init__(self, light):
 """Initialize with a light."""
 self.light = light

 def execute(self):
 """Turn the light on."""
 self.light.turn_on()

 def undo(self):
 """Undo turning the light on."""
 self.light.turn_off()

class LightOffCommand(Command):
 """Command to turn a light off."""

 def __init__(self, light):
 """Initialize with a light."""
 self.light = light

 def execute(self):
 """Turn the light off."""
 self.light.turn_off()

 def undo(self):
 """Undo turning the light off."""
 self.light.turn_on()

class RemoteControl:
 """A remote control with multiple buttons."""

 def __init__(self, slots=7):
 """Initialize with a number of slots."""
 self.on_commands = [None] * slots
 self.off_commands = [None] * slots
 self.undo_command = None

```

```

def set_command(self, slot, on_command, off_command):
 """Set commands for a slot."""
 self.on_commands[slot] = on_command
 self.off_commands[slot] = off_command

def press_on_button(self, slot):
 """Press the on button for a slot."""
 if self.on_commands[slot]:
 self.on_commands[slot].execute()
 self.undo_command = self.on_commands[slot]

def press_off_button(self, slot):
 """Press the off button for a slot."""
 if self.off_commands[slot]:
 self.off_commands[slot].execute()
 self.undo_command = self.off_commands[slot]

def press_undo_button(self):
 """Press the undo button."""
 if self.undo_command:
 self.undo_command.undo()
 self.undo_command = None

Test the Command pattern
living_room_light = Light("Living Room")
kitchen_light = Light("Kitchen")

living_room_light_on = LightOnCommand(living_room_light)
living_room_light_off = LightOffCommand(living_room_light)
kitchen_light_on = LightOnCommand(kitchen_light)
kitchen_light_off = LightOffCommand(kitchen_light)

remote = RemoteControl(2)
remote.set_command(0, living_room_light_on, living_room_light_off)
remote.set_command(1, kitchen_light_on, kitchen_light_off)

Use the remote
remote.press_on_button(0) # Living Room light is now ON
remote.press_off_button(0) # Living Room light is now OFF
remote.press_on_button(1) # Kitchen light is now ON
remote.press_undo_button() # Kitchen light is now OFF (undo)

Template Method Pattern
class DataProcessor:
 """
 Abstract base class defining a template method.

```

*The template method pattern defines the skeleton of an algorithm in a method, deferring some steps to subclasses. It lets subclasses redefine certain steps of an algorithm without changing the algorithm's structure.*

```
"""

def process(self, data):
 """
 Template method that defines the algorithm's skeleton.

 The steps are:
 1. Parse the data
 2. Validate the data
 3. Transform the data
 4. Analyze the data
 5. Report the results
 """
 parsed_data = self.parse(data)

 if not self.validate(parsed_data):
 print("Data validation failed.")
 return

 transformed_data = self.transform(parsed_data)
 results = self.analyze(transformed_data)
 self.report(results)

def parse(self, data):
 """Parse the data (hook method to be overridden)."""
 return data

def validate(self, data):
 """Validate the data (hook method to be overridden)."""
 return True

def transform(self, data):
 """Transform the data (hook method to be overridden)."""
 return data

def analyze(self, data):
 """Analyze the data (hook method to be overridden)."""
 return data

def report(self, results):
 """Report the results (hook method to be overridden)."""
```

```

 print(f"Results: {results}")

class NumericDataProcessor(DataProcessor):
 """Processor for numeric data."""

 def parse(self, data):
 """Parse string data into numbers."""
 print("Parsing numeric data")
 return [float(x) for x in data.split()]

 def validate(self, data):
 """Validate that all numbers are positive."""
 print("Validating numeric data")
 return all(x > 0 for x in data)

 def transform(self, data):
 """Scale the data."""
 print("Scaling numeric data")
 return [x / max(data) for x in data]

 def analyze(self, data):
 """Calculate statistics."""
 print("Analyzing numeric data")
 return {
 "mean": sum(data) / len(data),
 "min": min(data),
 "max": max(data)
 }

 def report(self, results):
 """Report statistics."""
 print(f"Numeric Data Analysis Results:")
 for key, value in results.items():
 print(f" {key}: {value:.4f}")

class TextDataProcessor(DataProcessor):
 """Processor for text data."""

 def parse(self, data):
 """Parse string into words."""
 print("Parsing text data")
 return data.lower().split()

 def validate(self, data):
 """Validate that there are words."""
 print("Validating text data")

```

```

 return len(data) > 0

 def transform(self, data):
 """Remove stop words."""
 print("Removing stop words")
 stop_words = {"the", "a", "an", "in", "of", "to"}
 return [word for word in data if word not in stop_words]

 def analyze(self, data):
 """Count word frequencies."""
 print("Analyzing text data")
 word_counts = {}
 for word in data:
 word_counts[word] = word_counts.get(word, 0) + 1
 return word_counts

 def report(self, results):
 """Report word frequencies."""
 print(f"Text Data Analysis Results:")
 for word, count in sorted(results.items(), key=lambda x: x[1], reverse=True)[:5]:
 print(f" {word}: {count}")

Test the Template Method pattern
numeric_processor = NumericDataProcessor()
text_processor = TextDataProcessor()

print("\nProcessing numeric data:")
numeric_processor.process("10 20 30 40 50")

print("\nProcessing text data:")
text_processor.process("The quick brown fox jumps over the lazy dog")

```

Design patterns provide reusable solutions to common problems, making your code more maintainable, flexible, and easier to understand. Python's dynamic nature and features like first-class functions, multiple inheritance, and duck typing allow for more concise and elegant implementations of these patterns compared to more rigid languages.

### Metaprogramming with `type()`, Dynamic Class Creation, `__new__`, Metaclasses, and `__metaclass__`

Metaprogramming is the practice of writing code that manipulates code itself. Python offers powerful metaprogramming capabilities, allowing you to create or modify classes and functions at runtime.

**Dynamic Class Creation with type()** The `type()` function can be used to create new classes dynamically:

```
Using type() to create a class dynamically
def say_hello(self):
 """Say hello with the instance's name."""
 return f"Hello, I'm {self.name}"

Create a class dynamically: type(name, bases, attributes)
Person = type('Person', (object,), {
 'name': '',
 'age': 0,
 'say_hello': say_hello,
 '__init__': lambda self, name, age: setattr(self, 'name', name) or setattr(self, 'age',
 })

Use the dynamically created class
person = Person("Alice", 30)
print(person.say_hello()) # Hello, I'm Alice
print(person.age) # 30
print(type(person)) # <class '__main__.Person'>
print(Person.__bases__) # (<class 'object'>,,)

Creating subclasses dynamically
Employee = type('Employee', (Person,), {
 'job_title': '',
 '__init__': lambda self, name, age, job_title: (
 Person.__init__(self, name, age) or setattr(self, 'job_title', job_title)
),
 'describe': lambda self: f"{self.say_hello()} and I work as a {self.job_title}"
 })

employee = Employee("Bob", 25, "Developer")
print(employee.describe()) # Hello, I'm Bob and I work as a Developer
print(isinstance(employee, Person)) # True

A factory function for creating classes
def create_class(name, fields, methods=None):
 """Create a class with the given name, fields, and methods."""
 if methods is None:
 methods = {}

 def __init__(self, **kwargs):
 for field, default in fields.items():
 setattr(self, field, kwargs.get(field, default))
```

```

 attributes = {'__init__': __init__}
 attributes.update(methods)

 return type(name, (object,), attributes)

Using the factory function
Product = create_class('Product',
 {'name': '', 'price': 0.0, 'quantity': 0},
 {'total_value': lambda self: self.price * self.quantity})

product = Product(name="Widget", price=10.0, quantity=5)
print(product.total_value()) # 50.0

```

**Custom `__new__` Method** The `__new__` method is a special method responsible for creating and returning a new instance of a class. It's called before `__init__` and can be used for advanced instance creation logic:

```

class Singleton:
 """A singleton class using __new__."""

 _instance = None

 def __new__(cls, *args, **kwargs):
 """Create a new instance only if none exists."""
 if cls._instance is None:
 cls._instance = super().__new__(cls)
 return cls._instance

Singleton example
s1 = Singleton()
s2 = Singleton()
print(s1 is s2) # True

Immutable class example
class Immutable:
 """An immutable class that prevents attribute modification after creation."""

 def __new__(cls, *args, **kwargs):
 """Create a new instance with custom behavior."""
 instance = super().__new__(cls)
 instance._frozen = False # Allow initialization
 return instance

 def __init__(self, x, y):
 """Initialize the instance."""
 self.x = x

```

```

 self.y = y
 self._frozen = True # Prevent further modifications

 def __setattr__(self, name, value):
 """Control attribute assignment."""
 if hasattr(self, '_frozen') and self._frozen and name != '_frozen':
 raise AttributeError("Cannot modify immutable instance")
 super().__setattr__(name, value)

Immutable object example
point = Immutable(1, 2)
print(point.x, point.y) # 1 2

try:
 point.x = 3 # Will raise AttributeError
except AttributeError as e:
 print(f"Error: {e}")

Factory method using __new__
class Shape:
 """A shape class with a factory method."""

 def __new__(cls, shape_type, **kwargs):
 """Create a specific shape based on shape_type."""
 if cls is Shape: # Only intercept when creating a Shape directly
 if shape_type == "circle":
 return super().__new__(Circle)
 elif shape_type == "rectangle":
 return super().__new__(Rectangle)
 else:
 raise ValueError(f"Unknown shape type: {shape_type}")
 return super().__new__(cls) # Normal creation for subclasses

 def area(self):
 """Calculate the area (abstract method)."""
 raise NotImplementedError("Subclass must implement area()")

class Circle(Shape):
 """A circle shape."""

 def __init__(self, shape_type=None, radius=1):
 """Initialize with a radius."""
 self.radius = radius

 def area(self):
 """Calculate the circle's area."""

```



```

 import math
 return math.pi * self.radius ** 2

class Rectangle(Shape):
 """A rectangle shape."""

 def __init__(self, shape_type=None, width=1, height=1):
 """Initialize with width and height."""
 self.width = width
 self.height = height

 def area(self):
 """Calculate the rectangle's area."""
 return self.width * self.height

Factory usage
circle = Shape("circle", radius=5)
rectangle = Shape("rectangle", width=3, height=4)

print(type(circle)) # <class '__main__.Circle'>
print(circle.area()) # ~78.54
print(type(rectangle)) # <class '__main__.Rectangle'>
print(rectangle.area()) # 12

```

**Metaclasses** Metaclasses are “classes of classes”—they define how classes themselves behave. The default metaclass for all classes is `type`.

```

Basic metaclass example
class Meta(type):
 """A simple metaclass that adds a class attribute."""

 def __new__(mcs, name, bases, attrs):
 """Create a new class with additional attributes."""
 # Add a class attribute
 attrs['metamessage'] = f"Class {name} was created using Meta"

 # Print a message when the class is created
 print(f"Creating class {name} with metaclass Meta")

 # Create and return the class
 return super().__new__(mcs, name, bases, attrs)

Using the metaclass
class MyClass(metaclass=Meta):
 """A class that uses the Meta metaclass."""

```

```

def __init__(self, value):
 """Initialize with a value."""
 self.value = value

def get_value(self):
 """Return the value."""
 return self.value

The metaclass has added the class attribute
print(MyClass.metamessage) # "Class MyClass was created using Meta"

Create an instance
obj = MyClass(42)
print(obj.get_value()) # 42

Metaclass that registers all subclasses
class RegisterMeta(type):
 """A metaclass that registers all of its classes in a registry."""

 registry = {}

 def __new__(mcs, name, bases, attrs):
 """Create a new class and register it."""
 cls = super().__new__(mcs, name, bases, attrs)

 # Don't register the base class itself
 if name != 'RegisteredClass':
 RegisterMeta.registry[name] = cls
 print(f"Registered class {name}")

 return cls

class RegisteredClass(metaclass=RegisterMeta):
 """Base class using RegisterMeta."""
 pass

class Service1(RegisteredClass):
 """A service class."""
 pass

class Service2(RegisteredClass):
 """Another service class."""
 pass

The metaclass has registered the classes
print(RegisterMeta.registry) # {'Service1': <class 'Service1'>, 'Service2': <class 'Service2'>}

```

```

Create instances from the registry
service1 = RegisterMeta.registry['Service1']()
service2 = RegisterMeta.registry['Service2']()

Metaclass that validates class attributes
class ValidateMeta(type):
 """A metaclass that validates class attributes based on a schema."""

 def __new__(mcs, name, bases, attrs):
 """Create a new class after validating attributes."""
 # Check if the class has a schema
 if 'schema' in attrs:
 schema = attrs['schema']

 # Validate attributes against the schema
 for attr_name, attr_type in schema.items():
 if attr_name in attrs and not isinstance(attrs[attr_name], attr_type):
 raise TypeError(f"Attribute {attr_name} must be of type {attr_type.__name__}")

 return super().__new__(mcs, name, bases, attrs)

class Validated(metaclass=ValidateMeta):
 """Base class for validated classes."""
 pass

class User(Validated):
 """A user class with a schema for validation."""

 schema = {
 'default_name': str,
 'max_age': int,
 'roles': list
 }

 default_name = "Guest"
 max_age = 120
 roles = ["user"]

This would raise an error because of the schema
try:
 class InvalidUser(Validated):
 schema = {
 'default_age': int
 }
 default_age = "not an integer" # This will fail validation

```

```

except TypeError as e:
 print(f"Error: {e}")

Metaclass for custom attribute access
class AttributeMeta(type):
 """A metaclass that customizes attribute access for classes."""

 def __new__(mcs, name, bases, attrs):
 """Create a new class with custom attribute access."""
 # Convert attributes to uppercase
 uppercase_attrs = {
 key.upper() if not key.startswith('__') else key: value
 for key, value in attrs.items()
 }

 return super().__new__(mcs, name, bases, uppercase_attrs)

class UppercaseClass(metaclass=AttributeMeta):
 """A class with uppercase attributes."""

 name = "attribute"
 value = 42

Attributes have been converted to uppercase
print(UppercaseClass.NAME) # "attribute"
print(UppercaseClass.VALUE) # 42

try:
 print(UppercaseClass.name) # AttributeError
except AttributeError as e:
 print(f"Error: Attribute 'name' doesn't exist (it's 'NAME')")

Advanced metaclass example: Abstract Base Class
class ABCMeta(type):
 """A metaclass for defining Abstract Base Classes."""

 def __new__(mcs, name, bases, attrs):
 """Create a new class after checking abstract methods."""
 # Mark the class as abstract if it has abstract methods
 abstract_methods = {
 key for key, value in attrs.items()
 if hasattr(value, '__isabstractmethod__') and value.__isabstractmethod__
 }

 attrs['__abstract_methods__'] = abstract_methods

```

```

 # Create the class
 cls = super().__new__(mcs, name, bases, attrs)

 # Check if the class is being instantiated
 original_new = cls.__new__
 def __new__(cls, *args, **kwargs):
 if abstract_methods and cls.__name__ != name:
 raise TypeError(f"Can't instantiate abstract class {cls.__name__} "
 f"with abstract methods {'', ' '.join(abstract_methods)}")
 return original_new(cls, *args, **kwargs)

 cls.__new__ = __new__

 return cls

def abstractmethod(func):
 """Decorator to mark a method as abstract."""
 func.__isabstractmethod__ = True
 return func

class AbstractClass(metaclass=ABCMeta):
 """An abstract base class."""

 @abstractmethod
 def abstract_method(self):
 """An abstract method that must be implemented by subclasses."""
 pass

class Concrete(AbstractClass):
 """A concrete implementation of AbstractClass."""

 def abstract_method(self):
 """Implementation of the abstract method."""
 return "Implemented"

Create instances
try:
 abstract_obj = AbstractClass() # Should raise TypeError
except TypeError as e:
 print(f"Error: {e}")

concrete_obj = Concrete()
print(concrete_obj.abstract_method()) # "Implemented"

```

**Dynamic Attribute Access with `__getattr__`, `__setattr__`, and `__delattr__`** These methods allow you to customize how attributes are accessed, set, and deleted:

```
class DynamicAttributes:
 """A class that handles attributes dynamically."""

 def __init__(self, **kwargs):
 """Initialize with keyword arguments."""
 self.__dict__.update(kwargs)

 def __getattr__(self, name):
 """Called when an attribute lookup fails."""
 print(f"Getting non-existent attribute '{name}'")
 # Create the attribute on the fly with a default value
 setattr(self, name, None)
 return None

 def __setattr__(self, name, value):
 """Called when an attribute is set."""
 print(f"Setting attribute '{name}' to {value}")
 # Use the built-in setattr to avoid infinite recursion
 super().__setattr__(name, value)

 def __delattr__(self, name):
 """Called when an attribute is deleted."""
 print(f"Deleting attribute '{name}'")
 # Use the built-in delattr to avoid infinite recursion
 super().__delattr__(name)

Using the dynamic attributes
obj = DynamicAttributes(name="Dynamic", age=30)
print(obj.name) # "Dynamic"
print(obj.address) # None (created on the fly)
obj.job = "Developer"
del obj.age

Dynamic attribute proxy
class AttributeProxy:
 """A proxy that forwards attribute access to another object."""

 def __init__(self, target):
 """Initialize with a target object."""
 self._target = target

 def __getattr__(self, name):
```

```

 """Forward attribute lookup to the target."""
 print(f"Proxying attribute lookup: {name}")
 return getattr(self._target, name)

 def __setattr__(self, name, value):
 """Forward attribute setting to the target if not starting with '_'."""
 if name.startswith('_'):
 super().__setattr__(name, value)
 else:
 print(f"Proxying attribute setting: {name} = {value}")
 setattr(self._target, name, value)

 def __delattr__(self, name):
 """Forward attribute deletion to the target if not starting with '_'."""
 if name.startswith('_'):
 super().__delattr__(name)
 else:
 print(f"Proxying attribute deletion: {name}")
 delattr(self._target, name)

Example target class
class Person:
 """A person class to be proxied."""

 def __init__(self, name, age):
 """Initialize with name and age."""
 self.name = name
 self.age = age

 def greet(self):
 """Return a greeting."""
 return f"Hello, I'm {self.name}"

Using the proxy
person = Person("Alice", 30)
proxy = AttributeProxy(person)

print(proxy.name) # "Alice" (proxied)
proxy.age = 31 # Sets person.age to 31 (proxied)
print(proxy.greet()) # "Hello, I'm Alice" (proxied)
print(person.age) # 31 (changed through proxy)

Attribute validation using __setattr__
class ValidatedPerson:
 """A person class with attribute validation."""

```

```

def __init__(self, name, age, email):
 """Initialize with name, age, and email."""
 # Call __setattr__ explicitly for validation
 self.name = name
 self.age = age
 self.email = email

def __setattr__(self, name, value):
 """Validate attributes before setting them."""
 if name == 'name':
 if not isinstance(value, str):
 raise TypeError("Name must be a string")
 if not value:
 raise ValueError("Name cannot be empty")
 elif name == 'age':
 if not isinstance(value, int):
 raise TypeError("Age must be an integer")
 if value < 0 or value > 120:
 raise ValueError("Age must be between 0 and 120")
 elif name == 'email':
 if not isinstance(value, str):
 raise TypeError("Email must be a string")
 if '@' not in value:
 raise ValueError("Invalid email format")

 super().__setattr__(name, value)

Using the validated person
try:
 person = ValidatedPerson("Bob", 25, "bob@example.com")
 print(f"{person.name}, {person.age}, {person.email}")

 person.age = 130 # Will raise ValueError
except ValueError as e:
 print(f"Error: {e}")

try:
 person.email = "invalid-email" # Will raise ValueError
except ValueError as e:
 print(f"Error: {e}")

```

**Advanced Descriptor Examples** Descriptors provide a powerful way to customize how attribute access works:

```

class Verbose:
 """A descriptor that logs accesses to an attribute."""

```



```

def __init__(self, name=None):
 """Initialize with an optional name."""
 self.name = name

def __set_name__(self, owner, name):
 """Called when the descriptor is assigned to a class."""
 # If no name was given in __init__, use the attribute name
 if self.name is None:
 self.name = name

def __get__(self, instance, owner):
 """Called when the attribute is accessed."""
 if instance is None:
 return self
 value = instance.__dict__.get(self.name)
 print(f"Getting {self.name}: {value}")
 return value

def __set__(self, instance, value):
 """Called when the attribute is set."""
 print(f"Setting {self.name} to {value}")
 instance.__dict__[self.name] = value

def __delete__(self, instance):
 """Called when the attribute is deleted."""
 print(f"Deleting {self.name}")
 if self.name in instance.__dict__:
 del instance.__dict__[self.name]

class TypedAttribute:
 """A descriptor that enforces a specific type for an attribute."""

 def __init__(self, expected_type, default=None):
 """Initialize with expected type and optional default value."""
 self.expected_type = expected_type
 self.default = default
 self.name = None

 def __set_name__(self, owner, name):
 """Called when the descriptor is assigned to a class."""
 self.name = name

 def __get__(self, instance, owner):
 """Called when the attribute is accessed."""
 if instance is None:

```

```

 return self
 return instance.__dict__.get(self.name, self.default)

def __set__(self, instance, value):
 """Called when the attribute is set."""
 if value is not None and not isinstance(value, self.expected_type):
 raise TypeError(f"{self.name} must be a {self.expected_type.__name__}")
 instance.__dict__[self.name] = value

class RangeValidator:
 """A descriptor that validates numeric values within a range."""

 def __init__(self, minimum=None, maximum=None):
 """Initialize with minimum and maximum bounds."""
 self.minimum = minimum
 self.maximum = maximum
 self.name = None

 def __set_name__(self, owner, name):
 """Called when the descriptor is assigned to a class."""
 self.name = name

 def __get__(self, instance, owner):
 """Called when the attribute is accessed."""
 if instance is None:
 return self
 return instance.__dict__.get(self.name)

 def __set__(self, instance, value):
 """Called when the attribute is set."""
 if not isinstance(value, (int, float)):
 raise TypeError(f"{self.name} must be a number")

 if self.minimum is not None and value < self.minimum:
 raise ValueError(f"{self.name} must be at least {self.minimum}")

 if self.maximum is not None and value > self.maximum:
 raise ValueError(f"{self.name} must be at most {self.maximum}")

 instance.__dict__[self.name] = value

class Employee:
 """An employee class with validated attributes."""

 name = TypedAttribute(str)
 age = RangeValidator(minimum=18, maximum=65)

```

```

salary = RangeValidator(minimum=0)
department = TypedAttribute(str, default="General")
status = Verbose()

def __init__(self, name, age, salary, department=None, status="Active"):
 """Initialize an employee."""
 self.name = name
 self.age = age
 self.salary = salary
 if department is not None:
 self.department = department
 self.status = status

Using the employee with descriptors
employee = Employee("Alice", 30, 50000, "Engineering")
print(employee.name) # Alice
print(employee.department) # Engineering
print(employee.status) # Getting status: Active, Active

employee.salary = 60000 # Valid
print(employee.salary) # 60000

try:
 employee.age = 70 # Will raise ValueError (> 65)
except ValueError as e:
 print(f"Error: {e}")

try:
 employee.name = 123 # Will raise TypeError (not a string)
except TypeError as e:
 print(f"Error: {e}")

Property Factory using descriptors
def typed_property(name, expected_type):
 """Create a property with type checking."""
 storage_name = f"_{name}"

 @property
 def prop(self):
 return getattr(self, storage_name)

 @prop.setter
 def prop(self, value):
 if not isinstance(value, expected_type):
 raise TypeError(f"{name} must be a {expected_type.__name__}")
 setattr(self, storage_name, value)

```

```

 return prop

class Product:
 """A product class using the typed_property factory."""

 name = typed_property("name", str)
 price = typed_property("price", (int, float))

 def __init__(self, name, price):
 self.name = name
 self.price = price

Using the product with typed properties
product = Product("Widget", 10.0)
print(product.name) # Widget
print(product.price) # 10.0

try:
 product.price = "invalid" # Will raise TypeError
except TypeError as e:
 print(f"Error: {e}")

```

### Descriptors: `__get__`, `__set__`, `__delete__`, and Use Cases

Descriptors are a powerful way to create reusable property behavior in Python. They're the mechanism behind properties, class methods, and static methods.

```

class LazyProperty:
 """A descriptor that computes a value only when needed and caches it."""

 def __init__(self, func):
 """Initialize with the function to compute the value."""
 self.func = func
 self.name = func.__name__

 def __get__(self, instance, owner):
 """Called when the attribute is accessed."""
 if instance is None:
 return self

 # Compute the value if it doesn't exist
 if self.name not in instance.__dict__:
 instance.__dict__[self.name] = self.func(instance)

 return instance.__dict__[self.name]

```

```

class Circle:
 """A circle with lazily computed properties."""

 def __init__(self, radius):
 """Initialize with radius."""
 self.radius = radius

 @LazyProperty
 def area(self):
 """Compute the area (expensive operation)."""
 print("Computing area...")
 import math
 return math.pi * self.radius ** 2

 @LazyProperty
 def circumference(self):
 """Compute the circumference (expensive operation)."""
 print("Computing circumference...")
 import math
 return 2 * math.pi * self.radius

Using the lazy properties
circle = Circle(5)
print(f"Radius: {circle.radius}")

First access computes the value
print(f"Area: {circle.area}") # Computing area..., 78.54...

Second access uses the cached value
print(f"Area again: {circle.area}") # 78.54... (no computation message)

First access to another property
print(f"Circumference: {circle.circumference}") # Computing circumference..., 31.42...

Unit conversion descriptor
class Unit:
 """A descriptor for converting between units."""

 def __init__(self, unit_from, unit_to, factor):
 """Initialize with conversion units and factor."""
 self.unit_from = unit_from
 self.unit_to = unit_to
 self.factor = factor
 self.name = None
 self.internal_name = None

```

```

def __set_name__(self, owner, name):
 """Called when the descriptor is assigned to a class."""
 self.name = name
 self.internal_name = f"_{name}"

def __get__(self, instance, owner):
 """Called when the attribute is accessed."""
 if instance is None:
 return self

 # Return the value in the display unit
 internal_value = getattr(instance, self.internal_name, 0)
 display_value = internal_value / self.factor
 return display_value

def __set__(self, instance, value):
 """Called when the attribute is set."""
 # Convert the value to the internal unit
 internal_value = value * self.factor
 setattr(instance, self.internal_name, internal_value)

def __delete__(self, instance):
 """Called when the attribute is deleted."""
 if hasattr(instance, self.internal_name):
 delattr(instance, self.internal_name)

class Distance:
 """A class for distances with unit conversion."""

 # 1 kilometer = 1000 meters
 kilometers = Unit("km", "m", 1000)

 # 1 mile = 1609.34 meters
 miles = Unit("mi", "m", 1609.34)

 # The base unit is meters
 meters = Unit("m", "m", 1)

 def __init__(self, **kwargs):
 """Initialize with distances in various units."""
 for unit, value in kwargs.items():
 setattr(self, unit, value)

Using the unit conversion
distance = Distance(kilometers=5)

```

```

print(f"{distance.kilometers} km") # 5.0 km
print(f"{distance.meters} m") # 5000.0 m
print(f"{distance.miles} mi") # 3.1069 mi

Change the distance in a different unit
distance.miles = 10
print(f"{distance.kilometers} km") # 16.0934 km
print(f"{distance.meters} m") # 16093.4 m
print(f"{distance.miles} mi") # 10.0 mi

Database field descriptor
class Field:
 """A descriptor for database fields."""

 def __init__(self, field_type, required=False, default=None):
 """Initialize with field type, required flag, and default value."""
 self.field_type = field_type
 self.required = required
 self.default = default
 self.name = None

 def __set_name__(self, owner, name):
 """Called when the descriptor is assigned to a class."""
 self.name = name

 def __get__(self, instance, owner):
 """Called when the attribute is accessed."""
 if instance is None:
 return self

 return instance.__dict__.get(self.name, self.default)

 def __set__(self, instance, value):
 """Called when the attribute is set."""
 if value is None and self.required:
 raise ValueError(f"{self.name} is required")

 if value is not None and not isinstance(value, self.field_type):
 raise TypeError(f"{self.name} must be a {self.field_type.__name__}")

 instance.__dict__[self.name] = value

class Model:
 """Base class for database models."""

 @classmethod

```

```

def fields(cls):
 """Return a dictionary of all fields."""
 return {
 name: descriptor
 for name, descriptor in cls.__dict__.items()
 if isinstance(descriptor, Field)
 }

def validate(self):
 """Validate all fields."""
 for name, field in self.__class__.fields().items():
 value = getattr(self, name)
 if field.required and value is None:
 raise ValueError(f"{name} is required")

def to_dict(self):
 """Convert to a dictionary."""
 return {
 name: getattr(self, name)
 for name in self.__class__.fields()
 }

class User(Model):
 """A user model with fields."""

 id = Field(int)
 name = Field(str, required=True)
 email = Field(str, required=True)
 age = Field(int)
 active = Field(bool, default=True)

Using the model
user = User()
user.id = 1
user.name = "Alice"
user.email = "alice@example.com"
user.age = 30

print(user.to_dict()) # {'id': 1, 'name': 'Alice', 'email': 'alice@example.com', 'age': 30}

try:
 # Validate will fail if required fields are missing
 empty_user = User()
 empty_user.validate()
except ValueError as e:
 print(f"Error: {e}")

```



```

try:
 # Type checking
 user.age = "thirty"
except TypeError as e:
 print(f"Error: {e}")

Cached property with invalidation
class CachedProperty:
 """A property that caches its value but can be invalidated."""

 def __init__(self, func):
 """Initialize with the function to compute the value."""
 self.func = func
 self.name = func.__name__
 self.cache_name = f"_cache_{func.__name__}"

 def __get__(self, instance, owner):
 """Called when the attribute is accessed."""
 if instance is None:
 return self

 if not hasattr(instance, self.cache_name):
 setattr(instance, self.cache_name, self.func(instance))

 return getattr(instance, self.cache_name)

 def invalidate(self, instance):
 """Invalidate the cached value."""
 if hasattr(instance, self.cache_name):
 delattr(instance, self.cache_name)

class Rectangle:
 """A rectangle with cached properties that depend on dimensions."""

 def __init__(self, width, height):
 """Initialize with width and height."""
 self.width = width
 self.height = height

 @CachedProperty
 def area(self):
 """Calculate the area (cached)."""
 print("Computing area...")
 return self.width * self.height

```

```

@CachedProperty
def perimeter(self):
 """Calculate the perimeter (cached)."""
 print("Computing perimeter...")
 return 2 * (self.width + self.height)

Method to update dimensions and invalidate cached properties
def resize(self, width, height):
 """Resize the rectangle and invalidate cached properties."""
 self.width = width
 self.height = height

 # Invalidate cached properties
 self.__class__.area.invalidate(self)
 self.__class__.perimeter.invalidate(self)

Using the cached properties
rect = Rectangle(3, 4)
print(f"Area: {rect.area}") # Computing area..., 12
print(f"Area again: {rect.area}") # 12 (cached)
print(f"Perimeter: {rect.perimeter}") # Computing perimeter..., 14

Resize and check that caches are invalidated
rect.resize(5, 6)
print(f"New area: {rect.area}") # Computing area..., 30
print(f"New perimeter: {rect.perimeter}") # Computing perimeter..., 22

```

## 11. Memory Optimization and Extensions

### `__slots__` for Memory Optimization

The `__slots__` attribute allows you to explicitly state which instance attributes you expect your objects to have, saving memory by avoiding the creation of a `__dict__` for each instance.

```

import sys
import time

Standard class with dictionary
class StandardPerson:
 """A standard class that uses a dictionary for attributes."""

 def __init__(self, name, age, email, address):
 self.name = name
 self.age = age
 self.email = email

```

```

 self.address = address

Class with __slots__
class SlottedPerson:
 """A memory-efficient class that uses __slots__."""

 __slots__ = ['name', 'age', 'email', 'address']

 def __init__(self, name, age, email, address):
 self.name = name
 self.age = age
 self.email = email
 self.address = address

Memory comparison
def compare_memory_usage():
 """Compare memory usage of standard vs. slotted classes."""
 # Create instances
 standard = StandardPerson("Alice", 30, "alice@example.com", "123 Main St")
 slotted = SlottedPerson("Alice", 30, "alice@example.com", "123 Main St")

 # Check memory usage
 standard_size = sys.getsizeof(standard) + sys.getsizeof(standard.__dict__)
 slotted_size = sys.getsizeof(slotted)

 print(f"Standard instance size: {standard_size} bytes")
 print(f"Slotted instance size: {slotted_size} bytes")
 print(f"Memory saved: {standard_size - slotted_size} bytes")
 print(f"Saving percentage: {(standard_size - slotted_size) / standard_size * 100:.2f}%")

 # Create many instances to demonstrate the impact
 count = 100000

 # Standard instances
 start = time.time()
 standard_instances = [
 StandardPerson(f"Person {i}", i % 100, f"person{i}@example.com", f"Address {i}")
 for i in range(count)
]
 standard_time = time.time() - start

 # Slotted instances
 start = time.time()
 slotted_instances = [
 SlottedPerson(f"Person {i}", i % 100, f"person{i}@example.com", f"Address {i}")
 for i in range(count)
]

```

```

]
slotted_time = time.time() - start

Approximate memory usage (not entirely accurate)
standard_total = standard_size * count
slotted_total = slotted_size * count

print(f"\nCreating {count} instances:")
print(f"Standard instances time: {standard_time:.2f} seconds")
print(f"Slotted instances time: {slotted_time:.2f} seconds")
print(f"Time saving: {(standard_time - slotted_time) / standard_time * 100:.2f}%")

print(f"\nApproximate memory usage for {count} instances:")
print(f"Standard instances: {standard_total / (1024 * 1024):.2f} MB")
print(f"Slotted instances: {slotted_total / (1024 * 1024):.2f} MB")
print(f"Memory saved: {(standard_total - slotted_total) / (1024 * 1024):.2f} MB")

Inheritance with __slots__
class Person:
 """Base person class with slots."""

 __slots__ = ['name', 'age']

 def __init__(self, name, age):
 self.name = name
 self.age = age

class Employee(Person):
 """Employee class that inherits from Person and adds more slots."""

 __slots__ = ['job_title', 'salary']

 def __init__(self, name, age, job_title, salary):
 super().__init__(name, age)
 self.job_title = job_title
 self.salary = salary

def demonstrate_slots_inheritance():
 """Demonstrate how __slots__ works with inheritance."""
 # Create an employee
 employee = Employee("Alice", 30, "Developer", 100000)

 # Access attributes
 print(f"Name: {employee.name}")
 print(f"Age: {employee.age}")
 print(f"Job Title: {employee.job_title}")

```

```

print(f"Salary: {employee.salary}")

Show that we can't add arbitrary attributes
try:
 employee.address = "123 Main St"
except AttributeError as e:
 print(f"Error adding attribute: {e}")

Check what slots are available
all_slots = set()
for cls in Employee.__mro__:
 if hasattr(cls, '__slots__'):
 # __slots__ can be string or iterable
 slots = cls.__slots__
 if isinstance(slots, str):
 all_slots.add(slots)
 else:
 all_slots.update(slots)

print(f"All available slots: {all_slots}")

Check if the instance has a __dict__
has_dict = hasattr(employee, '__dict__')
print(f"Instance has __dict__: {has_dict}")

Class with both __slots__ and __dict__
class FlexiblePerson:
 """A class with both __slots__ and __dict__."""

 __slots__ = ['name', 'age', '__dict__']

 def __init__(self, name, age, **kwargs):
 self.name = name
 self.age = age
 for key, value in kwargs.items():
 setattr(self, key, value)

def demonstrate_flexible_slots():
 """Demonstrate a class with both __slots__ and __dict__."""
 # Create an instance with both slot attributes and dynamic attributes
 flexible = FlexiblePerson("Alice", 30, email="alice@example.com", address="123 Main St")

 # Access attributes
 print(f"Name (slot): {flexible.name}")
 print(f"Age (slot): {flexible.age}")
 print(f"Email (dict): {flexible.email}")

```

```

print(f"Address (dict): {flexible.address}")

Add another dynamic attribute
flexible.phone = "555-1234"
print(f"Phone (dict): {flexible.phone}")

Check memory usage (will be higher than pure slots)
print(f"Instance size: {sys.getsizeof(flexible) + sys.getsizeof(flexible.__dict__)} bytes")

Weaknesses and limitations of __slots__
def slots_limitations():
 """Discuss the limitations of using __slots__."""
 print("Limitations of __slots__:")
 print("1. Cannot add attributes not defined in __slots__")
 print("2. No instance __dict__ unless explicitly included in __slots__")
 print("3. Cannot use __slots__ with certain metaclasses")
 print("4. Classes with __slots__ cannot use multiple inheritance unless all parent classes have __slots__")
 print("5. If a parent class has a __dict__, the child's __slots__ won't save memory")
 print("6. Properties or descriptors for slot attributes still use memory")
 print("7. Slots can make serialization/deserialization more complex")

When to use __slots__
def when_to_use_slots():
 """Discuss when to use __slots__."""
 print("When to use __slots__:")
 print("1. When you're creating many instances of a class (thousands or more)")
 print("2. When instance attributes are fixed and known in advance")
 print("3. When memory efficiency is critical")
 print("4. When attribute access speed is important")
 print("5. When you want to prevent users from adding arbitrary attributes")

 print("\nWhen NOT to use __slots__:")
 print("1. When you need dynamic attribute assignment")
 print("2. When you're using multiple inheritance with complex hierarchies")
 print("3. When the class design is still evolving")
 print("4. When memory usage isn't a concern")
 print("5. When you need to use weakref.WeakKeyDictionary with your instances")

Run demonstrations
compare_memory_usage()
demonstrate_slots_inheritance()
demonstrate_flexible_slots()
slots_limitations()
when_to_use_slots()

```

## Monkey Patching and Dynamic Attributes

Monkey patching refers to the practice of modifying classes or objects at runtime. Python's dynamic nature makes this possible, though it should be used with caution.

```
Original class
class Person:
 """A person class."""

 def __init__(self, name):
 self.name = name

 def greet(self):
 """Return a greeting."""
 return f"Hello, I'm {self.name}"

Create an instance of the original class
person = Person("Alice")
print(person.greet()) # Hello, I'm Alice

Monkey patching at the class level
def new_greet(self):
 """A new greeting method."""
 return f"Hi there! My name is {self.name}"

Replace the original method with our new method
Person.greet = new_greet

The existing instance now uses the new method
print(person.greet()) # Hi there! My name is Alice

Create a new instance - it also uses the new method
person2 = Person("Bob")
print(person2.greet()) # Hi there! My name is Bob

Monkey patching at the instance level
def special_greet(self):
 """A special greeting for one instance."""
 return f"Greetings, {self.name} here!"

Bind the new method to a specific instance
import types
person.greet = types.MethodType(special_greet, person)

Now the instances have different behaviors
```

```

print(person.greet()) # Greetings, Alice here!
print(person2.greet()) # Hi there! My name is Bob

Adding new methods to a class
def say_goodbye(self):
 """Say goodbye."""
 return f"Goodbye from {self.name}"

Add the new method to the class
Person.say_goodbye = say_goodbye

Both instances now have the new method
print(person.say_goodbye()) # Goodbye from Alice
print(person2.say_goodbye()) # Goodbye from Bob

Adding new attributes to instances
person.age = 30
print(f"{person.name} is {person.age} years old") # Alice is 30 years old

Adding new attributes to classes
Person.default_age = 25
print(f"Default age: {Person.default_age}") # Default age: 25

New instances get the class attribute as a default
person3 = Person("Charlie")
print(f"Default age for {person3.name}: {Person.default_age}") # Default age for Charlie: 25

Monkey patching built-in types (use with extreme caution)
def custom_upper(self):
 """Custom upper method that adds an exclamation mark."""
 return self.upper() + "!"

This is generally not recommended for built-in types
str.custom_upper = custom_upper

Now all strings have our custom method
print("hello".custom_upper()) # HELLO!

Dynamic attribute access with __getattr__ and __setattr__
class DynamicObject:
 """A class that creates attributes dynamically."""

 def __init__(self):
 self._storage = {}

 def __getattr__(self, name):

```



```

 """Called when attribute lookup fails."""
 print(f"Getting dynamic attribute: {name}")
 return self._storage.get(name)

 def __setattr__(self, name, value):
 """Called when setting an attribute."""
 if name == "_storage":
 # Allow setting the internal storage
 super().__setattr__(name, value)
 else:
 print(f"Setting dynamic attribute: {name} = {value}")
 self._storage[name] = value

Using the dynamic object
dynamic = DynamicObject()
dynamic.name = "Dynamic Object"
dynamic.value = 42
print(dynamic.name) # Getting dynamic attribute: name, Dynamic Object
print(dynamic.value) # Getting dynamic attribute: value, 42
print(dynamic.nonexistent) # Getting dynamic attribute: nonexistent, None

Monkey patching to add features to third-party libraries
Example with a fictional library
class ThirdPartyClass:
 """A class from a third-party library."""

 def original_method(self):
 """Original method."""
 return "Original behavior"

Suppose we need to extend this class but can't subclass it
def extended_method(self):
 """Add extended functionality."""
 original_result = self.original_method()
 return f"{original_result} with extensions"

Apply the monkey patch
ThirdPartyClass.extended_method = extended_method

Use the extended functionality
third_party = ThirdPartyClass()
print(third_party.original_method()) # Original behavior
print(third_party.extended_method()) # Original behavior with extensions

Best practices for monkey patching
def monkey_patchting_best_practices():

```

```

"""Discuss best practices for monkey patching."""
print("Monkey Patching Best Practices:")
print("1. Only monkey patch your own code or when absolutely necessary")
print("2. Document all monkey patches clearly")
print("3. Consider using decorators or mixins instead when possible")
print("4. Apply patches at application startup before code is used")
print("5. Use namespace packages to isolate patches")
print("6. Write tests specifically for monkey-patched behavior")
print("7. Consider using libraries like 'forbiddenfruit' for safer patching")
print("8. Be aware of backward compatibility issues if the target library changes")
print("9. Use feature detection to check if patching is needed")
print("10. Avoid patching built-in types and standard library modules when possible")

monkey_patching_best_practices()

```

## C Extensions and Interfacing with C using ctypes/cffi

Python can be extended with C code for performance-critical sections. The ctypes and cffi libraries provide ways to call C code from Python.

```

import ctypes
import platform
import time
import math

Using ctypes to call C standard library functions
def ctypes_basic_example():
 """Demonstrate basic ctypes usage with the C standard library."""
 print("Basic ctypes example:")

 # Load the C standard library
 if platform.system() == "Windows":
 libc = ctypes.cdll.msvcrt
 else:
 libc = ctypes.CDLL("libc.so.6")

 # Call the strlen function to get string length
 libc.strlen.argtypes = [ctypes.c_char_p]
 libc.strlen.restype = ctypes.c_size_t

 message = b"Hello, C!"
 length = libc.strlen(message)
 print(f"Length of '{message.decode()}': {length}")

 # Call the time function to get the current time
 libc.time.argtypes = [ctypes.POINTER(ctypes.c_long)]

```

```

libc.time.restype = ctypes.c_long

current_time = libc.time(None)
print(f"Current time (seconds since epoch): {current_time}")

Performance comparison between Python and C
def compute_sum_python(n):
 """Compute the sum of 1 to n in pure Python."""
 return sum(range(1, n + 1))

Create a simple C shared library for comparison
'''
// sum.c
#include <stdio.h>

long long compute_sum(long long n) {
 long long sum = 0;
 for (long long i = 1; i <= n; i++) {
 sum += i;
 }
 return sum;
}
'''

def performance_comparison():
 """Compare performance between Python and C implementations."""
 print("\nPerformance comparison:")

 n = 100000000 # A large number for meaningful comparison

 # Python implementation
 start = time.time()
 result_py = compute_sum_python(n)
 python_time = time.time() - start
 print(f"Python result: {result_py}")
 print(f"Python time: {python_time:.6f} seconds")

 # C implementation using ctypes
 try:
 # Try to load the compiled library
 if platform.system() == "Windows":
 lib = ctypes.CDLL("./sum.dll")
 else:
 lib = ctypes.CDLL("./sum.so")

 lib.compute_sum.argtypes = [ctypes.c_longlong]

```

```

lib.compute_sum.restype = ctypes.c_longlong

start = time.time()
result_c = lib.compute_sum(n)
c_time = time.time() - start

print(f"C result: {result_c}")
print(f"C time: {c_time:.6f} seconds")
print(f"Speedup: {python_time / c_time:.2f}x")
except (OSError, AttributeError) as e:
 print(f"Could not load C library: {e}")
 print("Note: You need to compile the C file with:")
 if platform.system() == "Windows":
 print(" gcc -shared -o sum.dll sum.c")
 else:
 print(" gcc -shared -fPIC -o sum.so sum.c")

Passing complex data to C
def passing_complex_data():
 """Demonstrate passing complex data between Python and C."""
 print("\nPassing complex data:")

 # Define a C-compatible structure
 class Point(ctypes.Structure):
 fields = [
 ("x", ctypes.c_double),
 ("y", ctypes.c_double)
]

 # Create and populate a Point structure
 point = Point(x=3.0, y=4.0)
 print(f"Point: ({point.x}, {point.y})")

 # Create an array of structures
 PointArray = Point * 3
 points = PointArray(
 Point(1.0, 2.0),
 Point(3.0, 4.0),
 Point(5.0, 6.0)
)

 for i, p in enumerate(points):
 print(f"Point {i}: ({p.x}, {p.y})")

Passing arrays to C
Example function: double calculate_distance(double x, double y);

```

```

Assuming we have a compiled library with this function
try:
 if platform.system() == "Windows":
 lib = ctypes.CDLL("./math_funcs.dll")
 else:
 lib = ctypes.CDLL("./math_funcs.so")

 # Define the function signature
 lib.calculate_distance.argtypes = [ctypes.c_double, ctypes.c_double]
 lib.calculate_distance.restype = ctypes.c_double

 # Call the C function
 distance = lib.calculate_distance(point.x, point.y)
 print(f"Distance from origin: {distance}")
except (OSError, AttributeError) as e:
 print(f"Could not load C library: {e}")
 # Simulate the calculation in Python
 distance = math.sqrt(point.x**2 + point.y**2)
 print(f"Calculated in Python - Distance from origin: {distance}")

Using callbacks (Python functions called from C)
def callbacks_example():
 """Demonstrate using Python callbacks from C code."""
 print("\nCallbacks example:")

 # Python function to be called from C
 @ctypes.CFUNCTYPE(ctypes.c_double, ctypes.c_double)
 def callback_func(x):
 """A Python function to be called from C."""
 result = math.sin(x)
 print(f"Python callback: sin({x}) = {result}")
 return result

 # Try to load a C library that uses callbacks
 try:
 if platform.system() == "Windows":
 lib = ctypes.CDLL("./callback_example.dll")
 else:
 lib = ctypes.CDLL("./callback_example.so")

 # Function that takes a callback: double integrate(double (*f)(double), double a, double b)
 lib.integrate.argtypes = [ctypes.c_void_p, ctypes.c_double, ctypes.c_double, ctypes.c_double]
 lib.integrate.restype = ctypes.c_double

 # Call the C function with our Python callback
 result = lib.integrate(callback_func, 0.0, math.pi, 1000)

```

```

 print(f"Integral of sin(x) from 0 to : {result}")
except (OSError, AttributeError) as e:
 print(f"Could not load C library: {e}")
 # Simulate the integration in Python
 def integrate_python(f, a, b, n):
 """Simple integration using rectangles."""
 width = (b - a) / n
 total = 0.0
 for i in range(n):
 x = a + (i + 0.5) * width
 total += f(x) * width
 return total

 result = integrate_python(callback_func, 0.0, math.pi, 1000)
 print(f"Calculated in Python - Integral of sin(x) from 0 to : {result}")

Introduction to CFFI (C Foreign Function Interface)
def cffi_introduction():
 """Introduce CFFI as an alternative to ctypes."""
 print("\nCFFI Introduction:")

 try:
 import cffi

 # Create a CFFI instance
 ffi = cffi.FFI()

 # Define the C function signature
 ffi.cdef("""
 double sqrt(double x);
 double sin(double x);
 double cos(double x);
 """)

 # Load the C library
 if platform.system() == "Windows":
 C = ffi.dlopen("msvcrt")
 else:
 C = ffi.dlopen("libm.so.6") # Math library on Linux

 # Call C functions
 x = 16.0
 sqrt_x = C.sqrt(x)
 sin_x = C.sin(1.0)
 cos_x = C.cos(1.0)

```

```

print(f"CFFI results:")
print(f" sqrt({x}) = {sqrt_x}")
print(f" sin(1.0) = {sin_x}")
print(f" cos(1.0) = {cos_x}")

Define and use a structure with CFFI
ffi.cdef("""
 typedef struct {
 double x;
 double y;
 } Point;

 static double calculate_point_distance(Point p) {
 return sqrt(p.x * p.x + p.y * p.y);
 }
""")

Compile the C function directly
lib = ffi.verify("""
 #include <math.h>

 typedef struct {
 double x;
 double y;
 } Point;

 double calculate_point_distance(Point p) {
 return sqrt(p.x * p.x + p.y * p.y);
 }
""")

Create a Point structure
point = ffi.new("Point *")
point.x = 3.0
point.y = 4.0

Call the C function
distance = lib.calculate_point_distance(point[0])
print(f"Distance from origin for point ({point.x}, {point.y}): {distance}")

except ImportError:
 print("CFFI not installed. You can install it with 'pip install cffi'")
except Exception as e:
 print(f"Error demonstrating CFFI: {e}")

Sample C extension

```

```

def c_extension_example():
 """Discuss how to write C extensions for Python."""
 print("\nC Extension Example:")
 print("To write a C extension for Python, you typically:")
 print("1. Create a C file implementing your functions")
 print("2. Create a setup.py file for building the extension")
 print("3. Build the extension with 'python setup.py build_ext --inplace'")
 print("4. Import and use the extension like a normal Python module")

 print("\nExample setup.py:")
 print("""
from setuptools import setup, Extension

module = Extension(
 'mymodule', # Extension name
 sources=['mymodule.c'], # C source files
 include_dirs=[], # Additional include directories
 library_dirs=[], # Additional library directories
 libraries=[] # Libraries to link with
)

setup(
 name='MyExtension',
 version='1.0',
 ext_modules=[module]
)
""")

 print("\nExample C file (mymodule.c):")
 print("""
#include <Python.h>

// Function to be called from Python
static PyObject* my_function(PyObject* self, PyObject* args) {
 int a, b;

 // Parse arguments from Python
 if (!PyArg_ParseTuple(args, "ii", &a, &b)) {
 return NULL; // Error parsing arguments
 }

 // Perform the calculation
 int result = a + b;

 // Return the result to Python
 return Py_BuildValue("i", result);
 """

```



```

}

// Method table
static PyMethodDef MyMethods[] = {
 {"add", my_function, METH_VARARGS, "Add two integers."},
 {NULL, NULL, 0, NULL} // Sentinel
};

// Module definition
static struct PyModuleDef mymodule = {
 PyModuleDef_HEAD_INIT,
 "mymodule", // Module name
 "A C extension for Python", // Module docstring
 -1, // Size of per-interpreter state or -1
 MyMethods // Method table
};

// Module initialization function
PyMODINIT_FUNC PyInit_mymodule(void) {
 return PyModule_Create(&mymodule);
}

"""

 print("\nUsing the extension in Python:")
 print("""
import mymodule

result = mymodule.add(5, 3)
print(f"Result: {result}") # Result: 8
""")

Run demonstrations
try:
ctypes_basic_example()
performance_comparison()
passing_complex_data()
callbacks_example()
cffi_introduction()
c_extension_example()
except Exception as e:
print(f"Error in C extension examples: {e}")

```

## 12. Profiling, Optimization, and Packaging

### Profiling with cProfile, line\_profiler, and Memory Profiling

Profiling is the process of analyzing your code's performance to identify bottlenecks. Python provides several tools for time and memory profiling.

```
import cProfile
import pstats
import io
import time
import random
import sys
from memory_profiler import profile as memory_profile

Function to profile
def inefficient_function():
 """A deliberately inefficient function for profiling."""
 result = []
 for i in range(1000000):
 result.append(i)
 if i % 10000 == 0:
 result = result[:-10] # Remove some elements periodically
 return len(result)

Another function with different performance characteristics
def string_operations():
 """A function that does a lot of string operations."""
 result = ""
 for i in range(10000):
 result += str(i)
 return len(result)

Function with nested calls
def outer_function():
 """An outer function that calls other functions."""
 time.sleep(0.1) # Simulate some work
 result1 = inner_function1()
 time.sleep(0.1) # More work
 result2 = inner_function2()
 return result1 + result2

def inner_function1():
 """First inner function."""
 time.sleep(0.1)
 return random.randint(1, 100)
```

```

def inner_function2():
 """Second inner function."""
 time.sleep(0.2)
 return random.randint(1, 100)

Using cProfile for basic profiling
def basic_profiling():
 """Demonstrate basic profiling with cProfile."""
 print("Basic profiling with cProfile:")

 # Profile the function
 cProfile.run('inefficient_function()')

 # More detailed profiling
 profile = cProfile.Profile()
 profile.enable()

 inefficient_function()

 profile.disable()

 # Print sorted stats
 s = io.StringIO()
 ps = pstats.Stats(profile, stream=s).sort_stats('cumulative')
 ps.print_stats(10) # Print top 10 functions
 print(s.getvalue())

Line profiler for detailed line-by-line profiling
def line_profiling():
 """Demonstrate line-by-line profiling."""
 print("\nLine-by-line profiling:")
 print("To use line_profiler:")
 print("1. Install with 'pip install line_profiler'")
 print("2. Add @profile decorator to functions (no import needed)")
 print("3. Run 'kernprof -l -v script.py'")

Example output:
print("\nExample line_profiler output:")
print("""
Line # Hits Time Per Hit % Time Line Contents
=====
 3 @profile
 4 def string_operations():
 5 '''A function that does a lot of string
 6 1 2.0 2.0 0.0 result = ""
 7 10001 16207.0 1.6 15.7 for i in range(10000):

```

```

8 10000 86752.0 8.7 84.2 result += str(i)
9 1 1.0 1.0 0.0 return len(result)
"""

```

```

Memory profiling with memory_profiler

```

```

@memory_profile

```

```

def memory_intensive_function():

```

```

 """A function that consumes a lot of memory."""

```

```

 # Create a large list

```

```

 large_list = [random.random() for _ in range(1000000)]

```

```

 # Create a large dictionary

```

```

 large_dict = {i: random.random() for i in range(100000)}

```

```

 # Create a large string

```

```

 large_string = "x" * 10000000

```

```

 # Print sizes to verify

```

```

 print(f"List size: {sys.getsizeof(large_list)} bytes")

```

```

 print(f"Dict size: {sys.getsizeof(large_dict)} bytes")

```

```

 print(f"String size: {sys.getsizeof(large_string)} bytes")

```

```

 # Return the total approximate size

```

```

 return len(large_list) + len(large_dict) + len(large_string)

```

```

def memory_profiling_demo():

```

```

 """Demonstrate memory profiling."""

```

```

 print("\nMemory profiling:")

```

```

 print("To use memory_profiler:")

```

```

 print("1. Install with 'pip install memory_profiler'")

```

```

 print("2. Add @profile decorator from memory_profiler")

```

```

 print("3. Run 'python -m memory_profiler script.py'")

```

```

 # Example output:

```

```

 print("\nExample memory_profiler output:")

```

```

 print("""

```

```

Line # Mem usage Increment Occurences Line Contents
=====

```

```

16 42.652 MiB 42.652 MiB 1 @profile

```

```

17 def memory_intensive_function():

```

```

18 42.652 MiB 0.000 MiB 1 '''A function that consumes a lot of mem

```

```

19 # Create a large list

```

```

20 120.223 MiB 77.570 MiB 1 large_list = [random.random() for _ in ra

```

```

21

```

```

22 # Create a large dictionary

```

```

23 136.633 MiB 16.410 MiB 1 large_dict = {i: random.random() for i in

```

```

24
25 # Create a large string
26 146.629 MiB 9.996 MiB 1 large_string = "x" * 10000000
27
28 # Print sizes to verify
29 146.637 MiB 0.008 MiB 1 print(f"List size: {sys.getsizeof(large_l
30 146.637 MiB 0.000 MiB 1 print(f"Dict size: {sys.getsizeof(large_d
31 146.637 MiB 0.000 MiB 1 print(f"String size: {sys.getsizeof(large
32
33 # Return the total approximate size
34 146.645 MiB 0.008 MiB 1 return len(large_list) + len(large_dict)
"""

Try running the actual function
try:
 result = memory_intensive_function()
 print(f"Memory intensive function returned: {result}")
except ImportError:
 print("memory_profiler not installed. Install with 'pip install memory_profiler'")
except Exception as e:
 print(f"Error running memory profiler: {e}")

Profiling web applications
def web_app_profiling():
 """Discuss profiling web applications."""
 print("\nProfiling web applications:")
 print("For web applications, you can use:")

 print("\n1. Django Debug Toolbar")
 print(" - Shows SQL queries, templates, signals, etc.")
 print(" - Install with 'pip install django-debug-toolbar'")

 print("\n2. Flask Debug Toolbar")
 print(" - Similar features for Flask applications")
 print(" - Install with 'pip install flask-debugtoolbar'")

 print("\n3. pyinstrument")
 print(" - Statistical profiler with call tree visualization")
 print(" - Install with 'pip install pyinstrument'")
 print(" - Example usage with Flask:")
 print("""
from pyinstrument import Profiler

@app.route('/api/endpoint')
def api_endpoint():
 profiler = Profiler()

```

```

 profiler.start()

 # Your view code here
 result = process_request()

 profiler.stop()
 return profiler.output_html()
 """

 print("\n4. py-spy")
 print(" - Sampling profiler that doesn't require code changes")
 print(" - Can attach to running processes")
 print(" - Install with 'pip install py-spy'")
 print(" - Example: 'py-spy record -o profile.svg -- python app.py'")

Using timeit for simple benchmarks
def timeit_benchmarks():
 """Demonstrate using timeit for simple benchmarks."""
 import timeit

 print("\nSimple benchmarks with timeit:")

 # Compare list creation methods
 list_comp_time = timeit.timeit('[i for i in range(1000)]', number=10000)
 list_func_time = timeit.timeit('list(range(1000))', number=10000)

 print(f"List comprehension: {list_comp_time:.6f} seconds")
 print(f"List function: {list_func_time:.6f} seconds")
 print(f"Ratio: {list_comp_time / list_func_time:.2f}x")

 # Compare string concatenation methods
 concat_time = timeit.timeit('''
s = ""
for i in range(100):
 s += str(i)
''', number=10000)

 join_time = timeit.timeit('''
parts = []
for i in range(100):
 parts.append(str(i))
s = ''.join(parts)
''', number=10000)

 print(f"\nString concatenation: {concat_time:.6f} seconds")
 print(f"String join: {join_time:.6f} seconds")

```

```

print(f"Ratio: {concat_time / join_time:.2f}x")

Time a function with arguments
def test_function(n):
 return sum(i * i for i in range(n))

Create a partial function with fixed arguments
import functools
test_with_arg = functools.partial(test_function, 1000)

Time the function
func_time = timeit.timeit(test_with_arg, number=1000)
print(f"\nFunction execution time: {func_time:.6f} seconds")

Profiling best practices
def profiling_best_practices():
 """Discuss profiling best practices."""
 print("\nProfiling best practices:")
 print("1. Profile before optimizing - identify the real bottlenecks")
 print("2. Focus on the functions that consume the most time or memory")
 print("3. Look for unexpected function calls or loops that might indicate inefficiency")
 print("4. Watch for Python's built-in functions in profiling output")
 print("5. Compare different algorithms and data structures for critical sections")
 print("6. Profile with realistic data and workloads")
 print("7. Consider both time and memory usage")
 print("8. Establish performance baselines before making changes")
 print("9. Profile in production-like environments")
 print("10. Use multiple profiling tools for different insights")

Run the profiling demonstrations
basic_profiling()
line_profiling()
memory_profiling_demo()
web_app_profiling()
timeit_benchmarks()
profiling_best_practices()

```

## Identifying and Fixing Bottlenecks

Once you've identified bottlenecks through profiling, the next step is to optimize your code. Here are common techniques and patterns.

```

import time
import random
import sys
from collections import defaultdict

```

```

import itertools
import functools

Example 1: Optimizing loop-intensive operations
def slow_sum_of_squares(n):
 """Calculate sum of squares inefficiently."""
 result = 0
 for i in range(n):
 result += i * i
 return result

def fast_sum_of_squares(n):
 """Calculate sum of squares efficiently using a formula."""
 # Formula: $\sum(i^2) = n(n+1)(2n+1)/6$
 return n * (n + 1) * (2 * n + 1) // 6

def optimize_algorithm():
 """Demonstrate algorithmic optimization."""
 print("Algorithmic optimization:")
 n = 10000000

 # Measure slow version
 start = time.time()
 slow_result = slow_sum_of_squares(n)
 slow_time = time.time() - start

 # Measure fast version
 start = time.time()
 fast_result = fast_sum_of_squares(n)
 fast_time = time.time() - start

 print(f"Slow version: {slow_time:.6f} seconds, result: {slow_result}")
 print(f"Fast version: {fast_time:.6f} seconds, result: {fast_result}")
 print(f"Speedup: {slow_time / fast_time:.2f}x")

Example 2: Optimizing data structures
def optimize_data_structures():
 """Demonstrate optimization with appropriate data structures."""
 print("\nData structure optimization:")

 words = ["apple", "banana", "cherry", "date", "elderberry", "fig", "grape"]
 word_length = 7

 # Generate a large list of random words
 large_word_list = random.choices(words, k=1000000)

```



```

Count word occurrences - inefficient way
start = time.time()
word_counts_slow = {}
for word in large_word_list:
 if word in word_counts_slow:
 word_counts_slow[word] += 1
 else:
 word_counts_slow[word] = 1
slow_time = time.time() - start

Count word occurrences - efficient way with Counter
start = time.time()
from collections import Counter
word_counts_fast = Counter(large_word_list)
fast_time = time.time() - start

print(f"Slow version (dict): {slow_time:.6f} seconds")
print(f"Fast version (Counter): {fast_time:.6f} seconds")
print(f"Speedup: {slow_time / fast_time:.2f}x")

Membership testing - list vs set
print("\nMembership testing - list vs set:")

Generate a large list of random numbers
data_list = list(range(1000000))
data_set = set(data_list)

Check for membership in list
start = time.time()
found_in_list = 999999 in data_list
list_time = time.time() - start

Check for membership in set
start = time.time()
found_in_set = 999999 in data_set
set_time = time.time() - start

print(f"List membership testing: {list_time:.6f} seconds")
print(f"Set membership testing: {set_time:.6f} seconds")
print(f"Speedup: {list_time / set_time:.2f}x")

Example 3: String concatenation
def optimize_string_operations():
 """Demonstrate string operation optimization."""
 print("\nString operation optimization:")

```

```

Inefficient string concatenation
start = time.time()
result = ""
for i in range(100000):
 result += str(i)
inefficient_time = time.time() - start
inefficient_length = len(result)

Efficient string concatenation with join
start = time.time()
parts = []
for i in range(100000):
 parts.append(str(i))
result = "".join(parts)
efficient_time = time.time() - start
efficient_length = len(result)

print(f"Inefficient concatenation: {inefficient_time:.6f} seconds, length: {inefficient_length}")
print(f"Efficient concatenation: {efficient_time:.6f} seconds, length: {efficient_length}")
print(f"Speedup: {inefficient_time / efficient_time:.2f}x")

Example 4: Loop optimization
def optimize_loops():
 """Demonstrate loop optimization techniques."""
 print("\nLoop optimization:")

 # Inefficient nested loops
 start = time.time()
 total = 0
 for i in range(500):
 for j in range(500):
 total += i * j
 inefficient_time = time.time() - start

 # More efficient approach with loop hoisting
 start = time.time()
 total = 0
 for i in range(500):
 # Hoist the invariant calculation out of the inner loop
 i_sum = i * sum(range(500))
 total += i_sum
 efficient_time = time.time() - start

 print(f"Inefficient nested loops: {inefficient_time:.6f} seconds")
 print(f"Optimized with loop hoisting: {efficient_time:.6f} seconds")
 print(f"Speedup: {inefficient_time / efficient_time:.2f}x")

```

```

List comprehension vs loop
print("\nList comprehension vs loop:")

Using a loop
start = time.time()
squares = []
for i in range(1000000):
 squares.append(i * i)
loop_time = time.time() - start

Using a list comprehension
start = time.time()
squares = [i * i for i in range(1000000)]
comprehension_time = time.time() - start

print(f"Loop: {loop_time:.6f} seconds")
print(f"List comprehension: {comprehension_time:.6f} seconds")
print(f"Speedup: {loop_time / comprehension_time:.2f}x")

Example 5: Memoization
def fibonacci_slow(n):
 """Calculate Fibonacci number recursively (inefficient)."""
 if n <= 1:
 return n
 return fibonacci_slow(n-1) + fibonacci_slow(n-2)

@functools.lru_cache(maxsize=None)
def fibonacci_fast(n):
 """Calculate Fibonacci number with memoization (efficient)."""
 if n <= 1:
 return n
 return fibonacci_fast(n-1) + fibonacci_fast(n-2)

def demonstrate_memoization():
 """Demonstrate the power of memoization."""
 print("\nMemoization:")

 # Test with a reasonable number for the slow version
 n_slow = 30
 start = time.time()
 result_slow = fibonacci_slow(n_slow)
 slow_time = time.time() - start

 # Test with a larger number for the fast version
 n_fast = 100

```

```

start = time.time()
result_fast = fibonacci_fast(n_fast)
fast_time = time.time() - start

print(f"Slow recursive Fibonacci({n_slow}): {slow_time:.6f} seconds, result: {result_slow}")
print(f"Memoized Fibonacci({n_fast}): {fast_time:.6f} seconds, result: {result_fast}")
if n_slow == n_fast:
 print(f"Speedup: {slow_time / fast_time:.2f}x")
else:
 print(f>Note: Used different n values due to slow recursive version")

Example 6: Generator expressions
def optimize_with_generators():
 """Demonstrate memory optimization with generators."""
 print("\nGenerator expressions:")

 # Using a list comprehension (materializes the entire list)
 start = time.time()
 sum_list = sum([i * i for i in range(10000000)])
 list_time = time.time() - start

 # Using a generator expression (processes values one at a time)
 start = time.time()
 sum_gen = sum(i * i for i in range(10000000))
 gen_time = time.time() - start

 print(f"List comprehension: {list_time:.6f} seconds, result: {sum_list}")
 print(f"Generator expression: {gen_time:.6f} seconds, result: {sum_gen}")
 print(f"Speedup: {list_time / gen_time:.2f}x")

 # Memory usage comparison
 list_comp = [i for i in range(1000000)]
 list_size = sys.getsizeof(list_comp)

 gen_exp = (i for i in range(1000000))
 gen_size = sys.getsizeof(gen_exp)

 print(f"\nMemory usage:")
 print(f"List with 1M elements: {list_size:,} bytes")
 print(f"Generator for 1M elements: {gen_size:,} bytes")
 print(f"Memory saving ratio: {list_size / gen_size:.2f}x")

Example 7: Using built-in functions and libraries
def optimize_with_builtins():
 """Demonstrate optimization using Python's built-in functions and libraries."""
 print("\nUsing built-in functions and libraries:")

```

```

Generate test data
data = [random.randint(1, 100) for _ in range(1000000)]

Manual max/min finding
start = time.time()
max_val = data[0]
min_val = data[0]
for item in data:
 if item > max_val:
 max_val = item
 if item < min_val:
 min_val = item
manual_time = time.time() - start

Using built-in max/min
start = time.time()
max_val_builtin = max(data)
min_val_builtin = min(data)
builtin_time = time.time() - start

print(f"Manual max/min: {manual_time:.6f} seconds")
print(f"Built-in max/min: {builtin_time:.6f} seconds")
print(f"Speedup: {manual_time / builtin_time:.2f}x")

Using NumPy for numerical operations
try:
 import numpy as np

 # Convert to NumPy array
 np_data = np.array(data)

 # Using NumPy for calculations
 start = time.time()
 np_mean = np.mean(np_data)
 np_std = np.std(np_data)
 np_time = time.time() - start

 # Manual calculation
 start = time.time()
 mean = sum(data) / len(data)
 squared_diff = [(x - mean) ** 2 for x in data]
 variance = sum(squared_diff) / len(data)
 std = variance ** 0.5
 manual_time = time.time() - start

```

```

 print(f"\nManual mean/std calculation: {manual_time:.6f} seconds")
 print(f"NumPy mean/std calculation: {np_time:.6f} seconds")
 print(f"Speedup: {manual_time / np_time:.2f}x")
 except ImportError:
 print("\nNumPy not installed. Install with 'pip install numpy'")

Example 8: Avoiding unnecessary computations
def optimize_redundant_work():
 """Demonstrate optimization by avoiding redundant computations."""
 print("\nAvoiding redundant work:")

 # Inefficient function with redundant work
 def process_inefficient(data):
 results = []
 for i, value in enumerate(data):
 # Redundant computation of the sum in each iteration
 data_sum = sum(data)
 results.append(value / data_sum)
 return results

 # Efficient function that avoids redundant work
 def process_efficient(data):
 # Compute the sum once
 data_sum = sum(data)
 # Use the precomputed sum
 return [value / data_sum for value in data]

 # Generate test data
 data = [random.random() for _ in range(100000)]

 # Measure inefficient version
 start = time.time()
 results_inefficient = process_inefficient(data)
 inefficient_time = time.time() - start

 # Measure efficient version
 start = time.time()
 results_efficient = process_efficient(data)
 efficient_time = time.time() - start

 print(f"Inefficient with redundant work: {inefficient_time:.6f} seconds")
 print(f"Efficient without redundant work: {efficient_time:.6f} seconds")
 print(f"Speedup: {inefficient_time / efficient_time:.2f}x")

Compile optimization tips
def optimization_tips():

```

```

"""Provide a compilation of optimization tips."""
print("\nGeneral optimization tips:")
print("1. Use appropriate data structures (list, set, dict) based on the operations you")
print("2. Avoid unnecessary computations by hoisting invariant code out of loops")
print("3. Use list comprehensions instead of building lists with for loops")
print("4. Use generator expressions for large datasets that don't need to be materialized")
print("5. Leverage memoization (@functools.lru_cache) for expensive recursive functions")
print("6. Use built-in functions and libraries (NumPy, pandas) for numerical operations")
print("7. Use the built-in functions like map, filter, and sorted with key functions")
print("8. Minimize attribute lookups in tight loops by assigning to local variables")
print("9. Use 'join' instead of '+' for string concatenation")
print("10. Consider using PyPy for CPU-bound applications")
print("11. Move CPU-intensive code to C extensions for critical sections")
print("12. Use multiprocessing for CPU-bound tasks and threading/asyncio for I/O-bound tasks")
print("13. Optimize database queries by minimizing round trips and using indexing")
print("14. Batch operations when possible (e.g., batch database inserts)")
print("15. Use __slots__ to reduce memory footprint of classes with many instances")

Run the optimization examples
optimize_algorithm()
optimize_data_structures()
optimize_string_operations()
optimize_loops()
demonstrate_memoization()
optimize_with_generators()
optimize_with_builtins()
optimize_redundant_work()
optimization_tips()

```

## Packaging and Distributing Modules

Python provides tools for packaging and distributing your code as modules or applications, making it easy to share your work with others.

```

import os
import sys

Example setup.py file
def setup_py_example():
 """Provide an example setup.py file."""
 print("Example setup.py file:")
 print("")

from setuptools import setup, find_packages

Read the long description from README.md
with open('README.md', 'r', encoding='utf-8') as f:

```

```

 long_description = f.read()

setup(
 name='mypackage',
 version='0.1.0',
 author='Your Name',
 author_email='your.email@example.com',
 description='A short description of your package',
 long_description=long_description,
 long_description_content_type='text/markdown',
 url='https://github.com/yourusername/mypackage',
 packages=find_packages(),
 classifiers=[
 'Programming Language :: Python :: 3',
 'License :: OSI Approved :: MIT License',
 'Operating System :: OS Independent',
],
 python_requires='>=3.6',
 install_requires=[
 'requests>=2.25.0',
 'numpy>=1.19.0',
],
 extras_require={
 'dev': [
 'pytest>=6.0.0',
 'flake8>=3.8.0',
 'black>=20.8b1',
],
 'docs': [
 'sphinx>=3.4.0',
 'sphinx-rtd-theme>=0.5.0',
],
 },
 entry_points={
 'console_scripts': [
 'mycommand=mypackage.cli:main',
],
 },
 include_package_data=True,
 package_data={
 'mypackage': ['data/*.json'],
 },
)
"""
print("\nThis setup.py file defines:")
print("- Basic metadata (name, version, author, description)")

```



```

 print("- Package discovery with find_packages()")
 print("- Classifiers for indexing on PyPI")
 print("- Required Python version")
 print("- Dependencies (required and optional)")
 print("- Command-line entry points")
 print("- Package data files to include")

Package structure example
def package_structure():
 """Describe a typical Python package structure."""
 print("\nTypical Python package structure:")
 print("""
mypackage/
 LICENSE
 README.md
 pyproject.toml
 setup.py
 setup.cfg
 MANIFEST.in
 requirements.txt
 mypackage/
 __init__.py
 module1.py
 module2.py
 data/
 config.json
 subpackage/
 __init__.py
 submodule.py
 tests/
 __init__.py
 test_module1.py
 test_module2.py
 docs/
 conf.py
 index.rst
""")
 print("\nKey files:")
 print("- LICENSE: Your project's license (e.g., MIT, Apache)")
 print("- README.md: Project description, installation and usage instructions")
 print("- pyproject.toml: PEP 517/518 build system specification")
 print("- setup.py: Installation script")
 print("- setup.cfg: Configuration for setup.py")
 print("- MANIFEST.in: Additional files to include in distributions")
 print("- requirements.txt: Dependencies for development or installation")
 print("- mypackage/__init__.py: Makes the directory a package")

```

```

 print("- tests/: Unit tests for your package")
 print("- docs/: Documentation files")

pyproject.toml example
def pyproject_toml_example():
 """Provide an example pyproject.toml file."""
 print("\nExample pyproject.toml file:")
 print("""
[build-system]
requires = ["setuptools>=42", "wheel"]
build-backend = "setuptools.build_meta"

[tool.black]
line-length = 88
target-version = ['py36', 'py37', 'py38']
include = '\.pyi?$'
exclude = ''
/(
 \\.git
 | \\.hg
 | \\.mypy_cache
 | \\.tox
 | \\.venv
 | _build
 | buck-out
 | build
 | dist
)/
'''

[tool.isort]
profile = "black"
multi_line_output = 3

[tool.pytest.ini_options]
minversion = "6.0"
addopts = "--cov=mypackage"
testpaths = ["tests"]
""")
 print("\nThis pyproject.toml file defines:")
 print("- Build system requirements (PEP 517/518)")
 print("- Configuration for Black code formatter")
 print("- Configuration for isort import sorter")
 print("- Configuration for pytest")

Building and distributing

```

```

def building_and_distributing():
 """Describe the process of building and distributing packages."""
 print("\nBuilding and distributing your package:")

 print("\n1. Create source distribution (sdist):")
 print(" python setup.py sdist")
 print(" or: python -m build --sdist")

 print("\n2. Create wheel distribution (bdist_wheel):")
 print(" python setup.py bdist_wheel")
 print(" or: python -m build --wheel")

 print("\n3. Build both sdist and wheel:")
 print(" python -m build")

 print("\n4. Upload to PyPI test server:")
 print(" twine upload --repository-url https://test.pypi.org/legacy/ dist/*")

 print("\n5. Upload to PyPI:")
 print(" twine upload dist/*")

 print("\nFor automated builds and uploads, consider using GitHub Actions or other CI/CD")

Entry points and console scripts
def entry_points_example():
 """Explain entry points and console scripts."""
 print("\nEntry points and console scripts:")

 print("\nIn setup.py, you can define entry points:")
 print("""
entry_points={
 'console_scripts': [
 'mycommand=mypackage.cli:main',
],
 'gui_scripts': [
 'mygui=mypackage.gui:main',
],
 'mypackage.plugins': [
 'plugin1=mypackage.plugins:Plugin1',
 'plugin2=mypackage.plugins:Plugin2',
],
}
""")

 print("\nExample CLI module (mypackage/cli.py):")
 print("""

```

```

def main():
 \"\"\"Main entry point for the command line.\"\"\"
 import argparse

 parser = argparse.ArgumentParser(description="My package CLI")
 parser.add_argument('--version', action='store_true', help='Print version')
 parser.add_argument('input', help='Input file', nargs='?')

 args = parser.parse_args()

 if args.version:
 from mypackage import __version__
 print(f"mypackage version {__version__}")
 return

 if args.input:
 print(f"Processing {args.input}")
 else:
 parser.print_help()

if __name__ == "__main__":
 main()
"""

 print("\nAfter installation, users can run your script from the command line:")
 print(" $ mycommand --version")
 print(" $ mycommand input.txt")

Package versioning
def package_versioning():
 """Explain package versioning strategies."""
 print("\nPackage versioning strategies:")

 print("\n1. Single-sourced versioning:")
 print(" Define version in one place and import it elsewhere")

 print("\nExample in mypackage/__init__.py:")
 print("""
__version__ = '0.1.0'
""")

 print("\nExample in setup.py:")
 print("""
import re

Read version from __init__.py

```

```

with open('mypackage/__init__.py', 'r') as f:
 version_match = re.search(r"^__version__ = ['\"]([^\"]*)['\"]", f.read(), re.M)
 if version_match:
 version = version_match.group(1)
 else:
 raise RuntimeError("Unable to find version string")

setup(
 name='mypackage',
 version=version,
 # ...
)
"""

 print("\n2. Using a separate version file:")
 print(" Version defined in a separate file (e.g., _version.py)")

 print("\nExample in mypackage/_version.py:")
 print("""
__version__ = '0.1.0'
""")

 print("\nExample in mypackage/__init__.py:")
 print("""
from ._version import __version__
""")

 print("\n3. Using setuptools_scm:")
 print(" Automatically derive version from git tags")

 print("\nExample in pyproject.toml:")
 print("""
[build-system]
requires = ["setuptools>=42", "wheel", "setuptools_scm[toml]>=6.0"]
build-backend = "setuptools.build_meta"

[tool.setuptools_scm]
write_to = "mypackage/_version.py"
""")

Documentation
def package_documentation():
 """Explain documentation approaches for Python packages."""
 print("\nDocumenting your package:")

 print("\n1. Docstrings in your code:")

```

```

 print(" Use Google, NumPy, or reStructuredText format")

 print("\nExample of Google-style docstring:")
 print("""
def process_data(data, config=None):
 \"\"\"Process the input data using the provided configuration.

 Args:
 data (list): The data to process.
 config (dict, optional): Configuration parameters. Defaults to None.

 Returns:
 dict: The processed data.

 Raises:
 ValueError: If the data is invalid.

 Examples:
 >>> process_data([1, 2, 3])
 {'result': [2, 4, 6]}
 \"\"\"
 """)

 print("\n2. Generating documentation with Sphinx:")
 print(" - Install with 'pip install sphinx sphinx-rtd-theme'")
 print(" - Initialize with 'sphinx-quickstart'")
 print(" - Use sphinx-apidoc to auto-generate API docs from docstrings")
 print(" - Build with 'make html'")

 print("\n3. Read the Docs integration:")
 print(" - Create .readthedocs.yml in your repository")
 print(" - Link your GitHub repository to Read the Docs")

 print("\nExample .readthedocs.yml:")
 print("""
version: 2

build:
 os: ubuntu-20.04
 tools:
 python: "3.8"

sphinx:
 configuration: docs/conf.py

python:

```

```

install:
 - method: pip
 path: .
 extra_requirements:
 - docs
"""

Testing your package
def package_testing():
 """Explain testing approaches for Python packages."""
 print("\nTesting your package:")

 print("\n1. Using pytest:")
 print(" - Install with 'pip install pytest pytest-cov'")
 print(" - Create tests in a 'tests' directory")
 print(" - Run with 'pytest'")

 print("\nExample test file (tests/test_module1.py):")
 print("""
import pytest
from mypackage.module1 import add, subtract

def test_add():
 assert add(2, 3) == 5
 assert add(-1, 1) == 0
 assert add(0, 0) == 0

def test_subtract():
 assert subtract(5, 3) == 2
 assert subtract(1, 1) == 0
 assert subtract(0, 5) == -5

@pytest.mark.parametrize("a,b,expected", [
 (1, 2, 3),
 (0, 0, 0),
 (-1, -1, -2),
])
def test_add_parametrized(a, b, expected):
 assert add(a, b) == expected
""")

 print("\n2. Setting up continuous integration with GitHub Actions:")

 print("\nExample .github/workflows/tests.yml:")
 print("""
name: Tests

```

```

on:
 push:
 branches: [main]
 pull_request:
 branches: [main]

jobs:
 test:
 runs-on: ubuntu-latest
 strategy:
 matrix:
 python-version: [3.7, 3.8, 3.9]

 steps:
 - uses: actions/checkout@v2
 - name: Set up Python ${ matrix.python-version }
 uses: actions/setup-python@v2
 with:
 python-version: ${ matrix.python-version }
 - name: Install dependencies
 run: |
 python -m pip install --upgrade pip
 pip install pytest pytest-cov
 pip install -e .[dev]
 - name: Test with pytest
 run: |
 pytest --cov=mypackage tests/
"""

```

*# Packaging best practices*

```

def packaging_best_practices():
 """Provide best practices for Python packaging."""
 print("\nPackaging best practices:")

 print("\n1. Package organization:")
 print(" - Use a clear, logical directory structure")
 print(" - Separate concerns into modules and subpackages")
 print(" - Use __init__.py to control public API")

 print("\n2. Dependencies:")
 print(" - Specify minimum versions but avoid overly restrictive upper bounds")
 print(" - Use environment markers for platform-specific dependencies")
 print(" - Separate optional dependencies with extras_require")

 print("\n3. Building and distributing:")

```



```

print(" - Use pyproject.toml for build system specification")
print(" - Always include both sdist and wheel distributions")
print(" - Use twine for secure uploads to PyPI")

print("\n4. Versioning:")
print(" - Follow semantic versioning (MAJOR.MINOR.PATCH)")
print(" - Consider using setuptools_scm for Git-based versioning")

print("\n5. Documentation:")
print(" - Write comprehensive docstrings")
print(" - Include a detailed README.md")
print(" - Consider generating API docs with Sphinx")

print("\n6. Testing:")
print(" - Write unit tests for all functionality")
print(" - Set up CI/CD for automated testing")
print(" - Track code coverage")

print("\n7. Metadata:")
print(" - Include proper classifiers")
print(" - Specify license correctly")
print(" - Include a clear description and long_description")

Example of modern Python packaging with pyproject.toml
def modern_packaging_example():
 """Show a complete modern packaging example using pyproject.toml."""
 print("\nModern packaging example (PEP 621):")

 print("\npyproject.toml:")
 print("""
[build-system]
requires = ["setuptools>=61.0", "wheel"]
build-backend = "setuptools.build_meta"

[project]
name = "mypackage"
version = "0.1.0"
description = "A sample Python package"
readme = "README.md"
authors = [
 {name = "Your Name", email = "your.email@example.com"},
]
license = {text = "MIT"}
classifiers = [
 "Programming Language :: Python :: 3",
 "License :: OSI Approved :: MIT License",

```

```

 "Operating System :: OS Independent",
]
requires-python = ">=3.7"
dependencies = [
 "requests>=2.25.0",
 "numpy>=1.19.0",
]

[project.optional-dependencies]
dev = [
 "pytest>=6.0.0",
 "pytest-cov>=2.12.0",
 "flake8>=3.9.0",
 "black>=21.5b0",
]
docs = [
 "sphinx>=4.0.0",
 "sphinx-rtd-theme>=0.5.0",
]

[project.urls]
Homepage = "https://github.com/yourusername/mypackage"
Documentation = "https://mypackage.readthedocs.io/"
Issues = "https://github.com/yourusername/mypackage/issues"

[project.scripts]
mycommand = "mypackage.cli:main"

[tool.setuptools]
packages = ["mypackage"]

[tool.setuptools.package-data]
mypackage = ["data/*.json"]
""")

print("\nThis modern approach:")
print("- Uses PEP 621 metadata specification")
print("- Centralizes configuration in pyproject.toml")
print("- Standardizes project metadata")
print("- Simplifies dependency specification")
print("- Works with the latest Python packaging tools")

Run the packaging examples
setup_py_example()
package_structure()
pyproject_toml_example()

```

```

building_and_distributing()
entry_points_example()
package_versioning()
package_documentation()
package_testing()
packaging_best_practices()
modern_packaging_example()

```

## 13. New Python Features

### Python 3.10 to 3.13 Features

Python continues to evolve with new releases introducing powerful features. Here's an overview of key features introduced in recent Python versions.

*# Python 3.10 Features (October 2021)*

```

def python_310_features():
 """Demonstrate Python 3.10 features."""
 print("Python 3.10 Features:")

 print("\n1. Structural Pattern Matching (PEP 634):")
 print(" The match statement provides a powerful pattern matching mechanism")

 def match_example(value):
 match value:
 case []:
 return "Empty list"
 case [a]:
 return f"List with one element: {a}"
 case [a, b]:
 return f"List with two elements: {a} and {b}"
 case [a, b, *rest]:
 return f"List with {len(rest) + 2} elements, starting with {a} and {b}"
 case {"name": name, "age": age}:
 return f"Person named {name}, age {age}"
 case {"error": msg}:
 return f"Error: {msg}"
 case str() as s if len(s) > 5:
 return f"Long string: {s}"
 case str() as s:
 return f"Short string: {s}"
 case _:
 return "Something else"

 examples = [
 [],
 [1],

```

```

 [1, 2],
 [1, 2, 3, 4],
 {"name": "Alice", "age": 30},
 {"error": "Not found"},
 "Hello, world!",
 "Hi",
 42
]

 for example in examples:
 print(f" match {example!r}: {match_example(example)}")

 print("\n2. Better Error Messages:")
 print(" - More precise syntax error messages")
 print(" - Suggestions for fixing errors")

 print("\n3. Parameter Specification Variables (PEP 612):")
 print(" TypeVar for specifying generic parameters in type annotations")
 print("""
from typing import Callable, ParamSpec, TypeVar

P = ParamSpec('P')
R = TypeVar('R')

def decorator(func: Callable[P, R]) -> Callable[P, R]:
 def wrapper(*args: P.args, **kwargs: P.kwargs) -> R:
 return func(*args, **kwargs)
 return wrapper
""")

 print("\n4. Union Types as X | Y (PEP 604):")
 print(" New syntax for Union types in type annotations")
 print("""
Old way
from typing import Union
def func(x: Union[int, str]) -> Union[float, bool]:
 ...

New way
def func(x: int | str) -> float | bool:
 ...
""")

 print("\n5. Context Managers in parentheses:")
 print(" Multi-line with statements can use parentheses:")
 print("""

```

```

with (
 open('file1.txt') as f1,
 open('file2.txt') as f2
):
 ...
"""

 print("\n6. Better Type Annotations:")
 print(" - Improved TypeAlias")
 print(" - Better error messages from type checkers")
 print(" - Type unions with |")

Python 3.11 Features (October 2022)
def python_311_features():
 """Demonstrate Python 3.11 features."""
 print("\nPython 3.11 Features:")

 print("\n1. Performance Improvements:")
 print(" - Up to 10-60% faster CPython interpreter")
 print(" - Faster startup and runtime")
 print(" - Optimized frame evaluation")

 print("\n2. Better Error Messages:")
 print(" - Shows the exact expression that caused the error")
 print(" - More detailed tracebacks")

 print("\n3. Exception Groups and except* (PEP 654):")
 print(" Allow raising and catching multiple exceptions at once")
 print("""
try:
 # Some code that might raise multiple exceptions
 ...
except* ValueError as e:
 # Handle ValueError instances from the exception group
 ...
except* TypeError as e:
 # Handle TypeError instances from the exception group
 ...
""")

 print("\n4. Task and Exception Groups in asyncio:")
 print(" New API for managing groups of tasks")
 print("""
async def main():
 async with asyncio.TaskGroup() as tg:
 task1 = tg.create_task(some_coro())

```

```

 task2 = tg.create_task(another_coro())
 # All tasks are awaited when exiting the context manager
 """

 print("\n5. Type Annotations Improvements:")
 print(" - Self type for returning the same type as the class")
 print(" - Variadic generics (TypeVarTuple)")
 print(" - Required and NotRequired for TypedDict items")

 print("\n6. TOML Support in Standard Library:")
 print(" - tomllib module for parsing TOML files")
 print("""
import tomllib
with open("config.toml", "rb") as f:
 data = tomllib.load(f)
""")

 print("\n7. New In-memory ZIP archives:")
 print(" Create ZIP archives without using temp files")
 print("""
import zipfile
archive = zipfile.ZipFile(None, mode='w')
archive.writestr('file.txt', 'content')
data = archive.get_in_memory_file_contents()
""")

Python 3.12 Features (October 2023)
def python_312_features():
 """Demonstrate Python 3.12 features."""
 print("\nPython 3.12 Features:")

 print("\n1. Type Parameter Syntax (PEP 695):")
 print(" - New syntax for generic type parameters")
 print(" - More concise type variable declaration")
 print("""
New syntax
def max[T](args: list[T]) -> T:
 ...

New class type parameter syntax
class TreeNode[T]:
 value: T
 left: "TreeNode[T] | None" = None
 right: "TreeNode[T] | None" = None
""")

```

```

print("\n2. Improved f-strings (PEP 701):")
print(" - Allow quotes inside f-strings that match the outer quotes")
print(" - Allow expressions with unbalanced braces")
print("""
name = "Alice"
f"Name: {name = }" # Name: name = 'Alice'
f"Dict: {{"key": "value"}} # Dict: {"key": "value"}
""")

print("\n3. Enhanced Modules:")
print(" - New pathlib.Path.walk() method")
print(" - New `is_closed` for file-like objects")
print(" - Enhanced shutil.copy* methods accepting path-like objects and open file descriptors")

print("\n4. Performance Improvements:")
print(" - Continued interpreter optimizations")
print(" - More C-implemented modules")
print(" - Optimized startup time")

print("\n5. Enhanced Error Messages:")
print(" - Even more detailed error messages")
print(" - Better suggestions for typos in names")

print("\n6. Removal of deprecated features:")
print(" - Removed old raise syntax: 'raise Exception, args'")
print(" - Removed deprecated modules (aifc, audioop, chunk, etc.)")
print(" - Removed deprecated unittest methods")

print("\n7. New Itertools Utilities:")
print(" - itertools.batched() to chunk an iterable into fixed-length batches")
print("""
from itertools import batched
for batch in batched(range(10), 3):
 print(batch) # (0, 1, 2), (3, 4, 5), (6, 7, 8), (9,)
""")

Python 3.13 Features (Expected October 2024)
def python_313_features():
 """Discuss expected Python 3.13 features."""
 print("\nPython 3.13 Features (Expected October 2024):")
 print("Note: These are proposed/expected features and may change")

 print("\n1. Static Typing Improvements:")
 print(" - Further enhancements to type annotations")
 print(" - Better support for generics")
 print(" - More expressive type system")

```

```

print("\n2. Continued Performance Improvements:")
print(" - More optimizations in the CPython interpreter")
print(" - Further development of the JIT compiler (Faster CPython)")
print(" - Memory usage optimizations")

print("\n3. Pattern Matching Improvements:")
print(" - Optimized pattern matching")
print(" - Enhanced features for structural pattern matching")

print("\n4. Standard Library Enhancements:")
print(" - More modules moved to C implementations")
print(" - New utility functions in common modules")
print(" - Enhanced concurrency primitives")

print("\n5. GIL Improvements:")
print(" - Continued work on the 'nogil' fork of CPython")
print(" - Potential GIL-free capabilities")
print(" - Better multi-core CPU utilization")

print("\n6. Enhanced Error Messages:")
print(" - Continuing the trend of more helpful error messages")
print(" - Better debugging information")

print("\n7. More TOML Support:")
print(" - Potential addition of a tomlib writer")
print(" - Expanded TOML capabilities")

Comparison with older Python versions
def python_version_comparison():
 """Compare feature sets across Python versions."""
 print("\nFeature comparison across Python versions:")

 features_table = [
 ("Feature", "3.9", "3.10", "3.11", "3.12", "3.13"),
 ("Dictionary Merging (|)", " ", " ", " ", " ", " "),
 ("Type Annotations Without Imports", " ", " ", " ", " ", " "),
 ("Pattern Matching", " ", " ", " ", " ", " "),
 ("Union Types with |", " ", " ", " ", " ", " "),
 ("Exception Groups", " ", " ", " ", " ", " "),
 ("Self Type", " ", " ", " ", " ", " "),
 ("TOML Parser", " ", " ", " ", " ", " "),
 ("Type Parameter Syntax", " ", " ", " ", " ", " "),
 ("Enhanced f-strings", " ", " ", " ", " ", " "),
 ("batched() in itertools", " ", " ", " ", " ", " "),
 ("JIT Compilation", " ", " ", " ", " ", "Partial")
]

```



```
]
```

```
Print the table
for i, row in enumerate(features_table):
 # Header row
 if i == 0:
 print(" " + " | ".join(cell.ljust(20) for cell in row))
 print(" " + "-" * 120)
 # Data rows
 else:
 print(" " + " | ".join(cell.ljust(20) for cell in row))

Running the examples
python_310_features()
python_311_features()
python_312_features()
python_313_features()
python_version_comparison()
```